

DTS103TC

Data Structure and Algorithms

Lecture 9: Minimum Spanning Tree and Shortest Path

Dr. Qi Chen
School of AI and Advanced Computing

Learning Outcome

- Understand what Minimum Spanning Tree is
 - Able to apply Kruskal's algorithm to find minimum spanning tree
 - Able to apply Prim's algorithm to find minimum spanning tree
- Understand what Single-source shortest path is
 - Able to apply Dijkstra algorithm to solve shortest path problem
 - Able to apply Bellman-Ford algorithm to solve shortest path problem

Minimum Spanning tree (MST)

Given an undirected connected graph G

- The edges are labelled by weight

Spanning tree of G

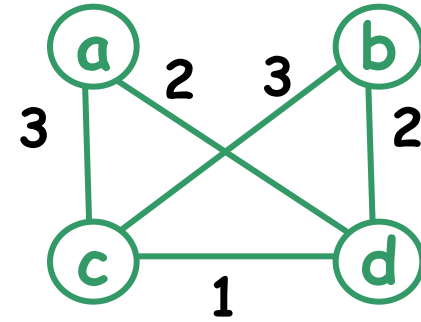
- a tree containing all vertices in G

Minimum spanning tree of G

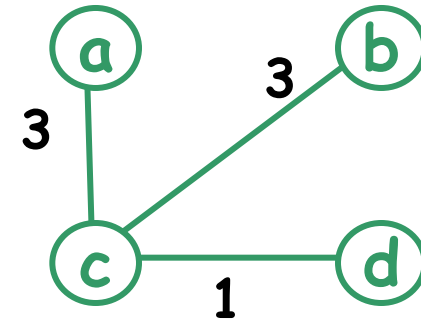
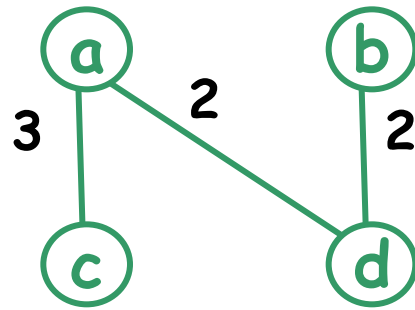
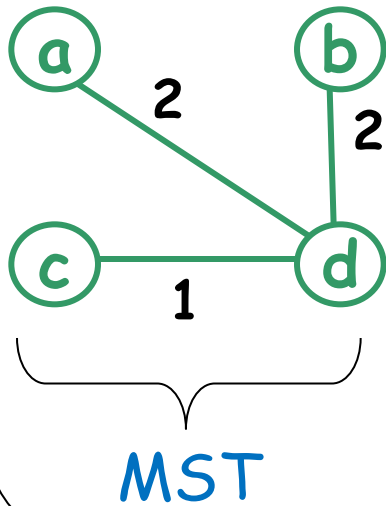
- a spanning tree of G with minimum weight

Examples

Graph G
(edge label is weight)

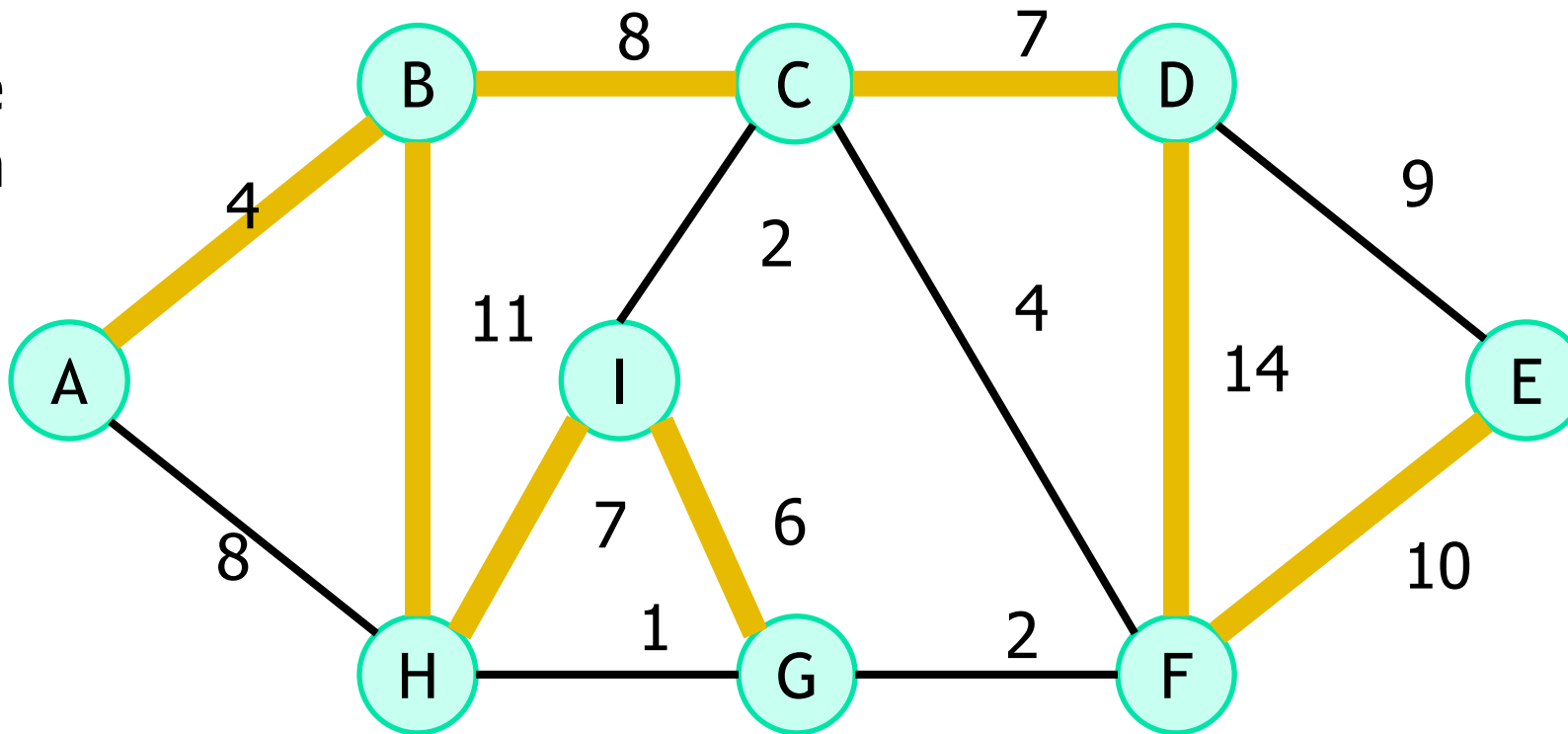


Spanning trees of G



Spanning Tree

The **cost** of a spanning tree is the sum of the weights on the edges.



This is a spanning tree.

It has cost 67

A **tree** is a connected undirected graph with no cycles!

A **spanning tree** is a **tree** that connects all of the vertices.

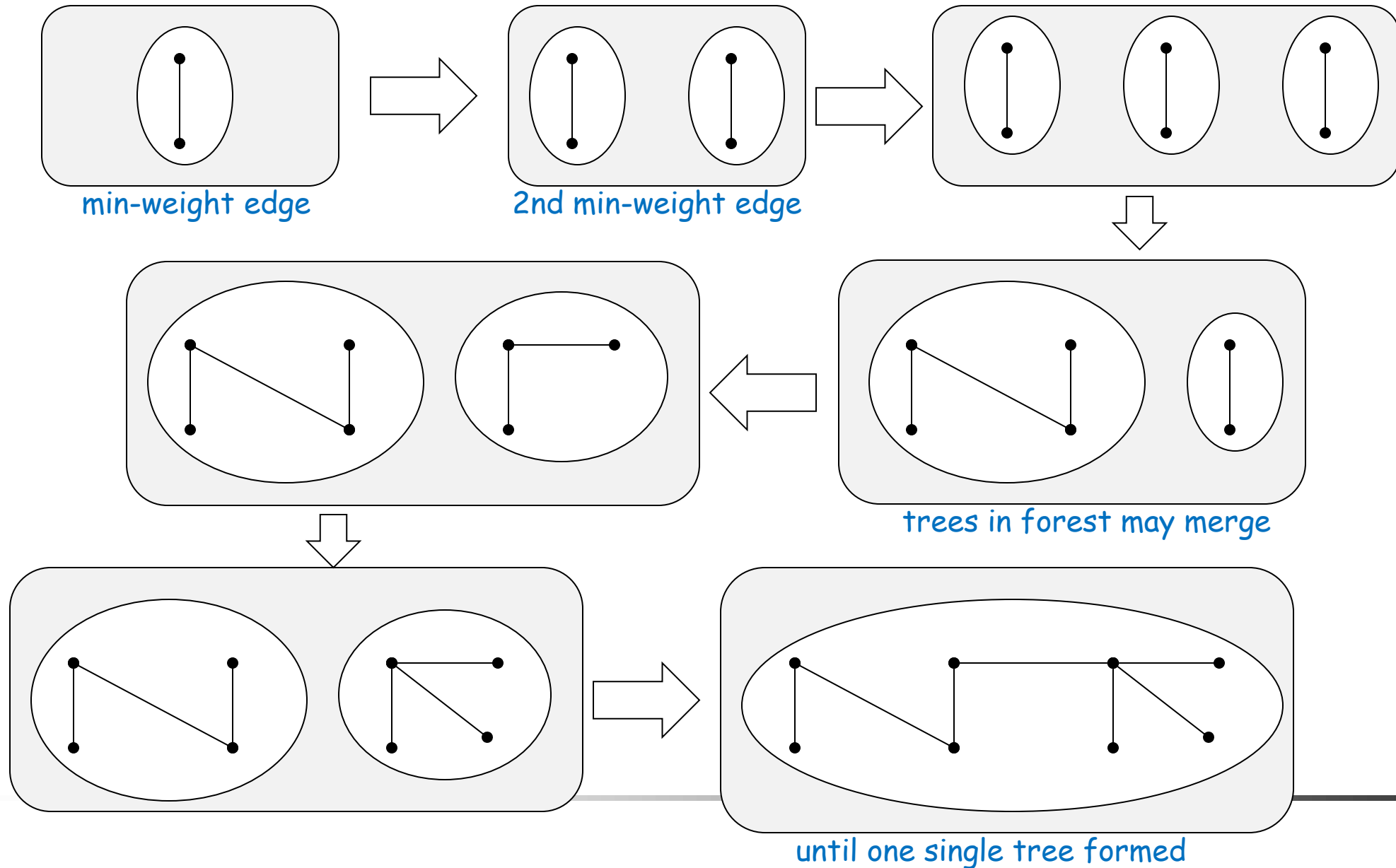
 Xi'an Jiaotong-Liverpool University



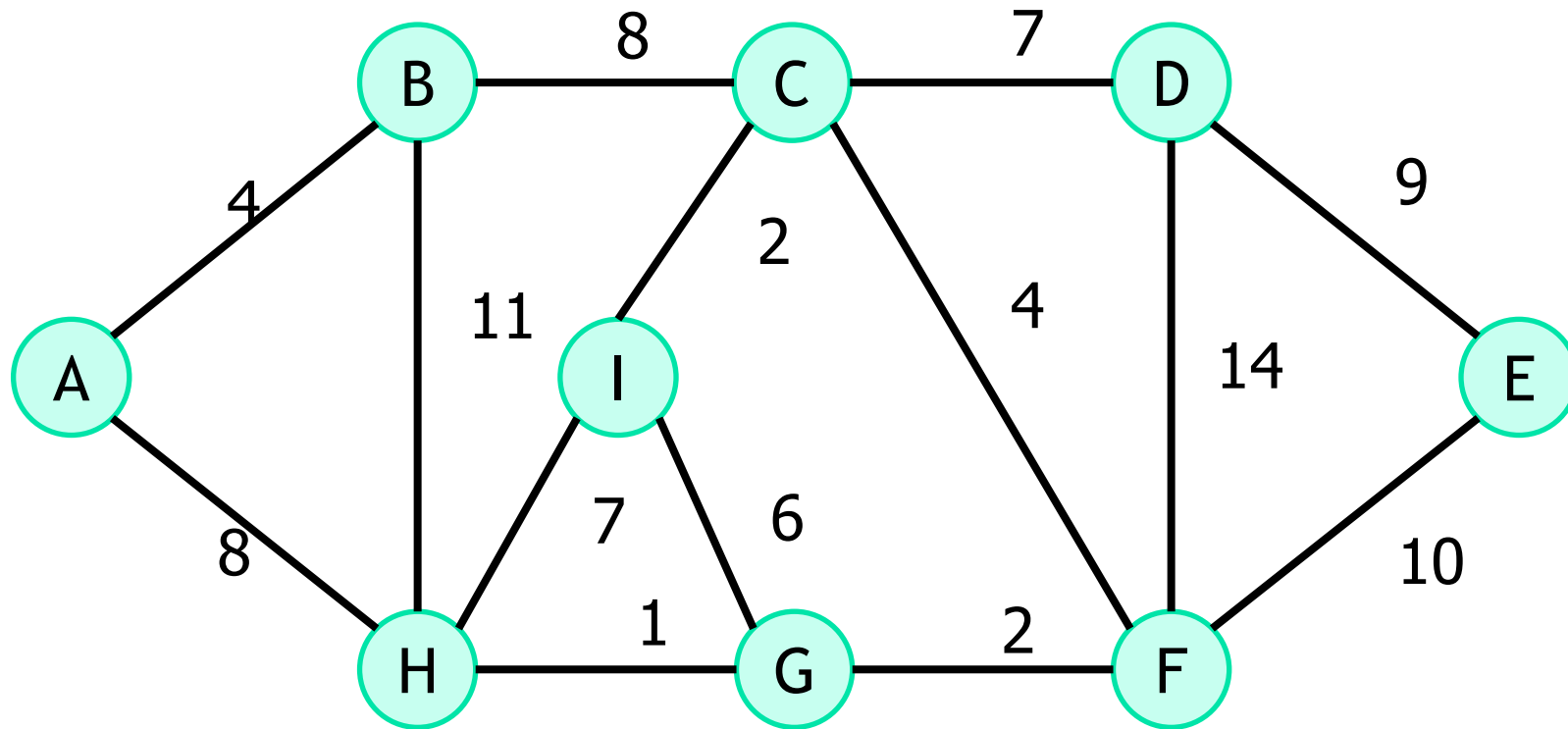
that connects all of the vertices.

Kruskal's algorithm

Idea of Kruskal's algorithm - MST



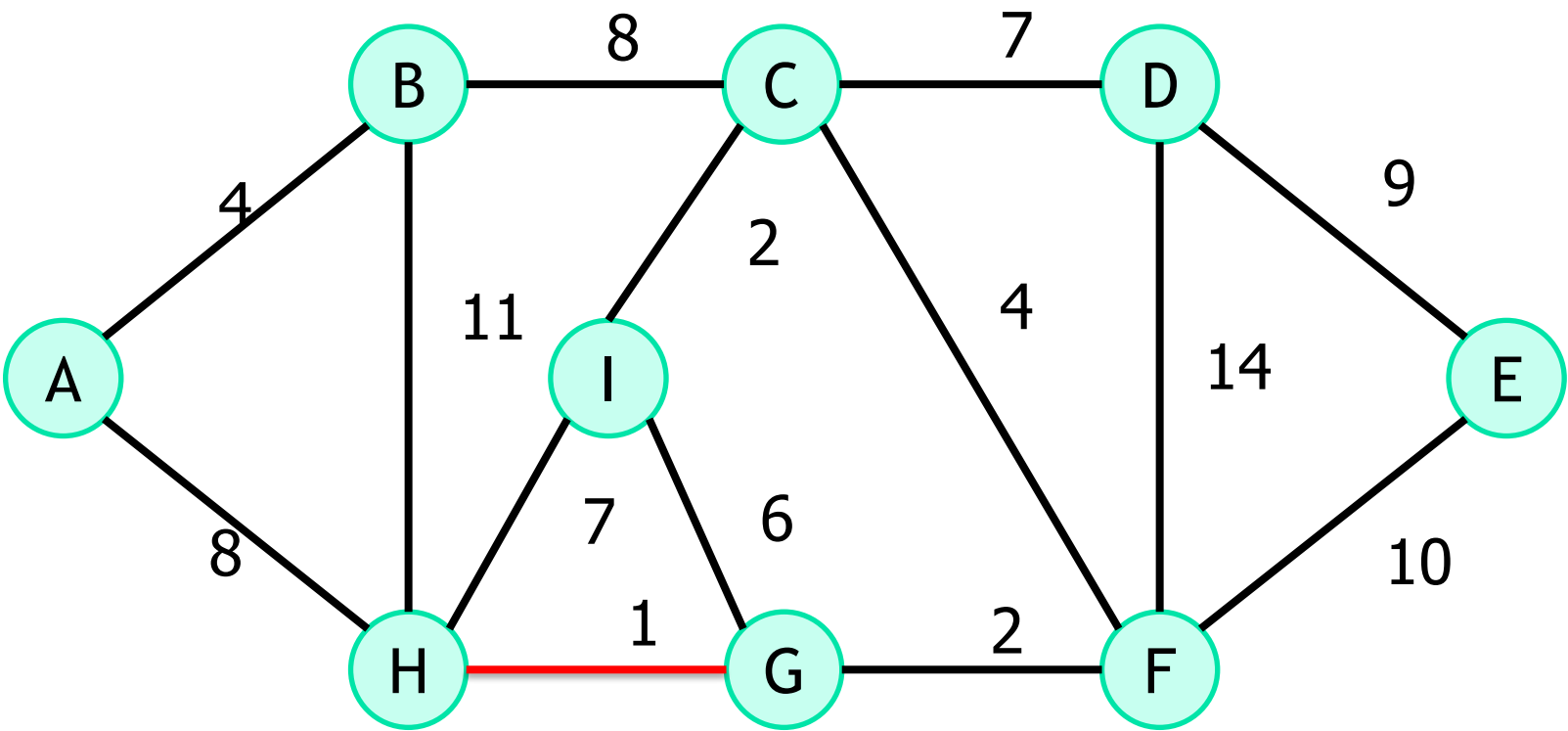
Kruskal's algorithm - MST



(h,g)	1
(i,c)	2
(g,f)	2
(a,b)	4
(c,f)	4
(i,g)	6
(c,d)	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

Arrange edges from smallest to largest weight

Kruskal's algorithm - MST



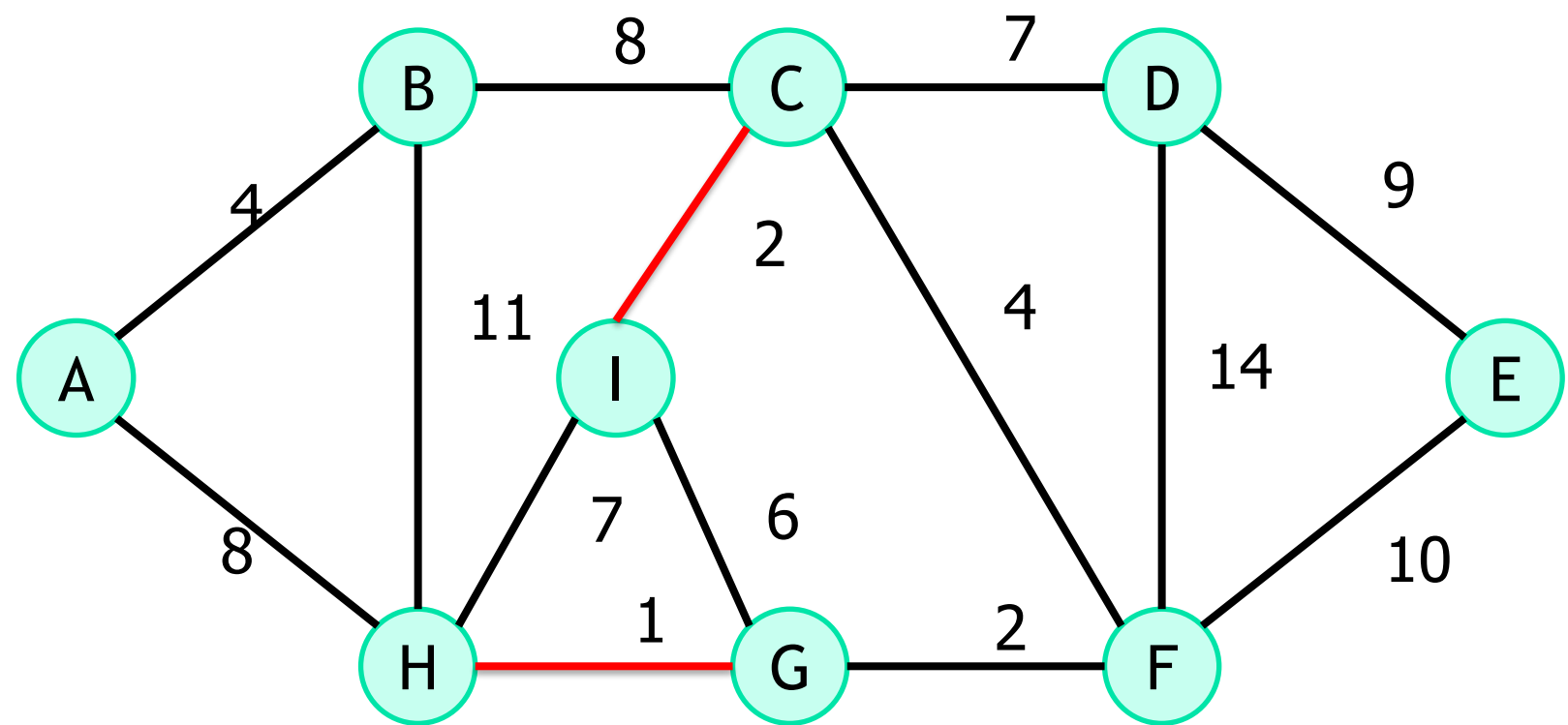
Choose the minimum weight edge

→

<i>(h,g)</i>	<i>1</i>
(i,c)	2
(g,f)	2
(a,b)	4
(c,f)	4
(i,g)	6
(c,d)	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST

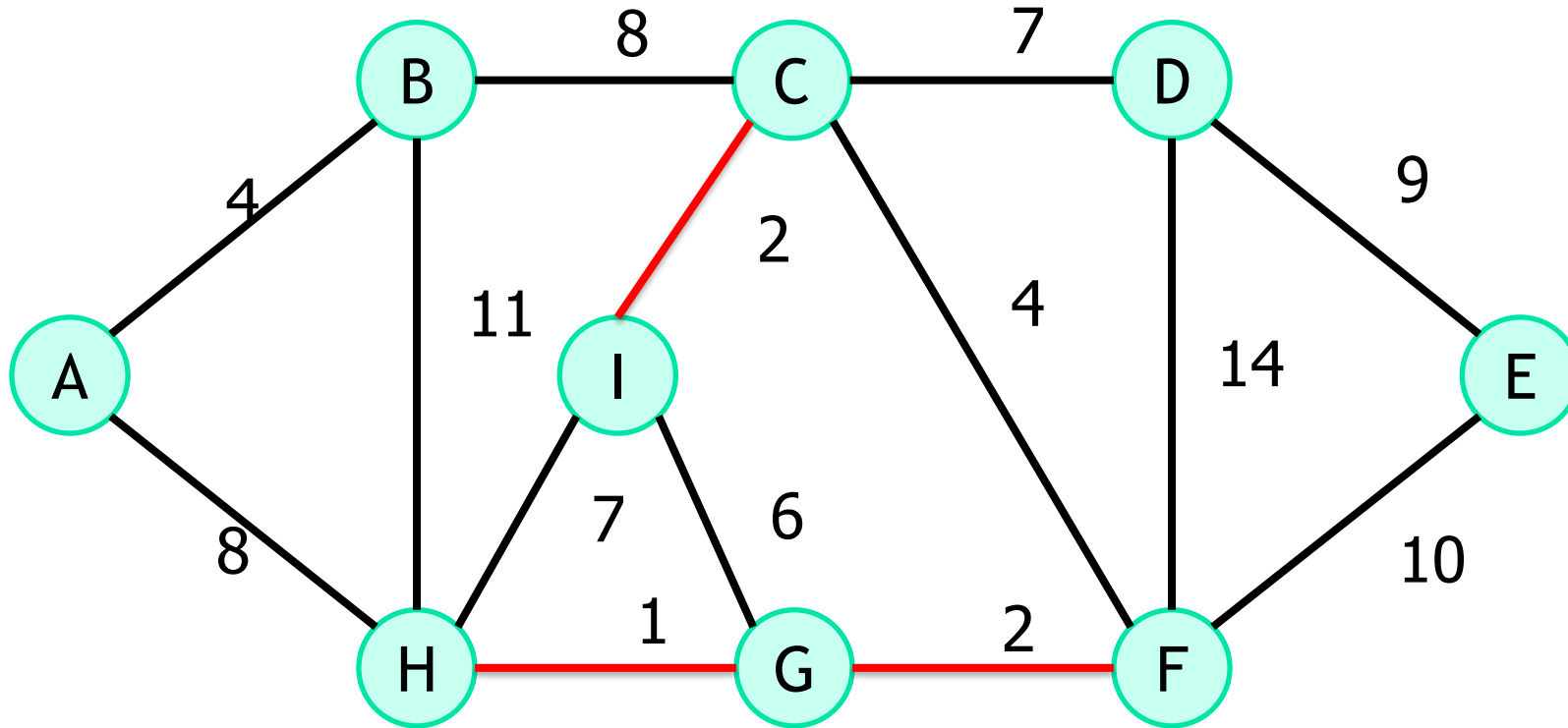


Choose the next minimum weight edge

<i>(h,g)</i>	<i>1</i>
<i>(i,c)</i>	<i>2</i>
(g,f)	2
(a,b)	4
(c,f)	4
(i,g)	6
(c,d)	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



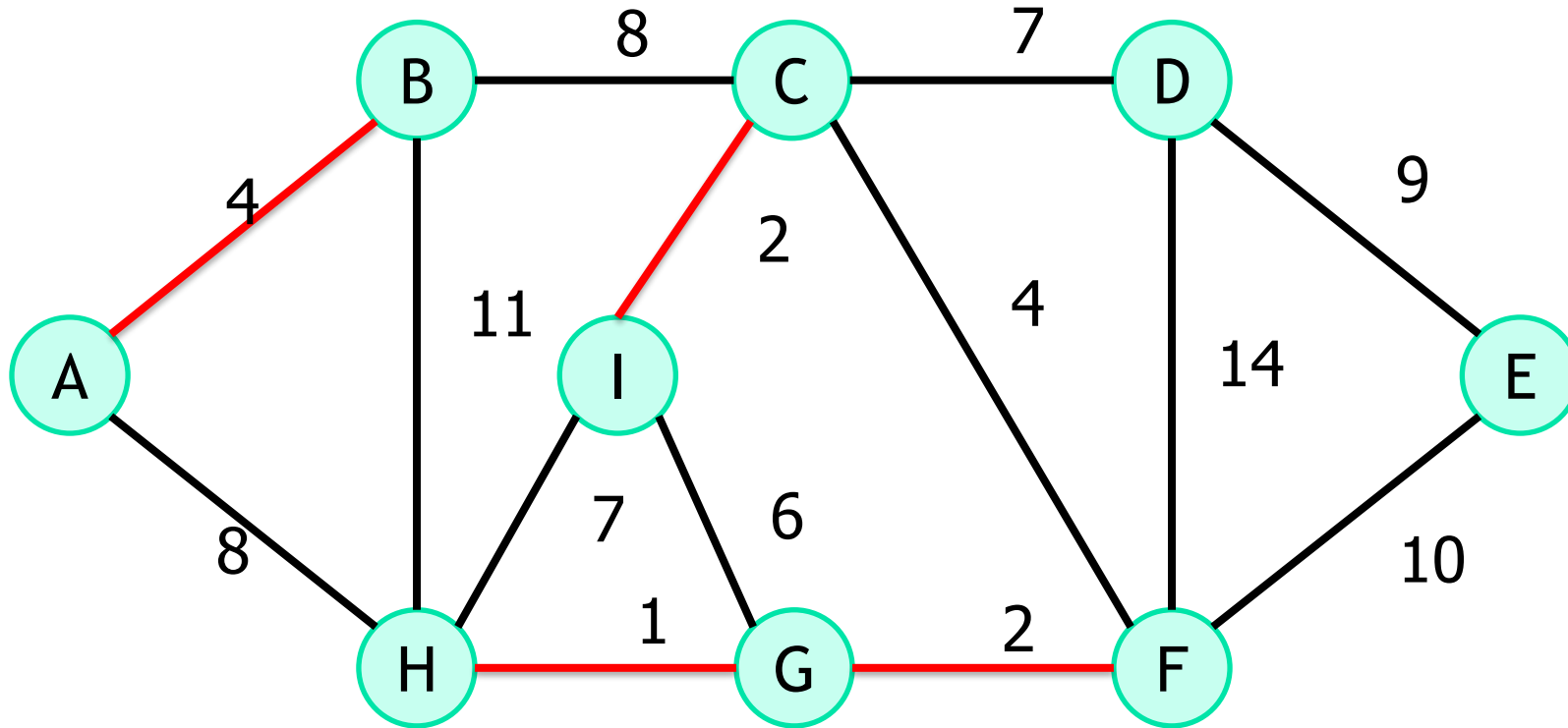
Continue as long as no cycle forms



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
(a,b)	4
(c,f)	4
(i,g)	6
(c,d)	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



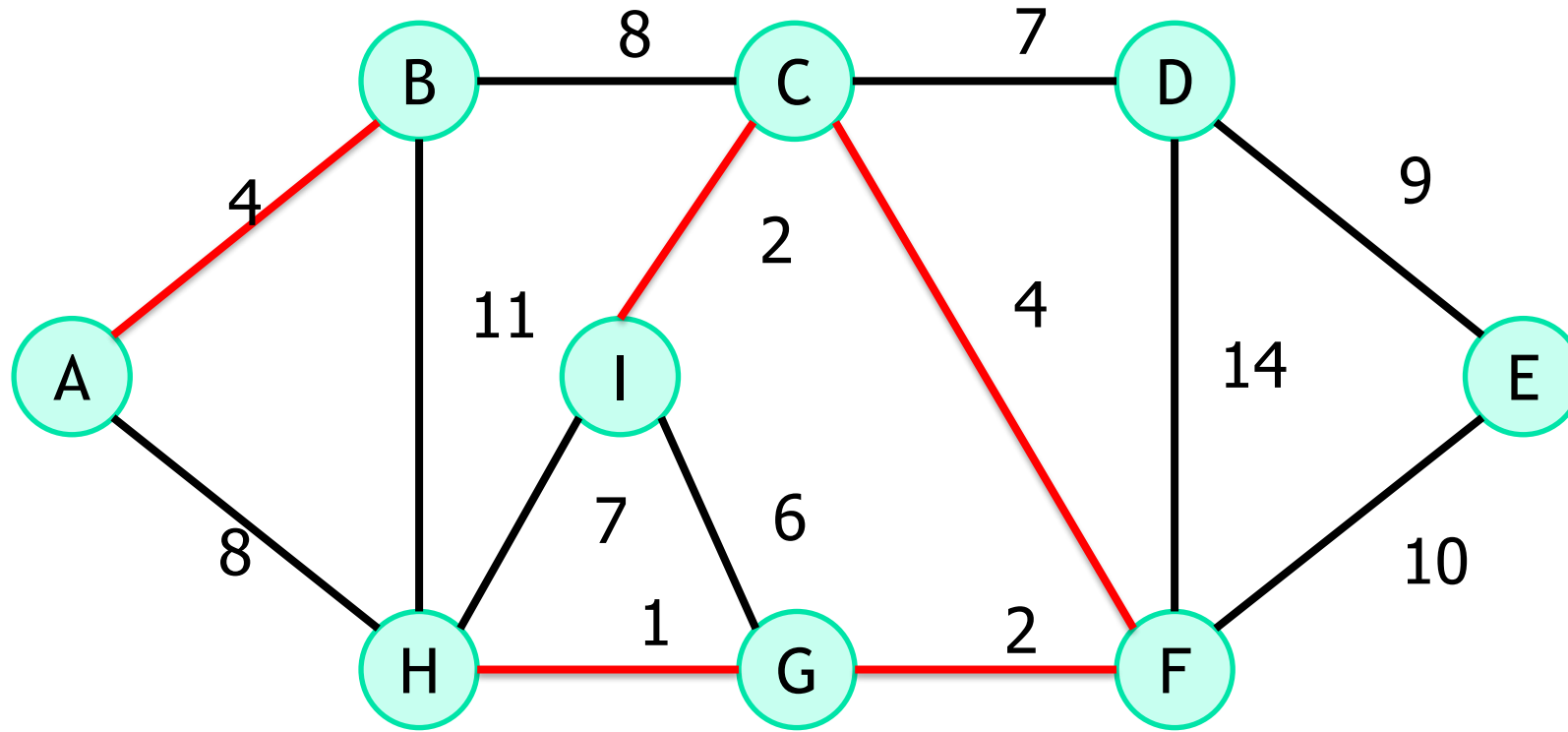
Continue as long as no cycle forms



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
(c,f)	4
(i,g)	6
(c,d)	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



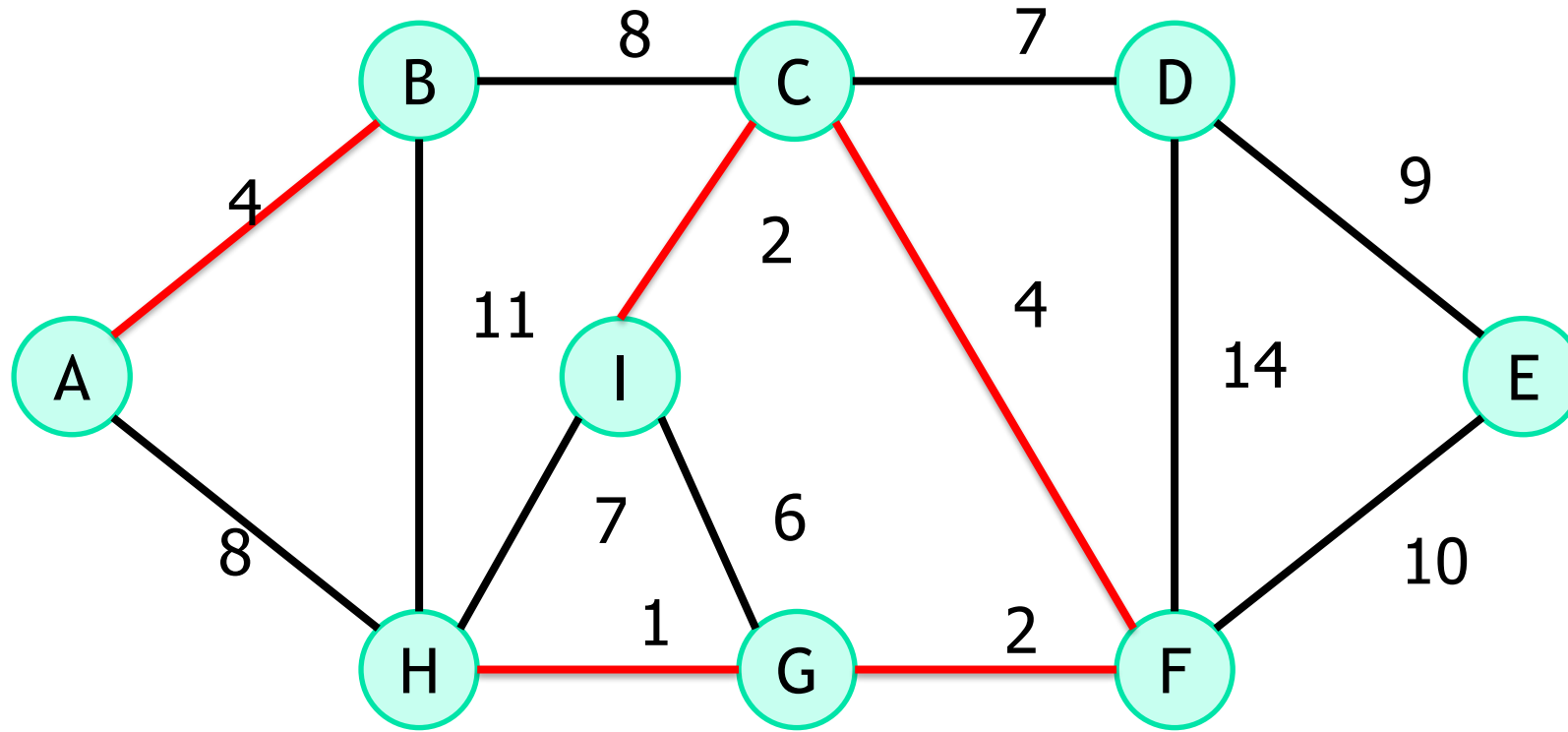
Continue as long as no cycle forms



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
(c,d)	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



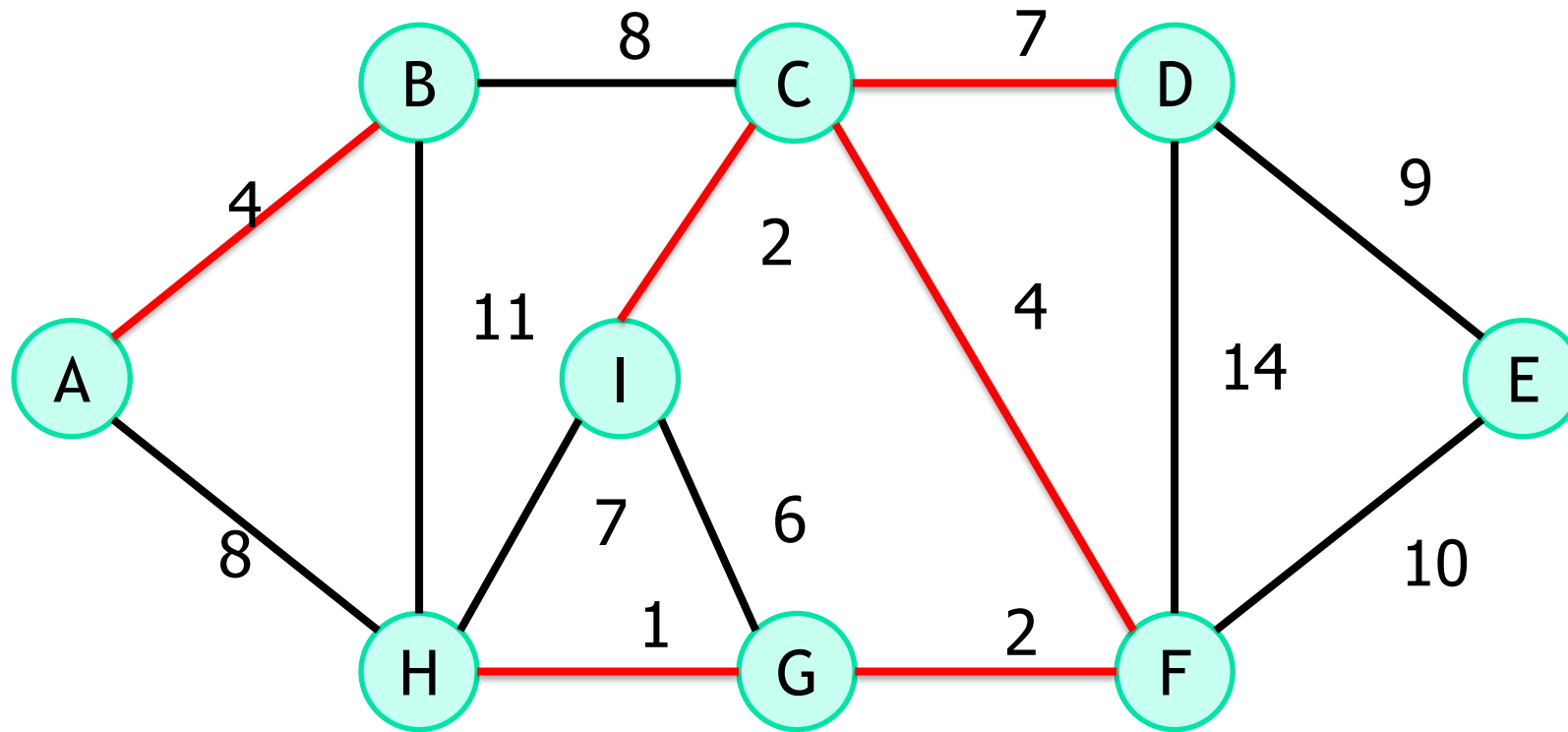
(i,g) cannot be included, otherwise, a cycle is formed



<i>(h,g)</i>	<i>1</i>
<i>(i,c)</i>	<i>2</i>
<i>(g,f)</i>	<i>2</i>
<i>(a,b)</i>	<i>4</i>
<i>(c,f)</i>	<i>4</i>
<i>(i,g)</i>	<i>6</i>
<i>(c,d)</i>	<i>7</i>
<i>(h,i)</i>	<i>7</i>
<i>(b,c)</i>	<i>8</i>
<i>(a,h)</i>	<i>8</i>
<i>(d,e)</i>	<i>9</i>
<i>(f,e)</i>	<i>10</i>
<i>(b,h)</i>	<i>11</i>
<i>(d,f)</i>	<i>14</i>

italic: chosen

Kruskal's algorithm - MST



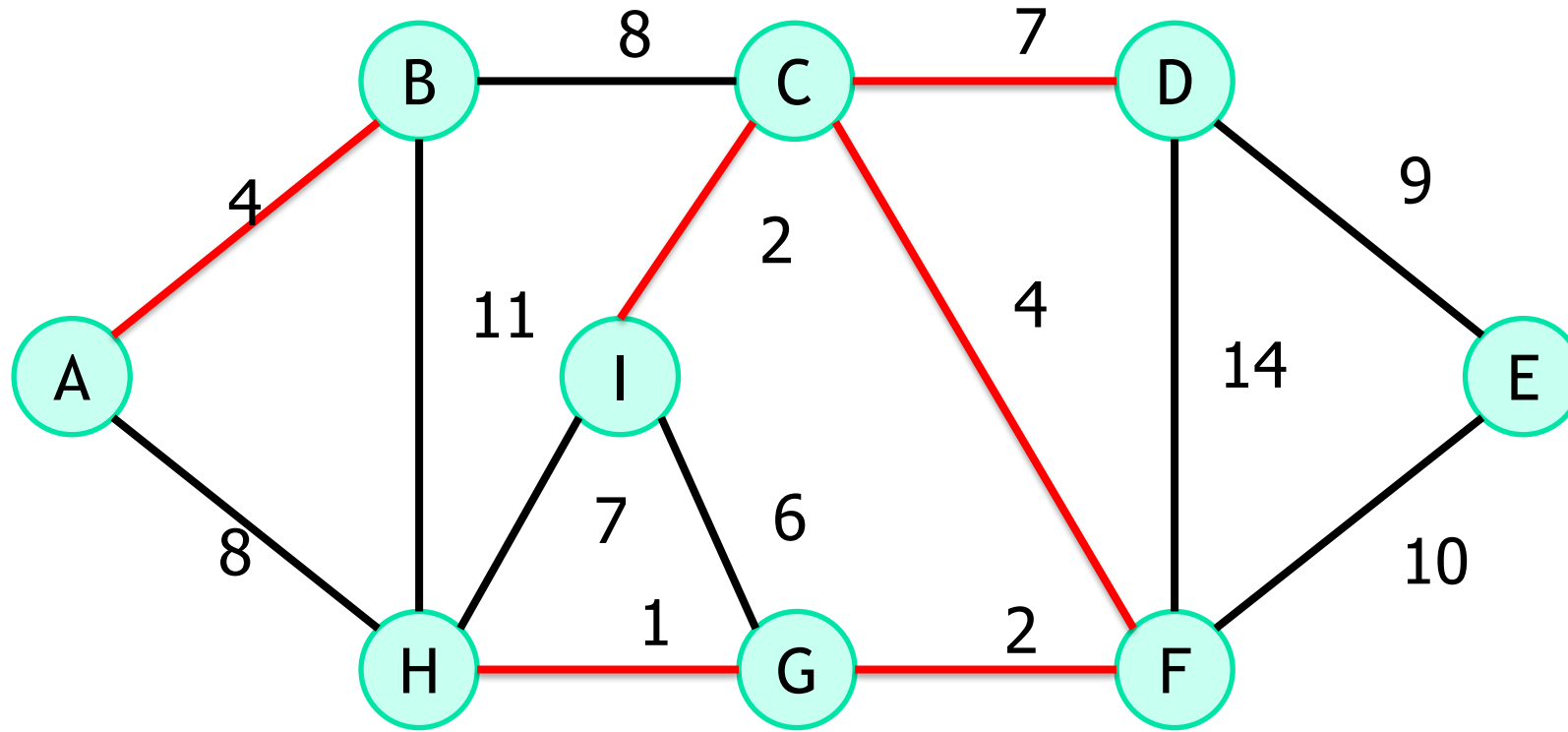
Continue as long as no cycle forms



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



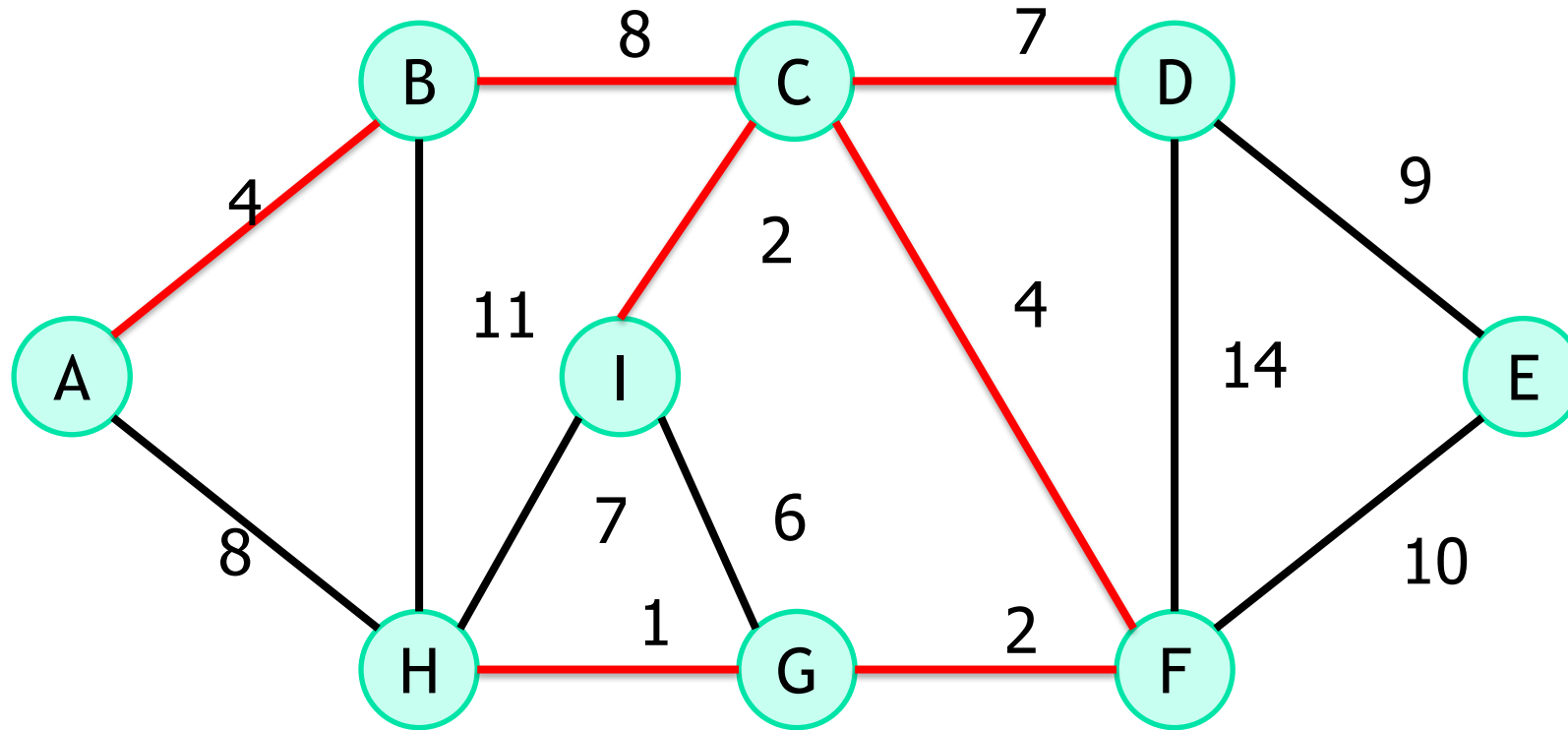
(h,i) cannot be included, otherwise, a cycle is formed



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
(b,c)	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



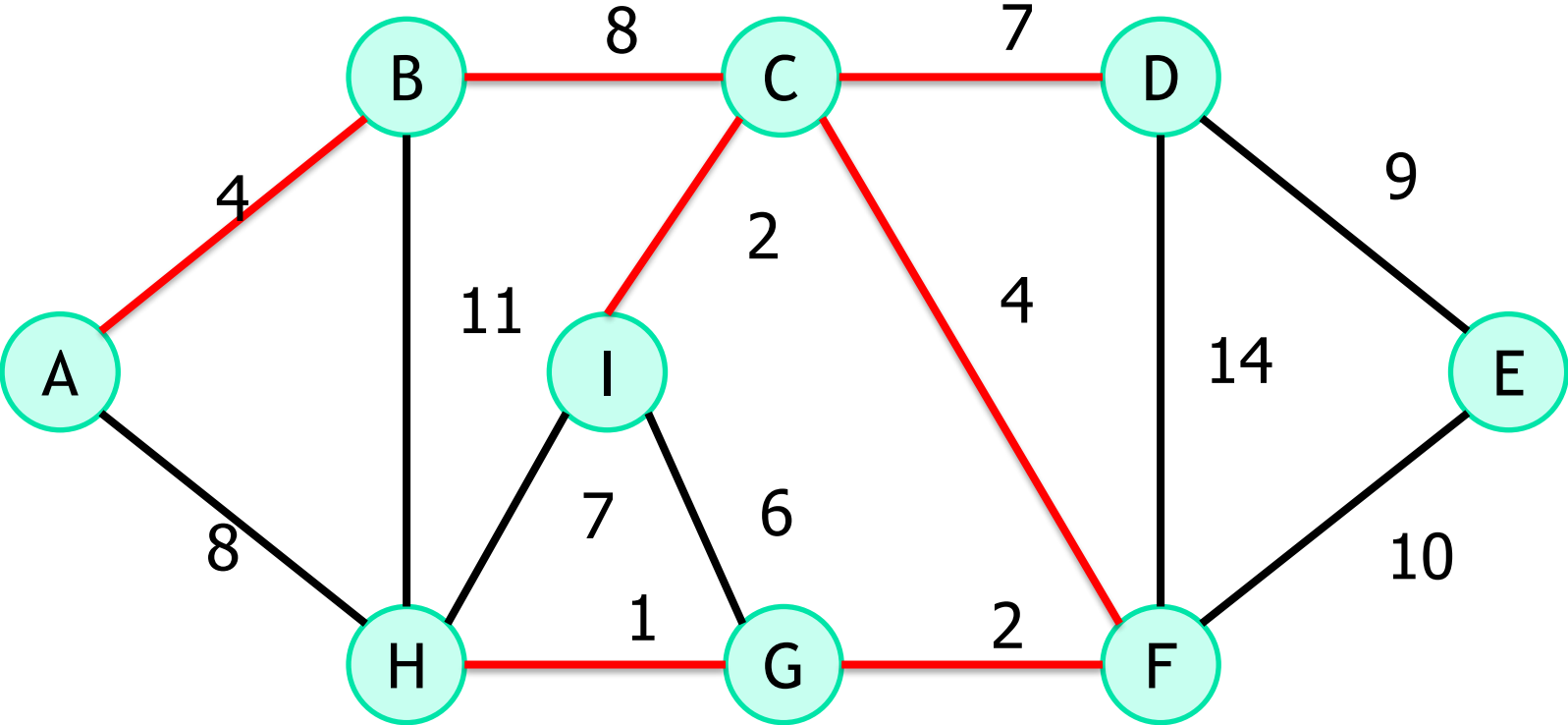
Choose the next minimum weight edge



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
<i>(b,c)</i>	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST



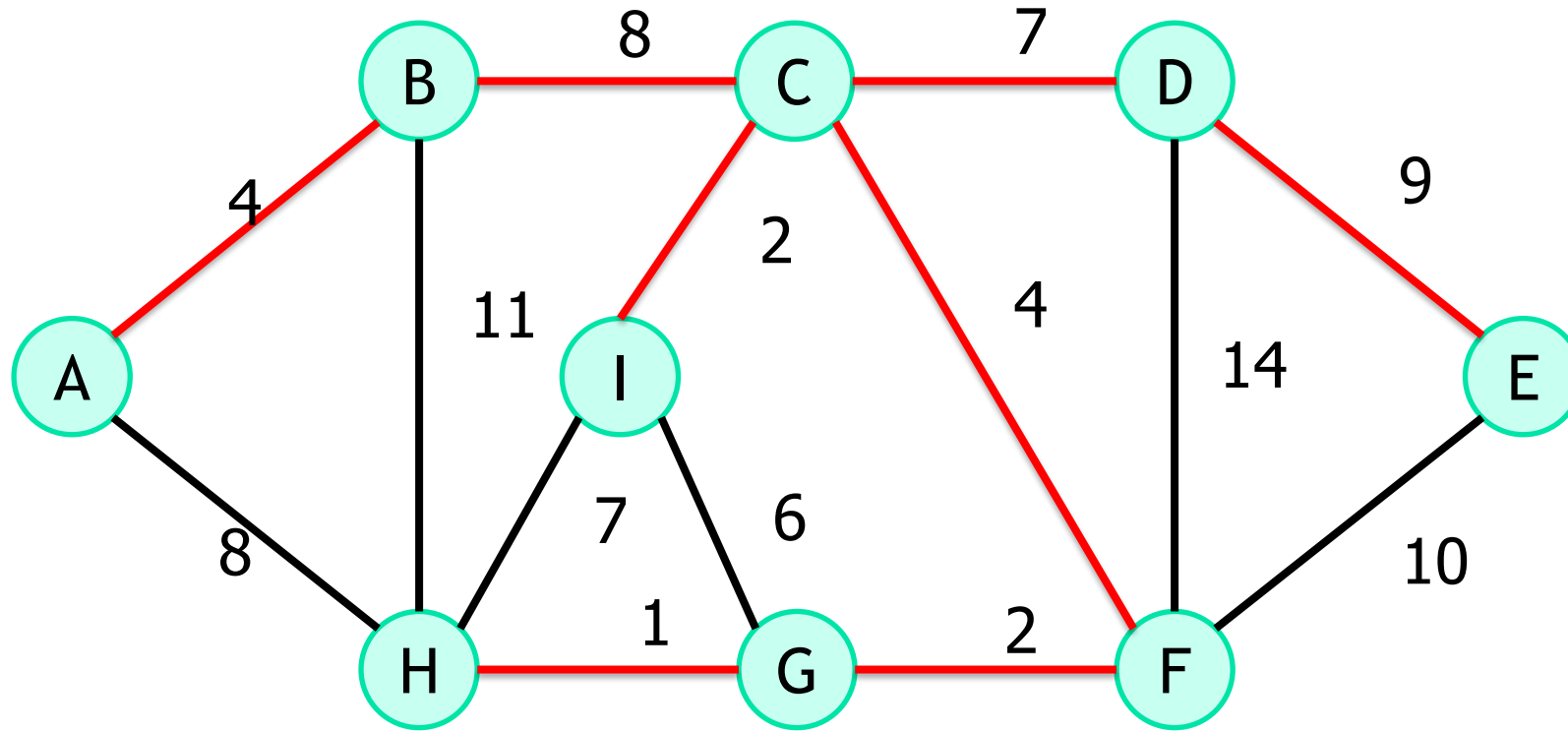
(a,h) cannot be included, otherwise, a cycle is formed



<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
<i>(b,c)</i>	8
(a,h)	8
(d,e)	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST

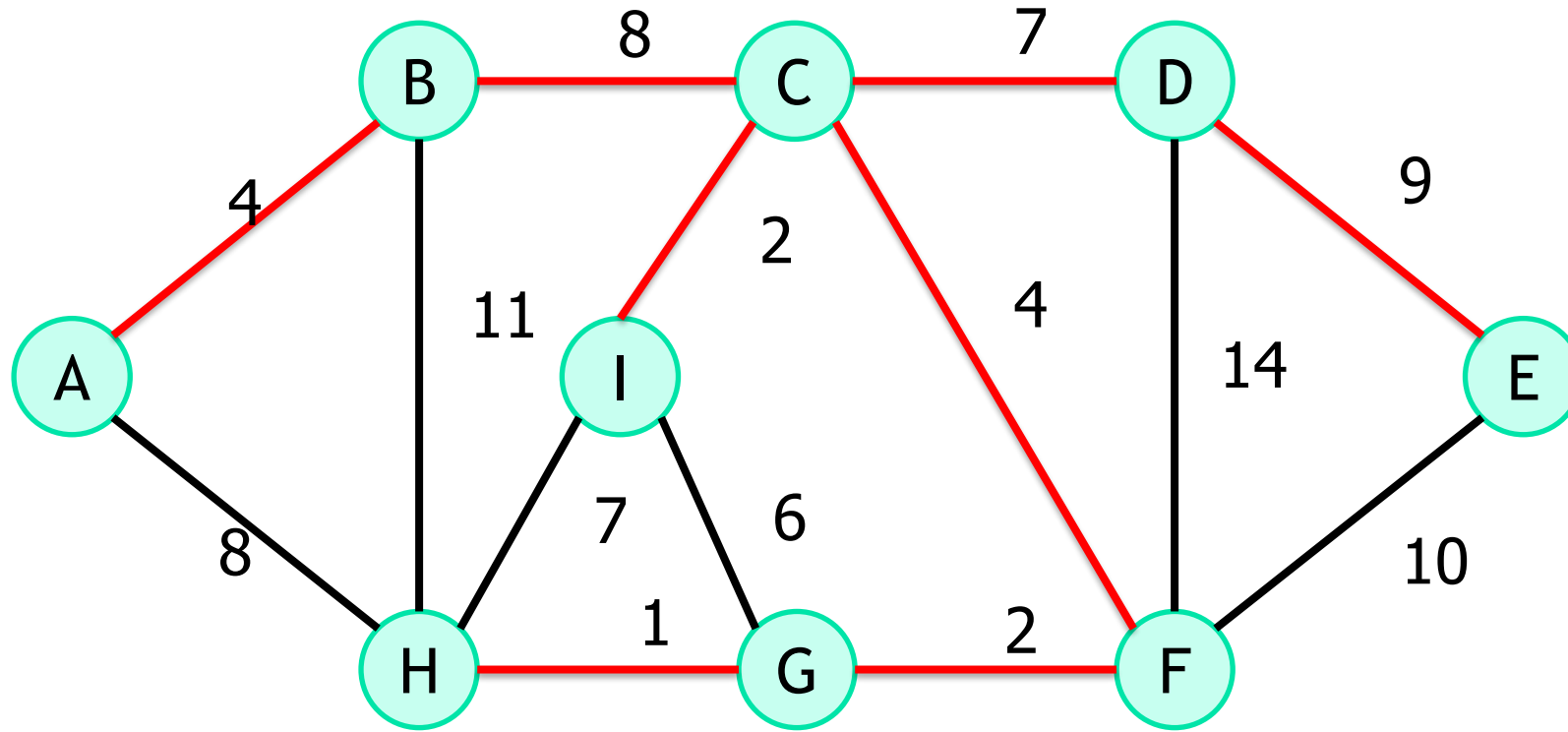


Choose the next minimum weight edge

<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
<i>(b,c)</i>	8
(a,h)	8
<i>(d,e)</i>	9
(f,e)	10
(b,h)	11
(d,f)	14

italic: chosen

Kruskal's algorithm - MST

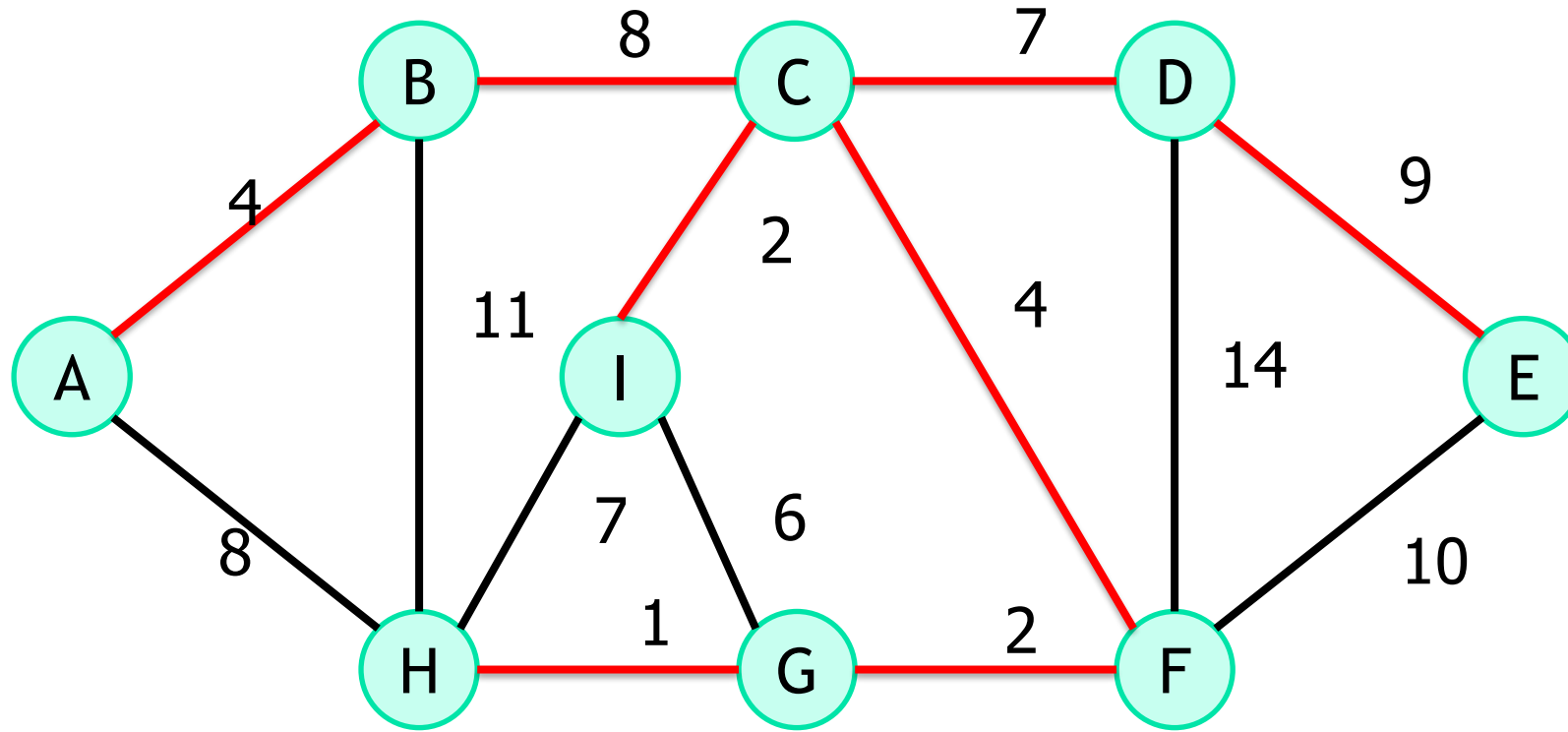


(f,e) cannot be included, otherwise, a cycle is formed

<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
<i>(b,c)</i>	8
(a,h)	8
<i>(d,e)</i>	9
(f,e)	10
<i>(b,h)</i>	11
<i>(d,f)</i>	14

italic: chosen

Kruskal's algorithm - MST

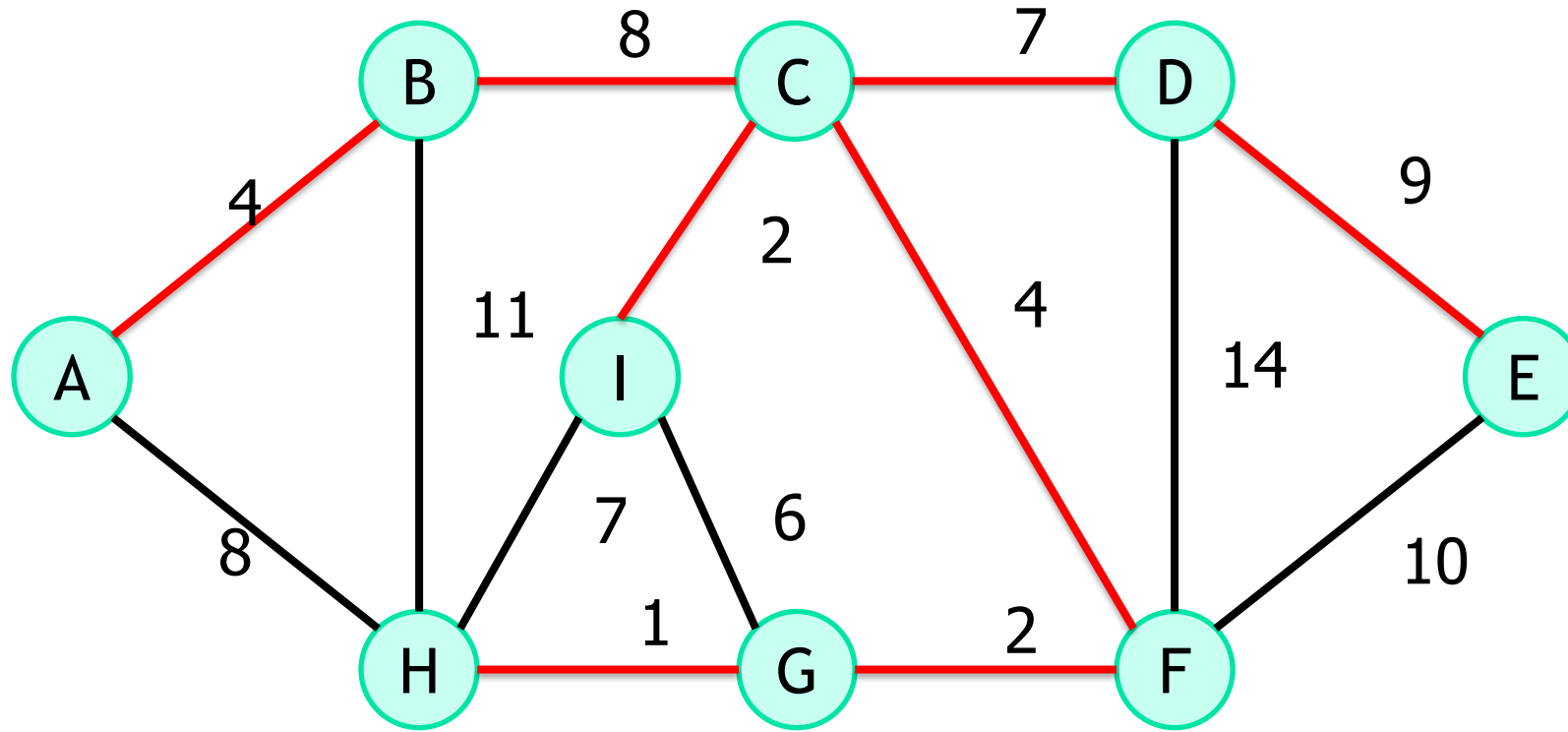


(b,h) cannot be included, otherwise, a cycle is formed

<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
<i>(b,c)</i>	8
(a,h)	8
<i>(d,e)</i>	9
(f,e)	10
(b,h)	11
<i>(d,f)</i>	14

→
italic: chosen

Kruskal's algorithm - MST

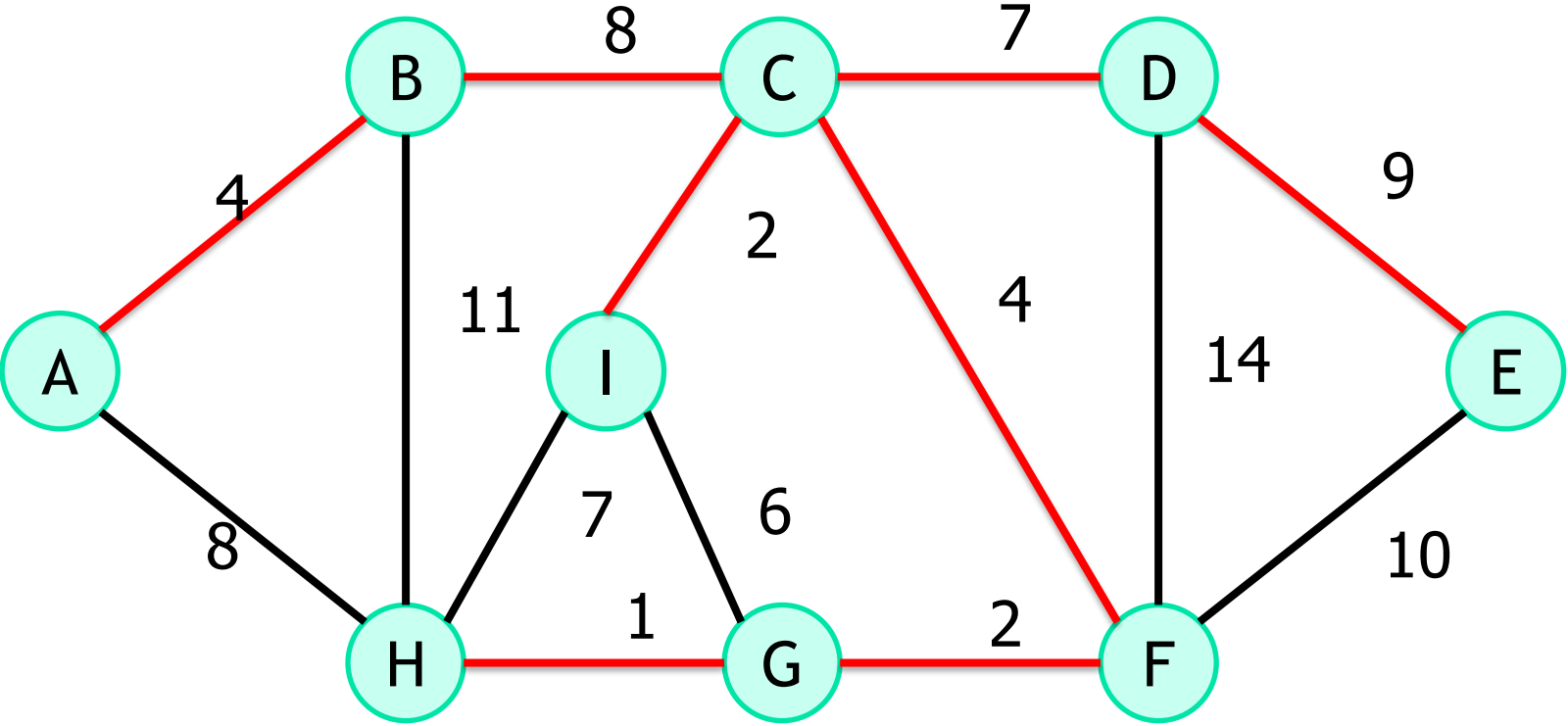


(d,f) cannot be included, otherwise, a cycle is formed

<i>(h,g)</i>	1
<i>(i,c)</i>	2
<i>(g,f)</i>	2
<i>(a,b)</i>	4
<i>(c,f)</i>	4
(i,g)	6
<i>(c,d)</i>	7
(h,i)	7
<i>(b,c)</i>	8
(a,h)	8
<i>(d,e)</i>	9
(f,e)	10
(b,h)	11
(d,f)	14

→ *italic: chosen*

Kruskal's algorithm - MST

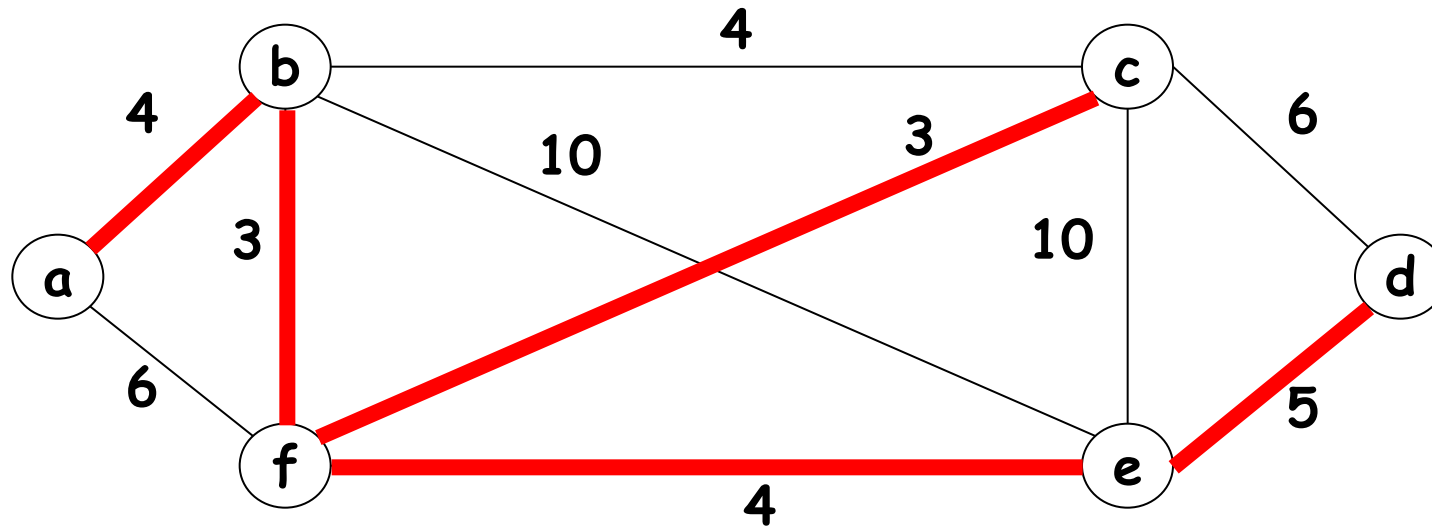


MST is found when all edges are examined

<i>(h,g)</i>	<i>1</i>
<i>(i,c)</i>	<i>2</i>
<i>(g,f)</i>	<i>2</i>
<i>(a,b)</i>	<i>4</i>
<i>(c,f)</i>	<i>4</i>
(i,g)	6
<i>(c,d)</i>	<i>7</i>
(h,i)	7
<i>(b,c)</i>	<i>8</i>
(a,h)	8
<i>(d,e)</i>	<i>9</i>
(f,e)	10
(b,h)	11
(d,f)	14

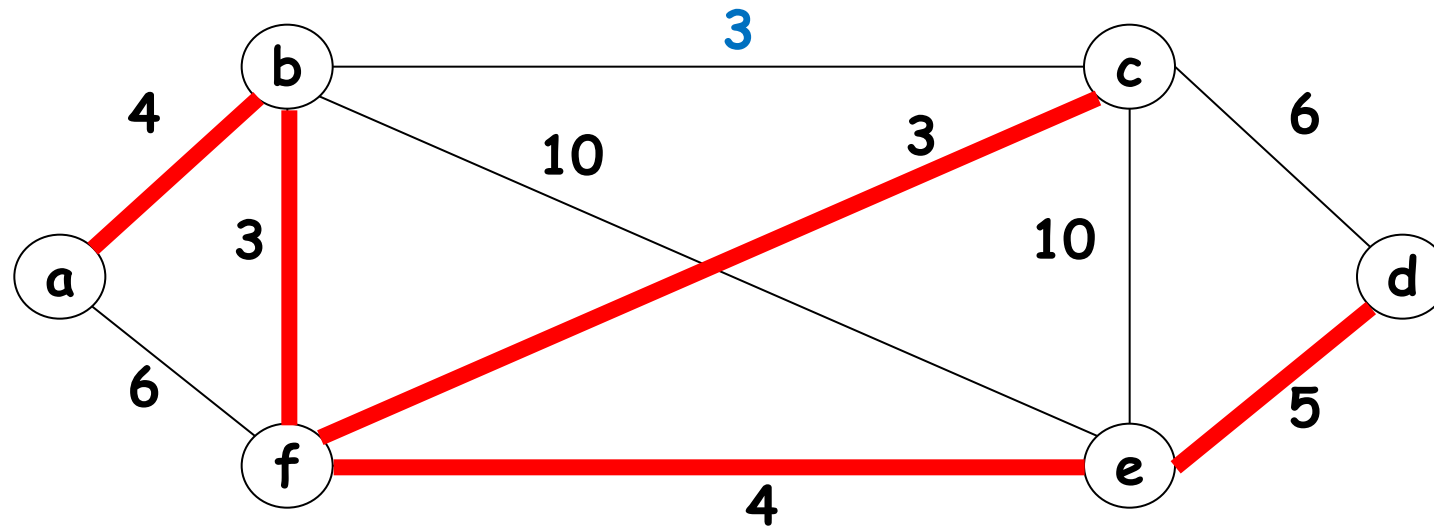
italic: chosen

Exercise – Find MST for this graph



order of (edges) selection: (b,f), (c,f), (a,b), (f,e), (e,d)

Exercise – Find MST for this graph



Kruskal's algorithm

m is the number of edges in the graph
 n is the number of vertices in the graph

slowKruskal($G = (V, E)$):

Sort the edges in E by non-decreasing weight.

$MST = \{\}$

for e in E (in sorted order):

if adding e to MST won't cause a cycle:
 add e to MST .

return MST

m iterations through
this loop

How do we
check this?

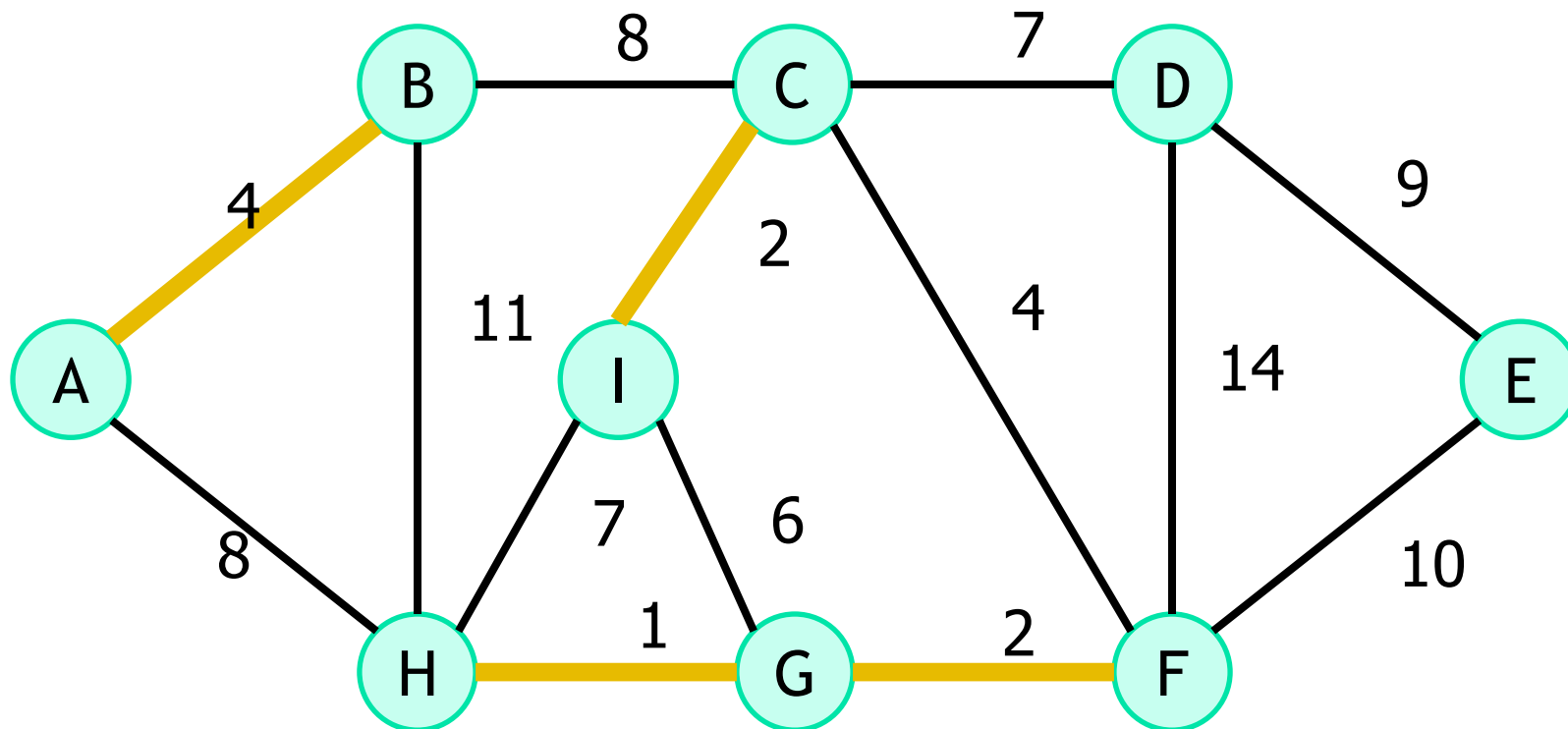
Naively, the running time is $O(mn)$:

- For each of m iterations of the for loop:
 - Check if adding e would cause a cycle...

Kruskal's algorithm

At each step of Kruskal's, we are maintaining a forest.

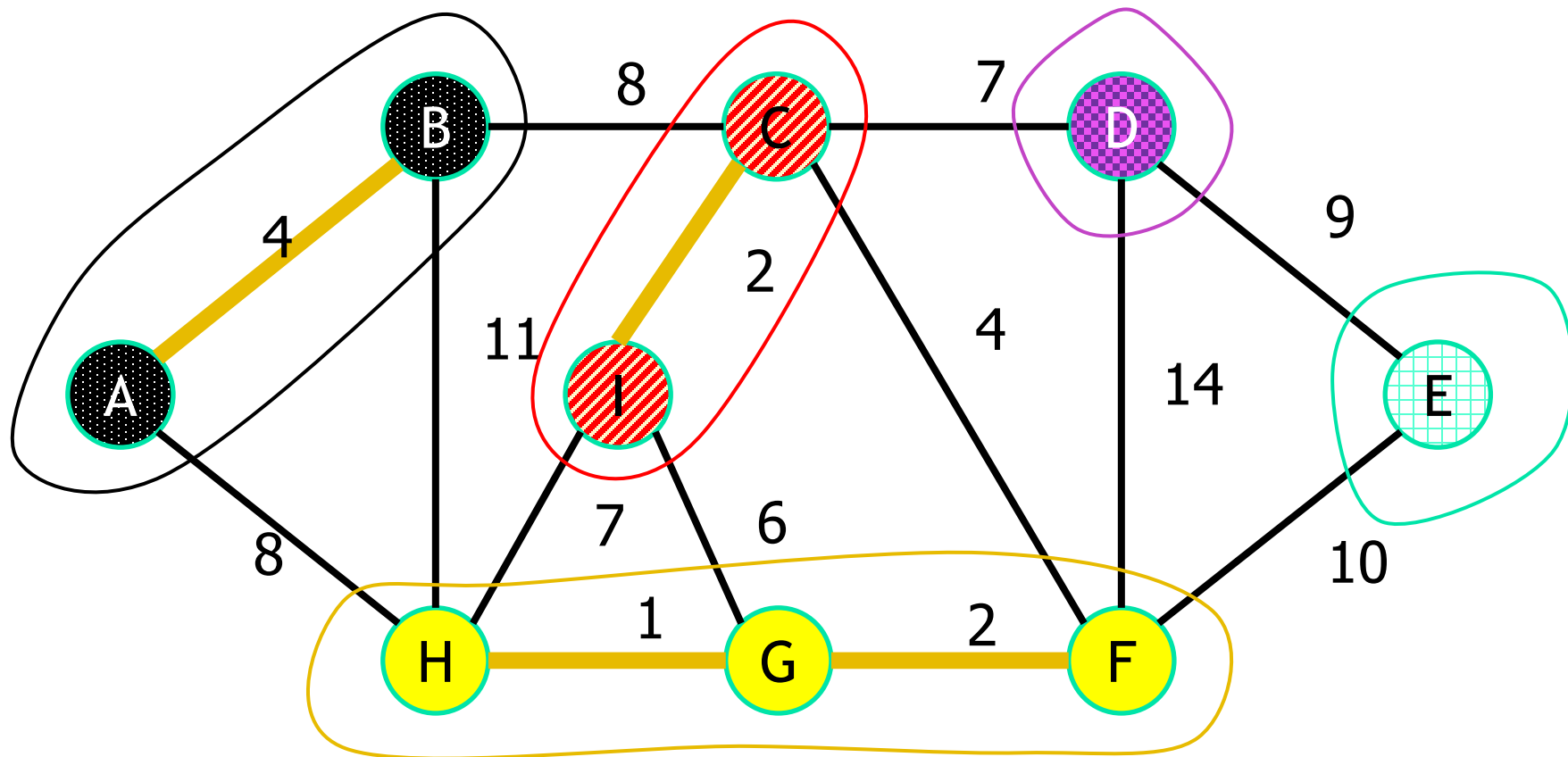
A **forest** is
a collection
of disjoint
trees



Kruskal's algorithm

At each step of Kruskal's, we are maintaining a forest.

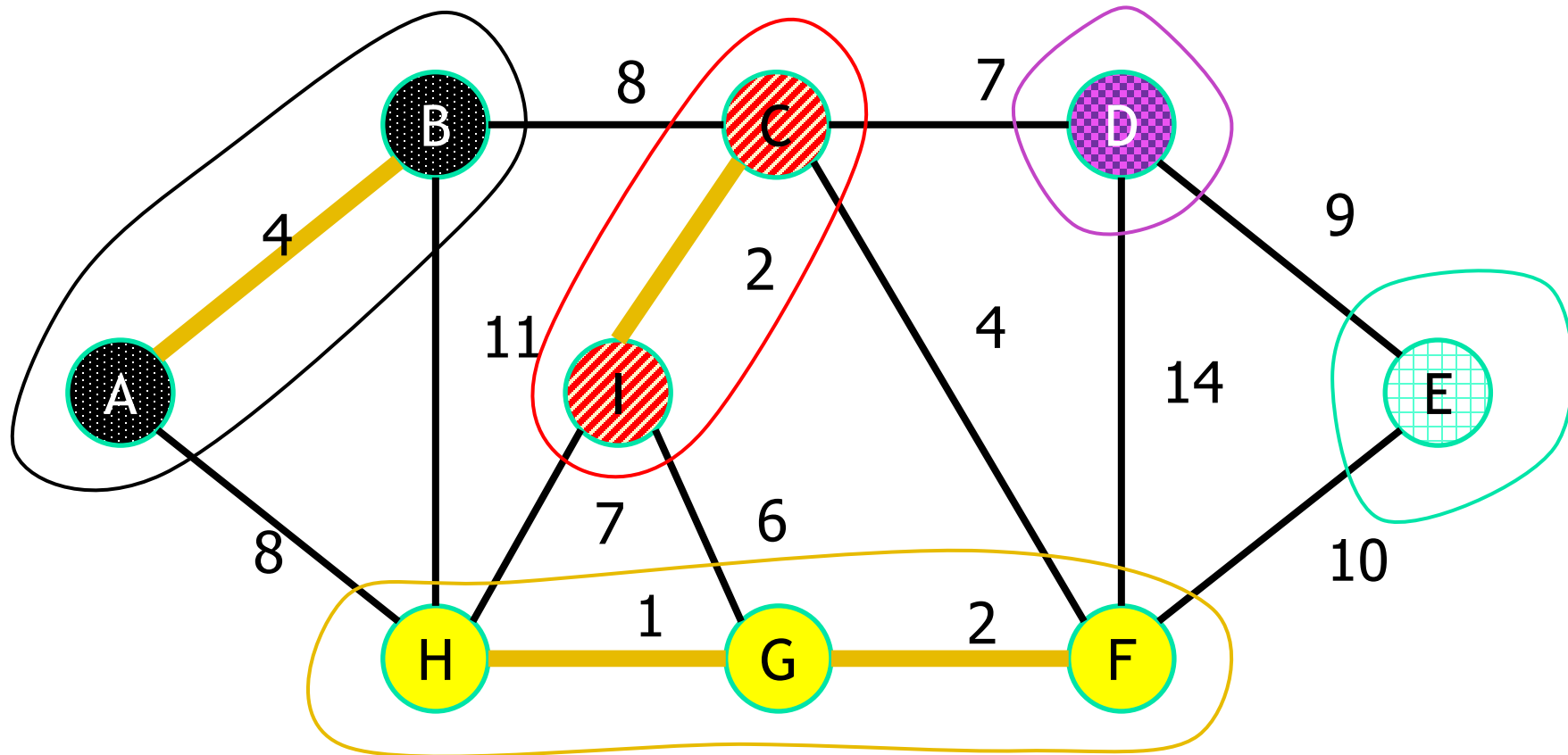
A **forest** is a collection of disjoint trees



Kruskal's algorithm

When we add an edge, we merge two trees:

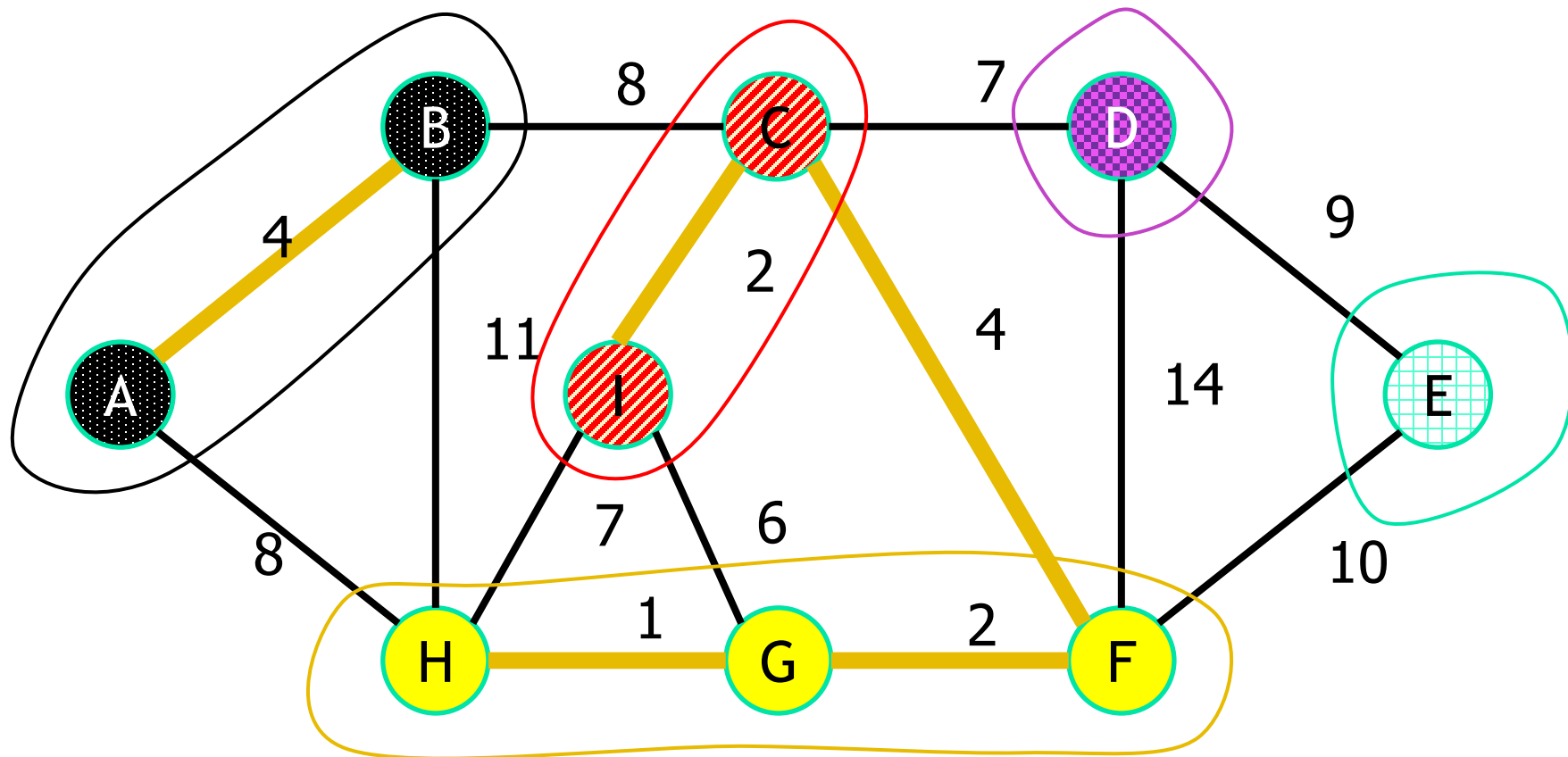
A **forest** is a collection of disjoint trees



Kruskal's algorithm

When we add an edge, we merge two trees:

A **forest** is a collection of disjoint trees

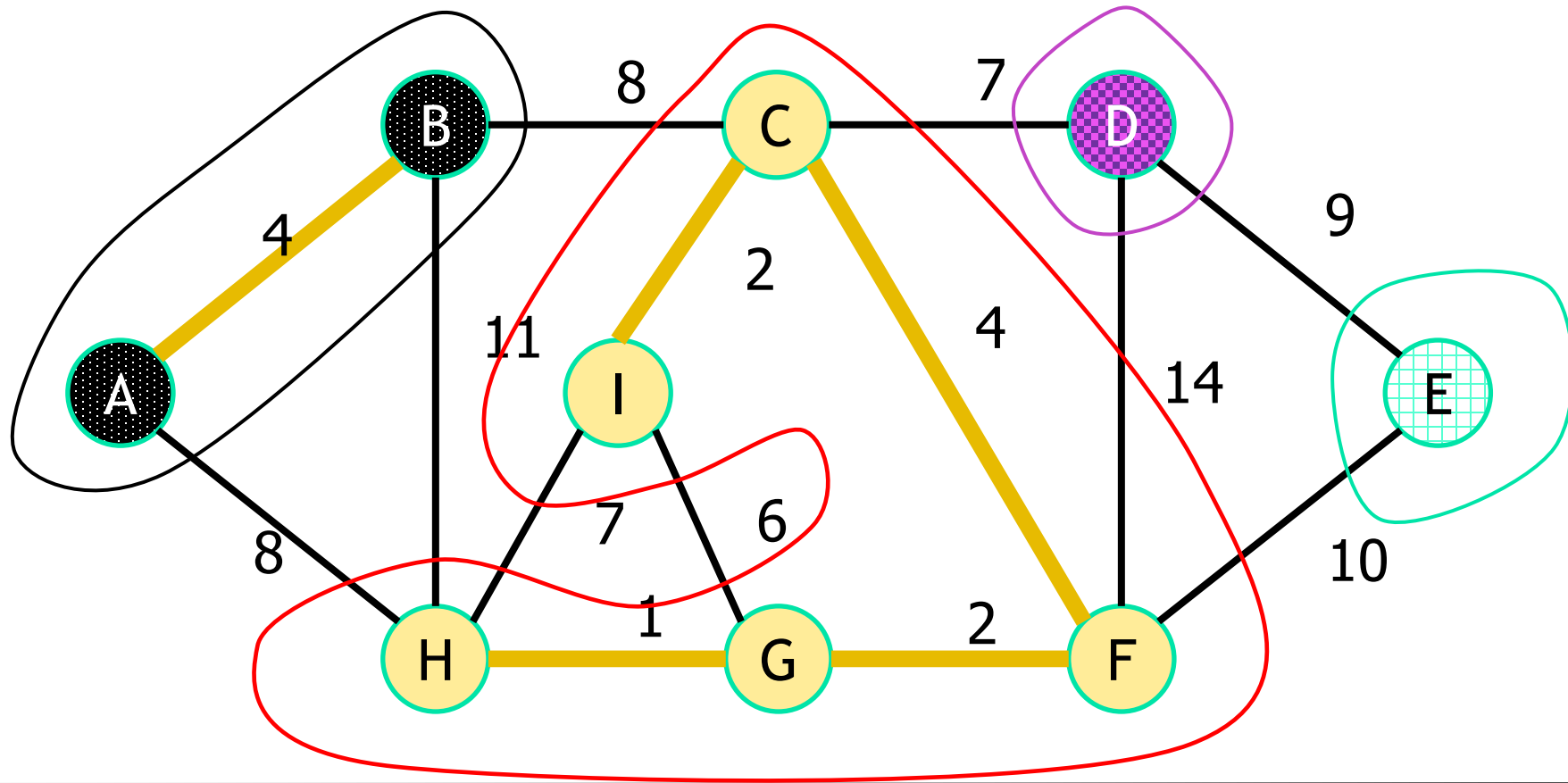


Kruskal's algorithm

When we add an edge, we merge two trees.

We never add an edge within a tree since that would create a cycle

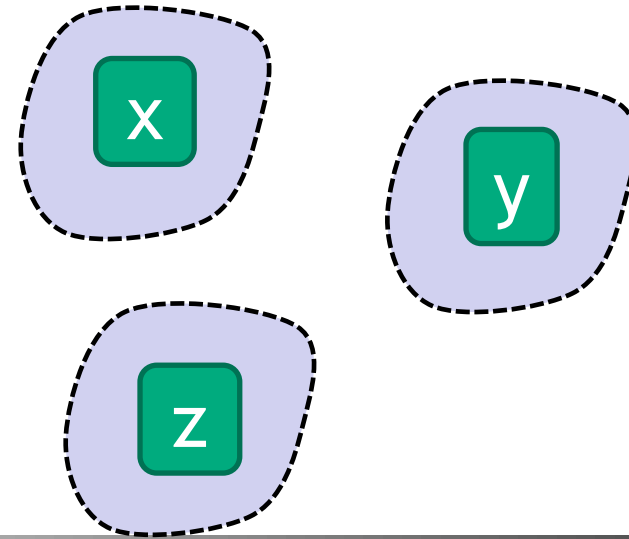
A **forest** is a collection of disjoint trees



Disjoint-set data structure

- Used for storing collections of sets
- Supports:
 - **makeSet(u)**: create a set {u}
 - **find(u)**: return the set that u is in
 - **union(u,v)**: merge the set that u is in with the set that v is in.

```
makeSet (x)  
makeSet (y)  
makeSet (z)
```



Disjoint-set data structure

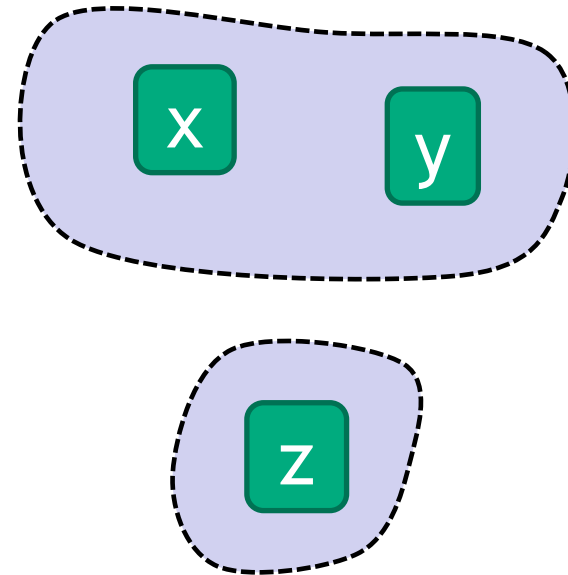
- Used for storing collections of sets
- Supports:
 - **makeSet(u)**: create a set {u}
 - **find(u)**: return the set that u is in
 - **union(u,v)**: merge the set that u is in with the set that v is in.

`makeSet (x)`

`makeSet (y)`

`makeSet (z)`

`union (x, y)`



Disjoint-set data structure

- Used for storing collections of sets
- Supports:
 - **makeSet(u)**: create a set {u}
 - **find(u)**: return the set that u is in
 - **union(u,v)**: merge the set that u is in with the set that v is in.

Refer to Textbook
Chapter 21 for more
info

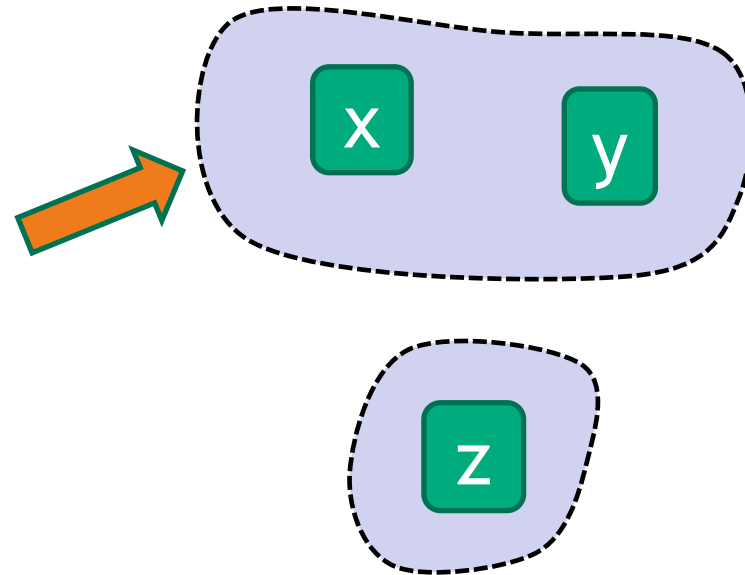
`makeSet (x)`

`makeSet (y)`

`makeSet (z)`

`union (x, y)`

`find (x)`



Kruskal pseudo-code

kruskal($G = (V, E)$):

Sort E by weight in non-decreasing order

$MST = \{\}$ // initialize an empty tree

for v in V :

makeSet(v) // put each vertex in its own tree in the forest

for (u, v) in E : // go through the edges in sorted order

if $\text{find}(u) \neq \text{find}(v)$: // if u and v are not in the same tree

 add (u, v) to MST

union(u, v) // merge u 's tree with v 's tree

return MST

Running time

- Sorting the edges takes $O(m \log n)$
- For the rest:
 - n calls to **makeSet**
 - put each vertex in its own set
 - $2m$ calls to **find**
 - for each edge, **find** its endpoints
 - n calls to **union**
 - we will never add more than $n-1$ edges to the tree,
 - so we will never call **union** more than $n-1$ times.
- Total running time:
 - Worst-case $O(m \log n)$.

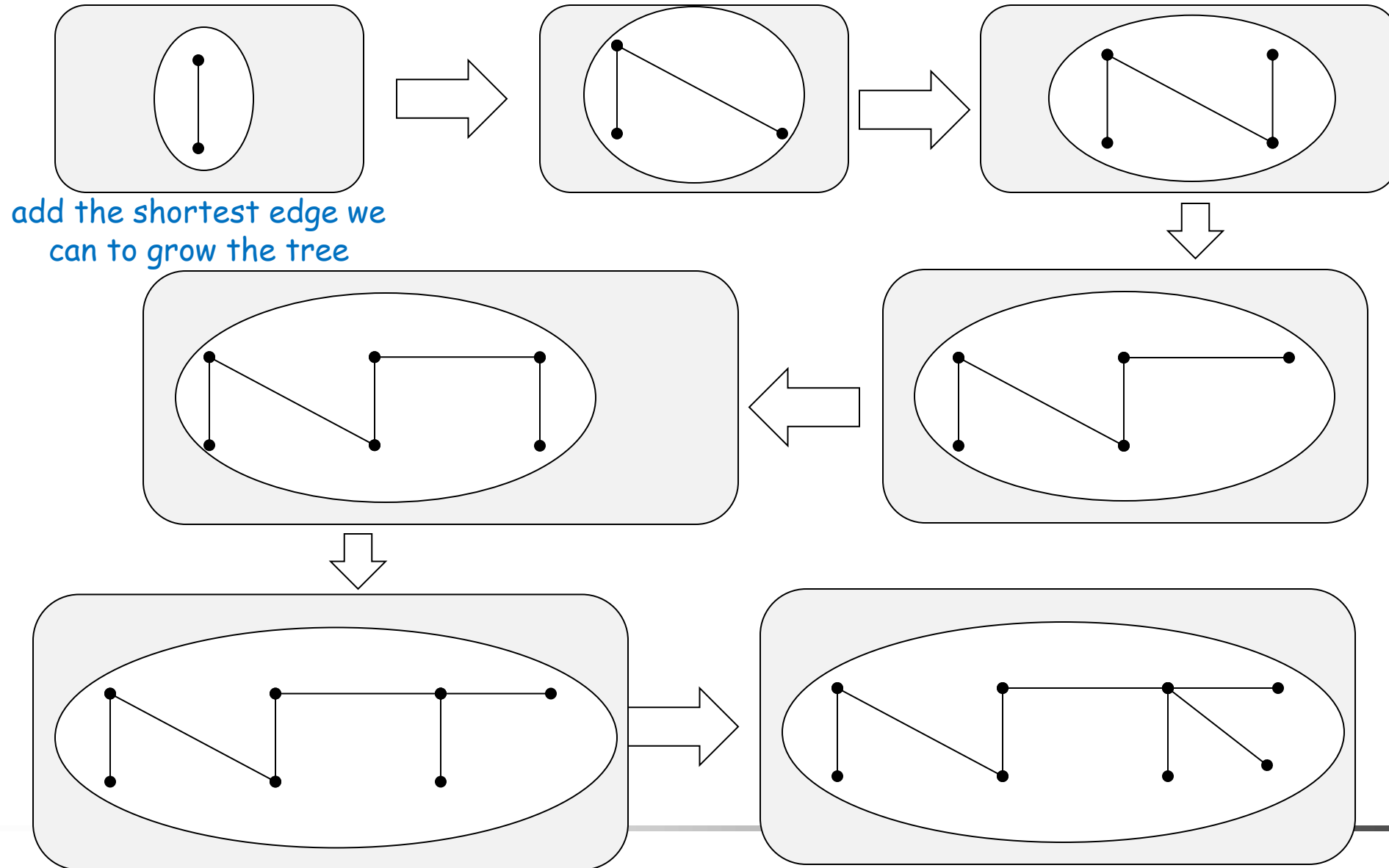
In any graph, $m = O(n^2)$
Therefore,
 $O(m \log m) = O(m \log n^2)$
 $= O(m \log n)$

In practice, each of
makeSet, **find**, and
union run in constant
time*

*technically, they run in *amortized time* $O(\alpha(n))$, where $\alpha(n)$ is the *inverse Ackermann function*.
 $\alpha(n) \leq 4$ provided that n is smaller than the number of atoms in the universe.

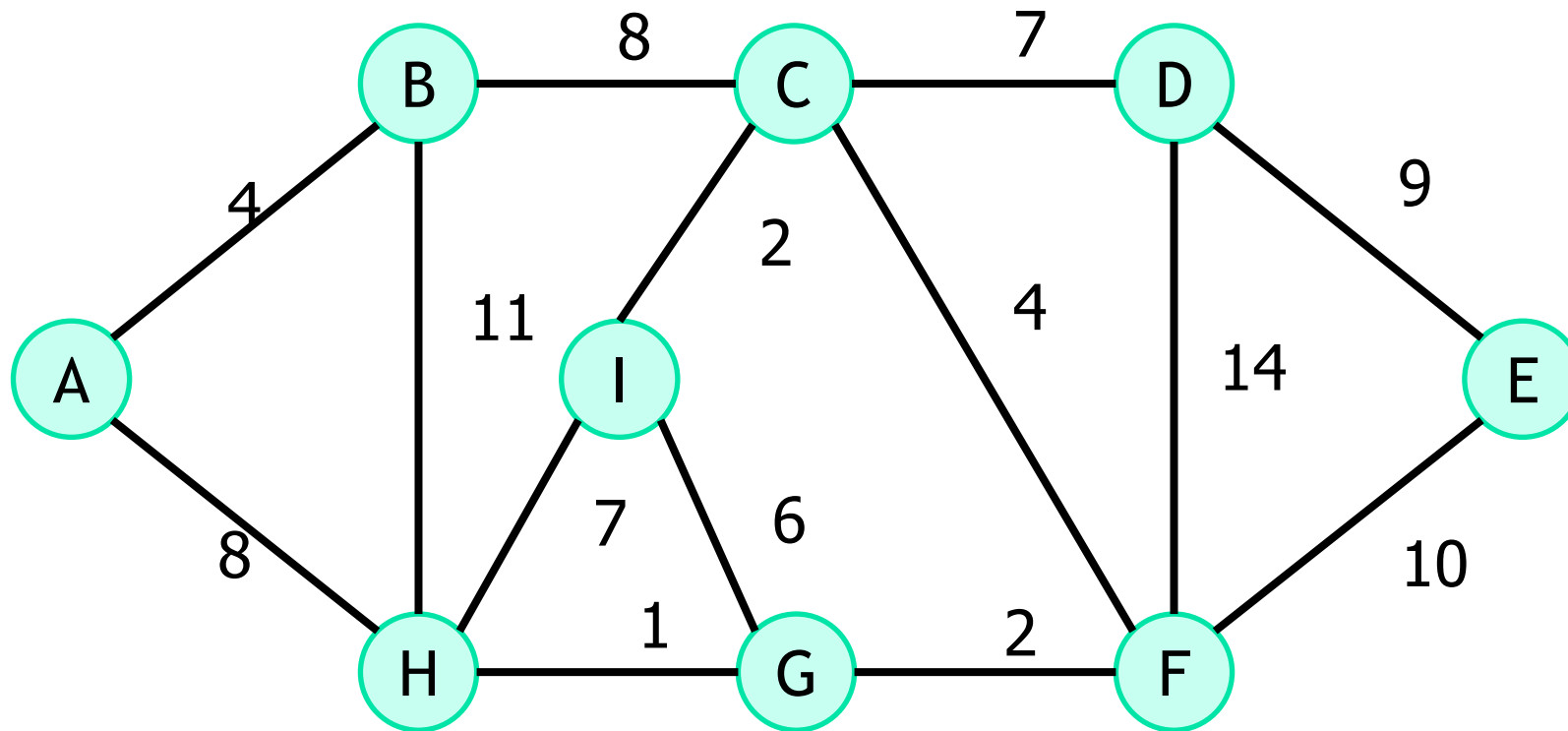
Prim's Algorithm

Idea of Prim's Algorithm - MST



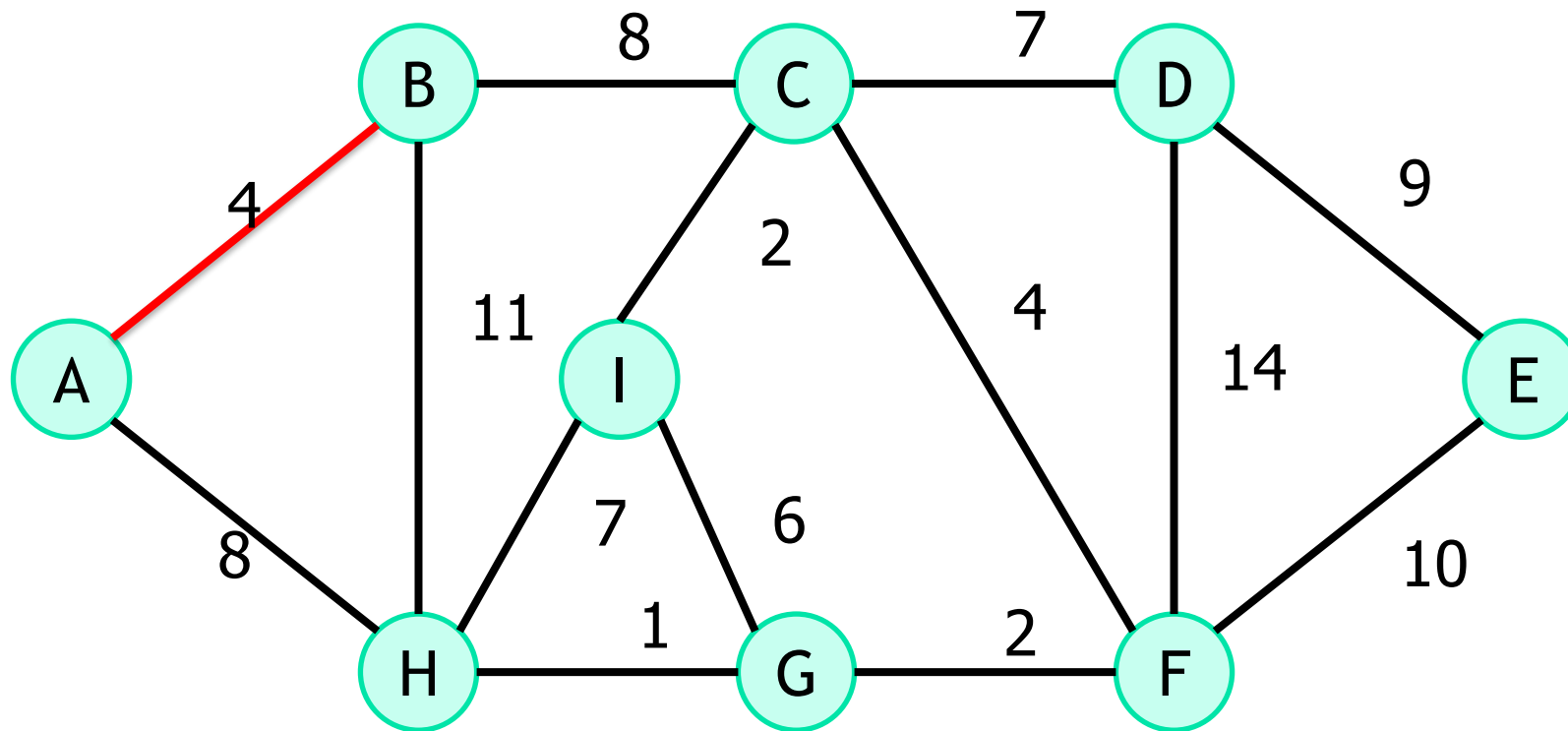
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



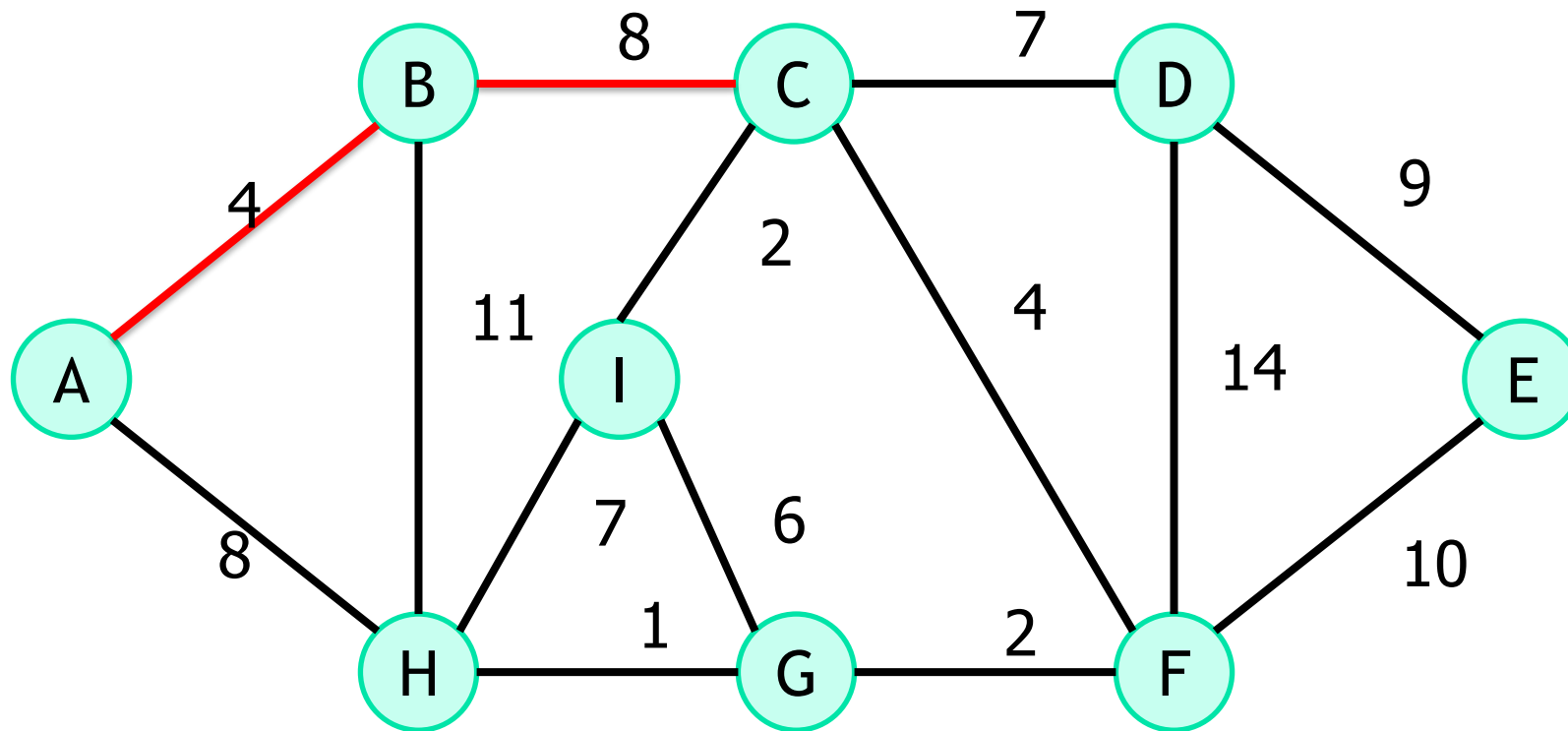
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



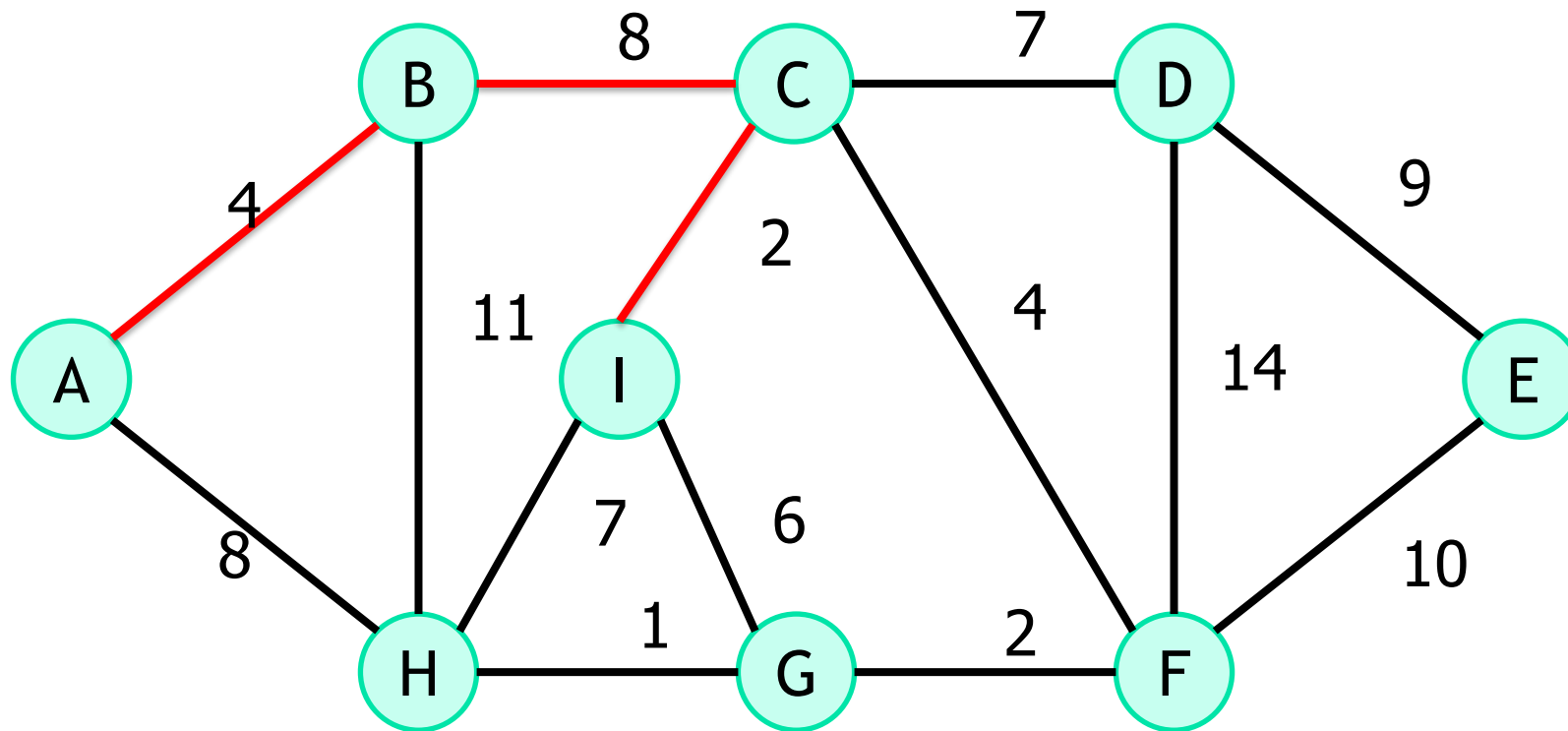
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



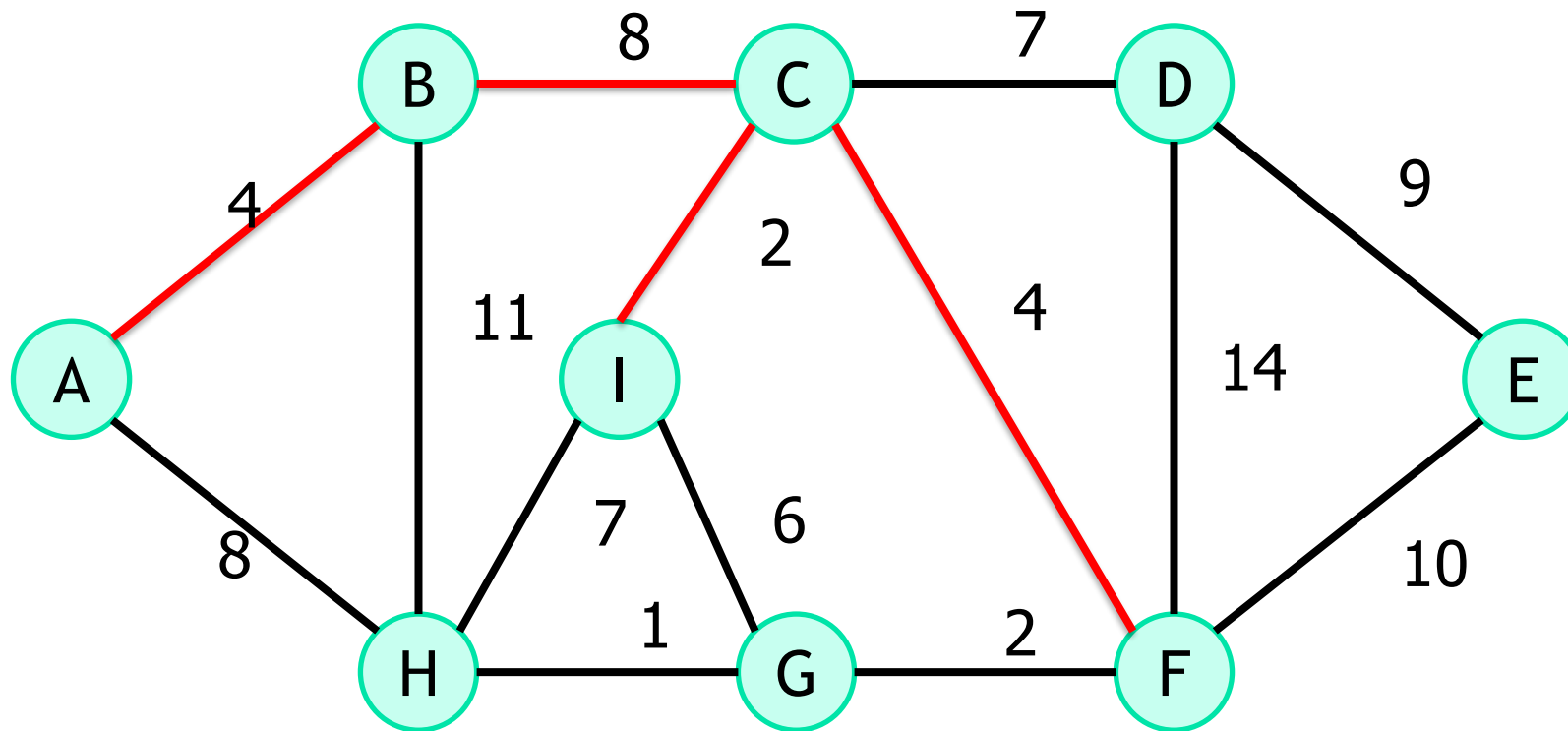
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



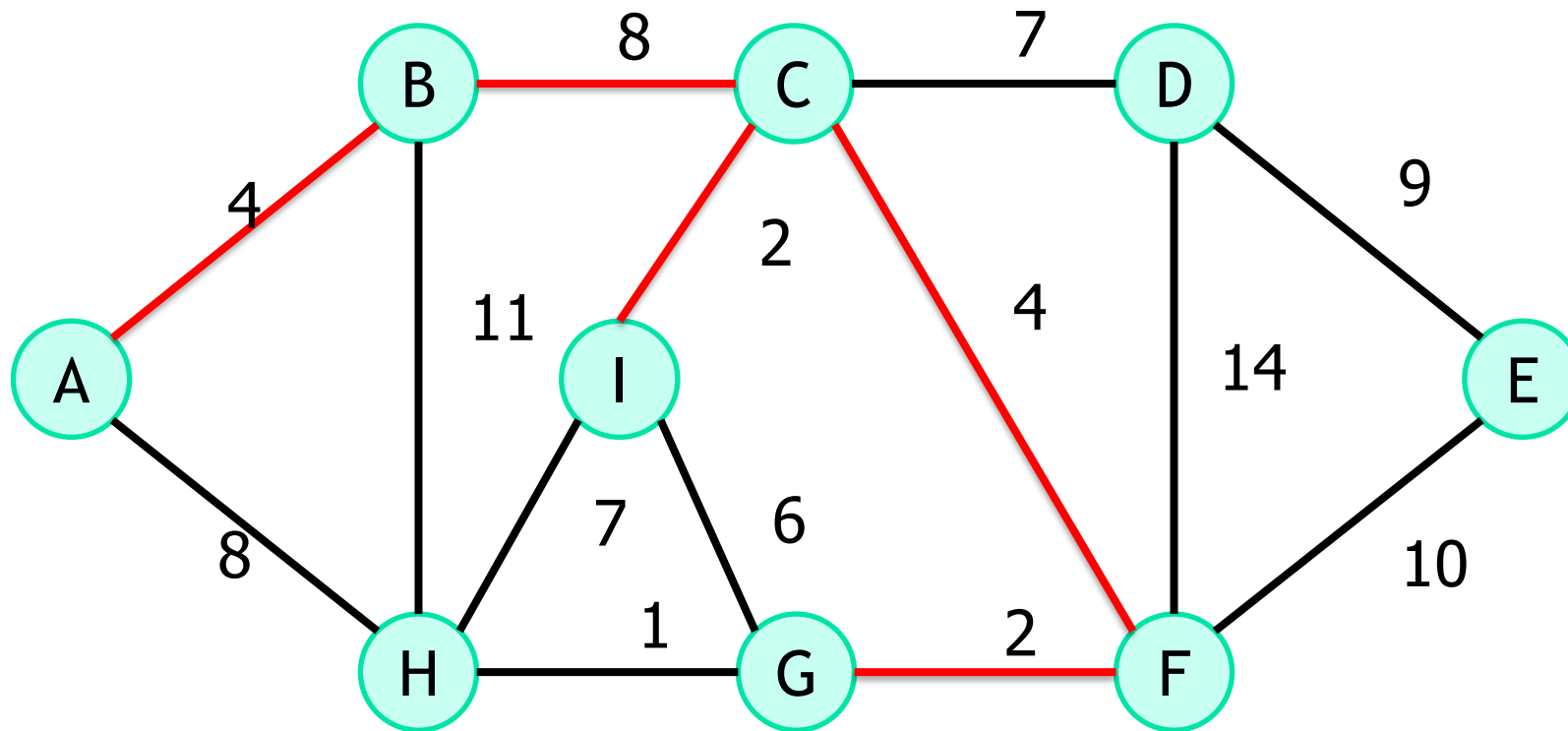
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



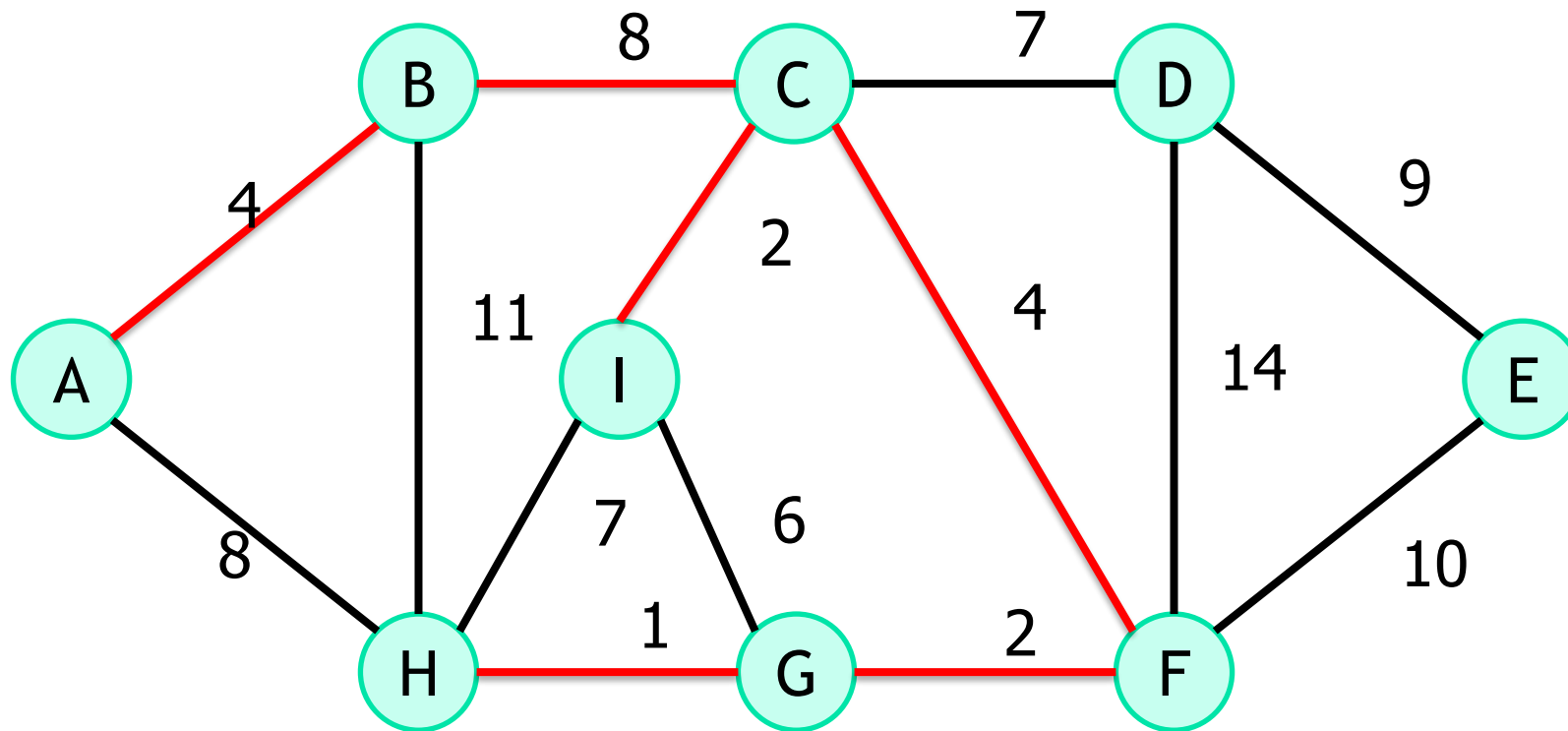
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



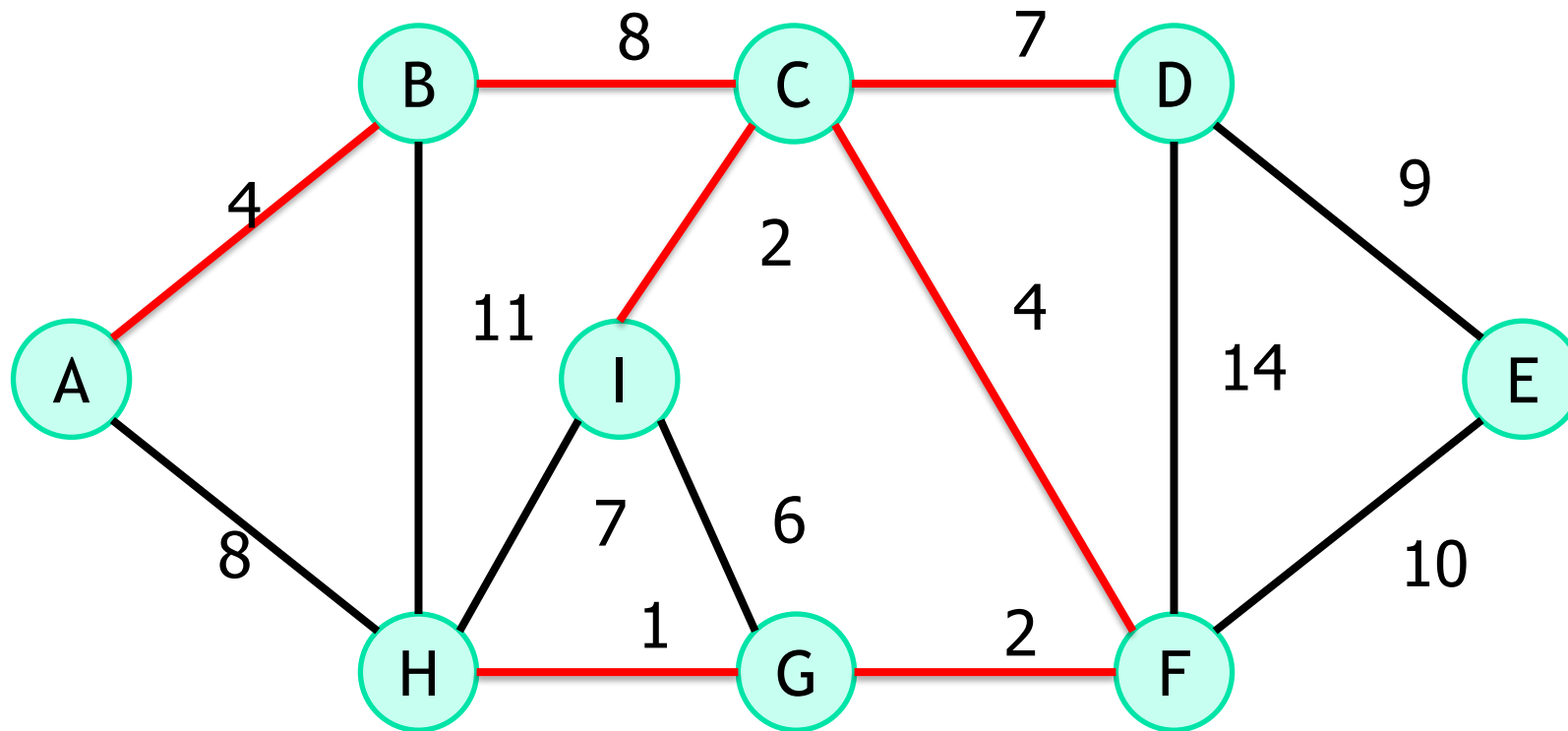
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



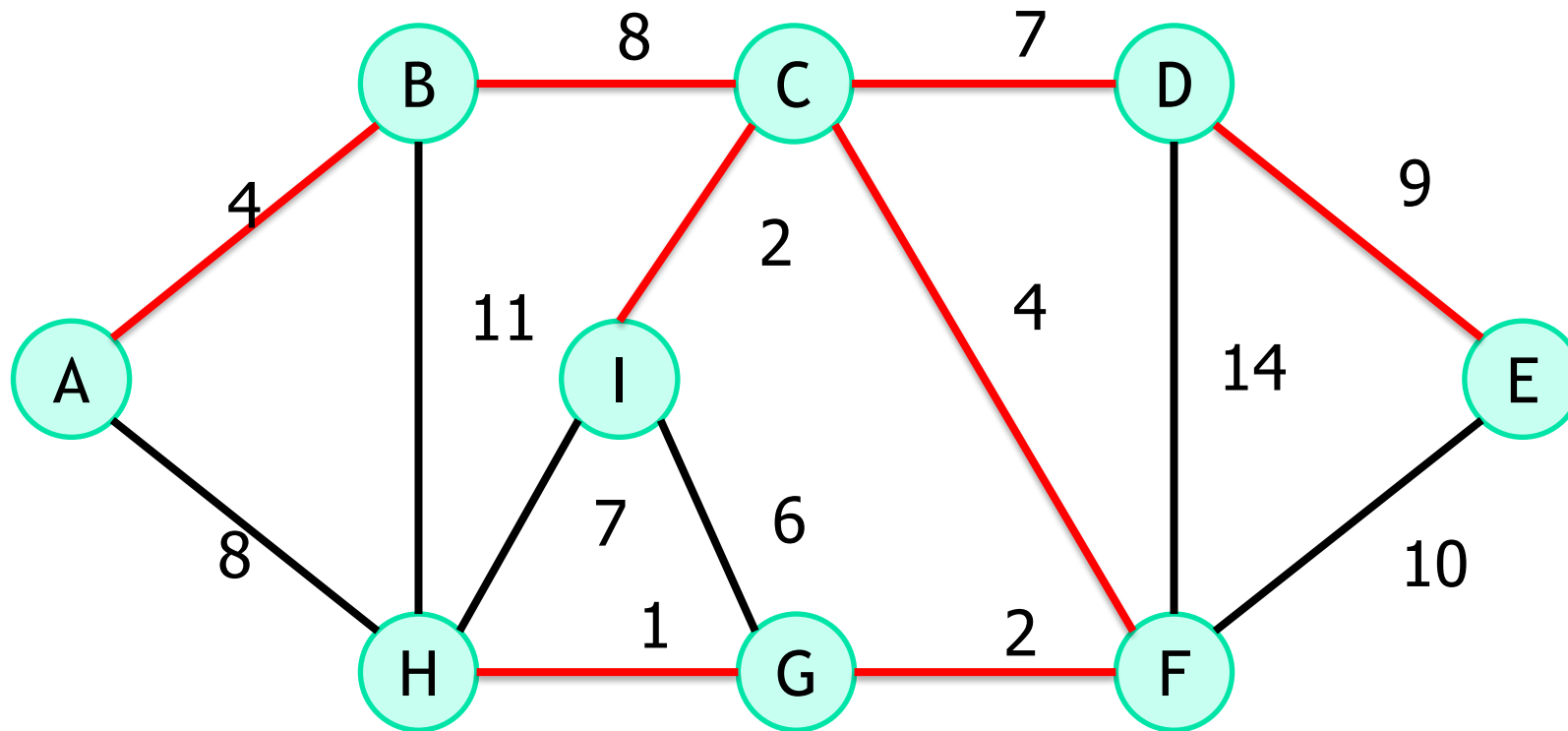
Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



Prim's Algorithm

- Start growing a tree, greedily add the shortest edge we can to grow the tree.



Prim's algorithm

slowPrim($G = (V, E)$, starting vertex s):

MST = {}

verticesVisited = { s }

while |verticesVisited| < | V |:

 find the lightest edge (x, v) in E so that:

x is in verticesVisited

v is not in verticesVisited

 add (x, v) to MST

 add v to verticesVisited

return MST

n iterations
of this while
loop.

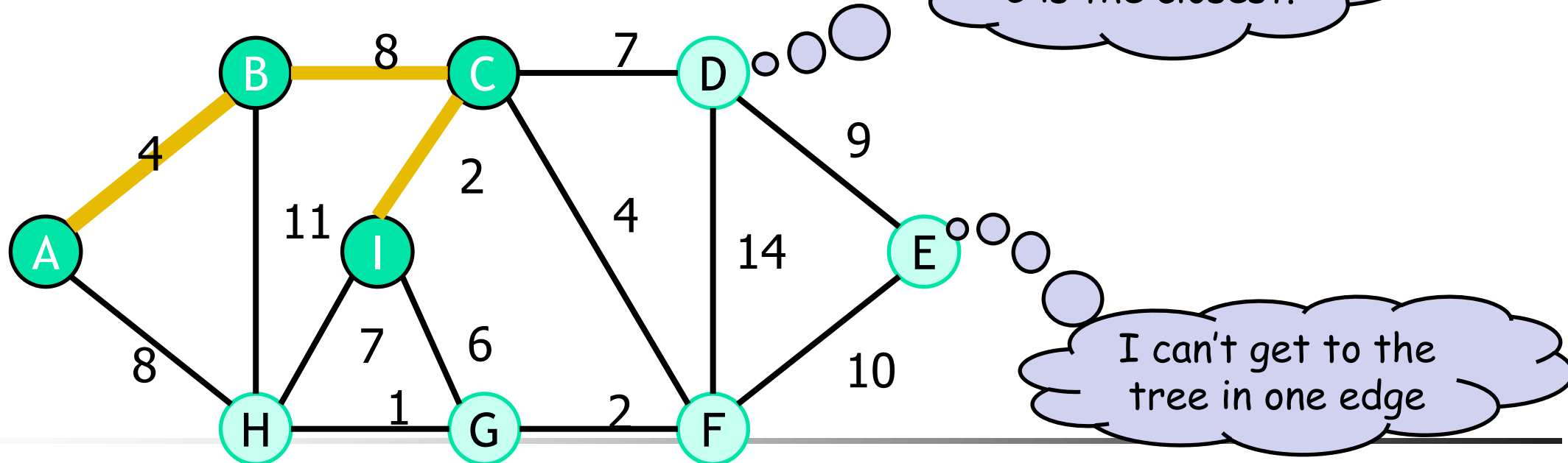
Maybe take
time m to go
through all
the edges
and find the
lightest.

Naively, the running time is $O(nm)$:

- For each of $n-1$ iterations of the while loop:
 - Maybe go through all the edges.

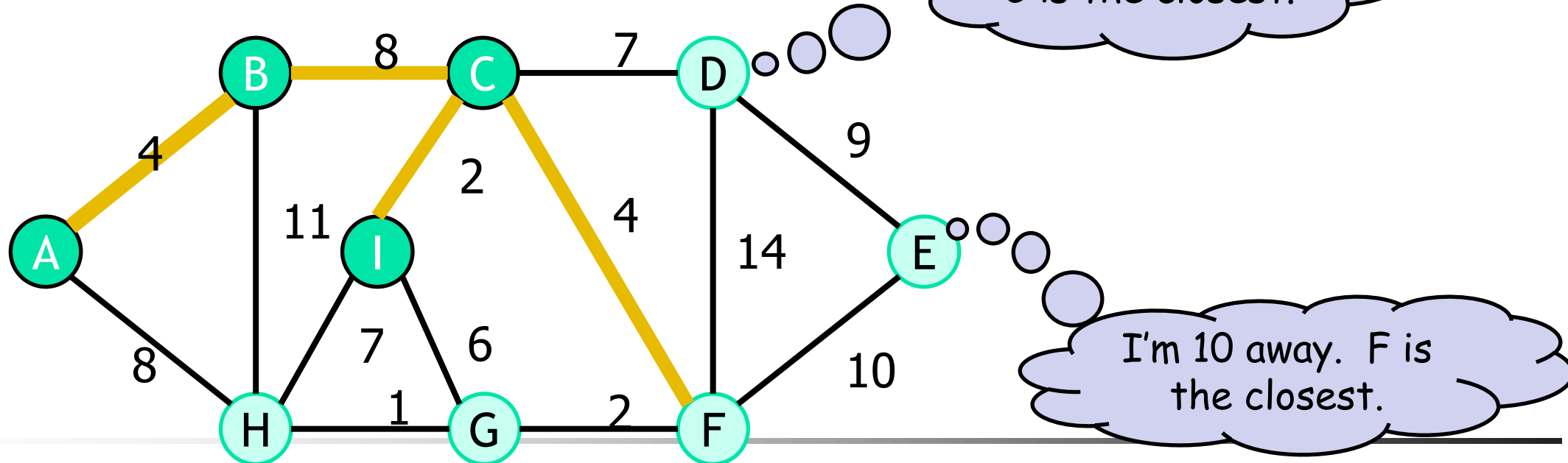
Efficient implementation

- Each vertex keeps:
 - the distance from itself to the **growing spanning tree** if you can get there in one edge.
 - **how to get there.**
- Choose the closest vertex, add it.



Efficient implementation

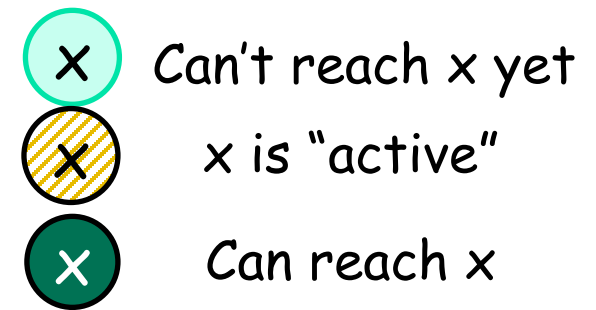
- Each vertex keeps:
 - the distance from itself to the **growing spanning tree** if you can get there in one edge.
 - **how to get there.**
- Choose the closest vertex, add it.
- Update stored info.



Efficient implementation

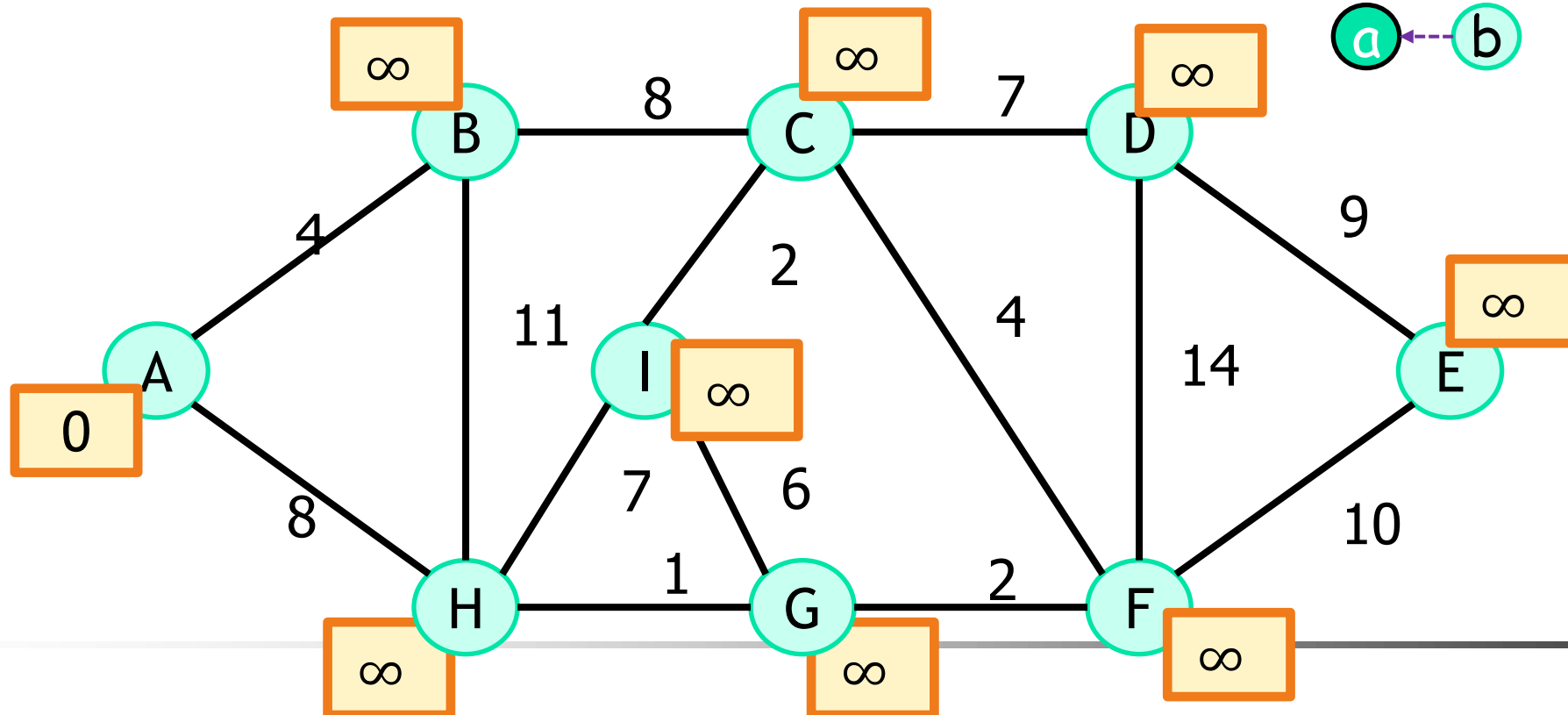
Every vertex has a key and a parent

Until all the vertices are **reached**:



$k[x]$ is the distance of x from the growing tree

$p[b] = a$, meaning that a was the vertex that $k[b]$ comes from.

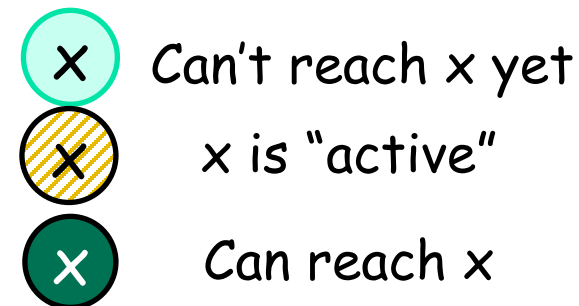


Efficient implementation

Every vertex has a key and a parent

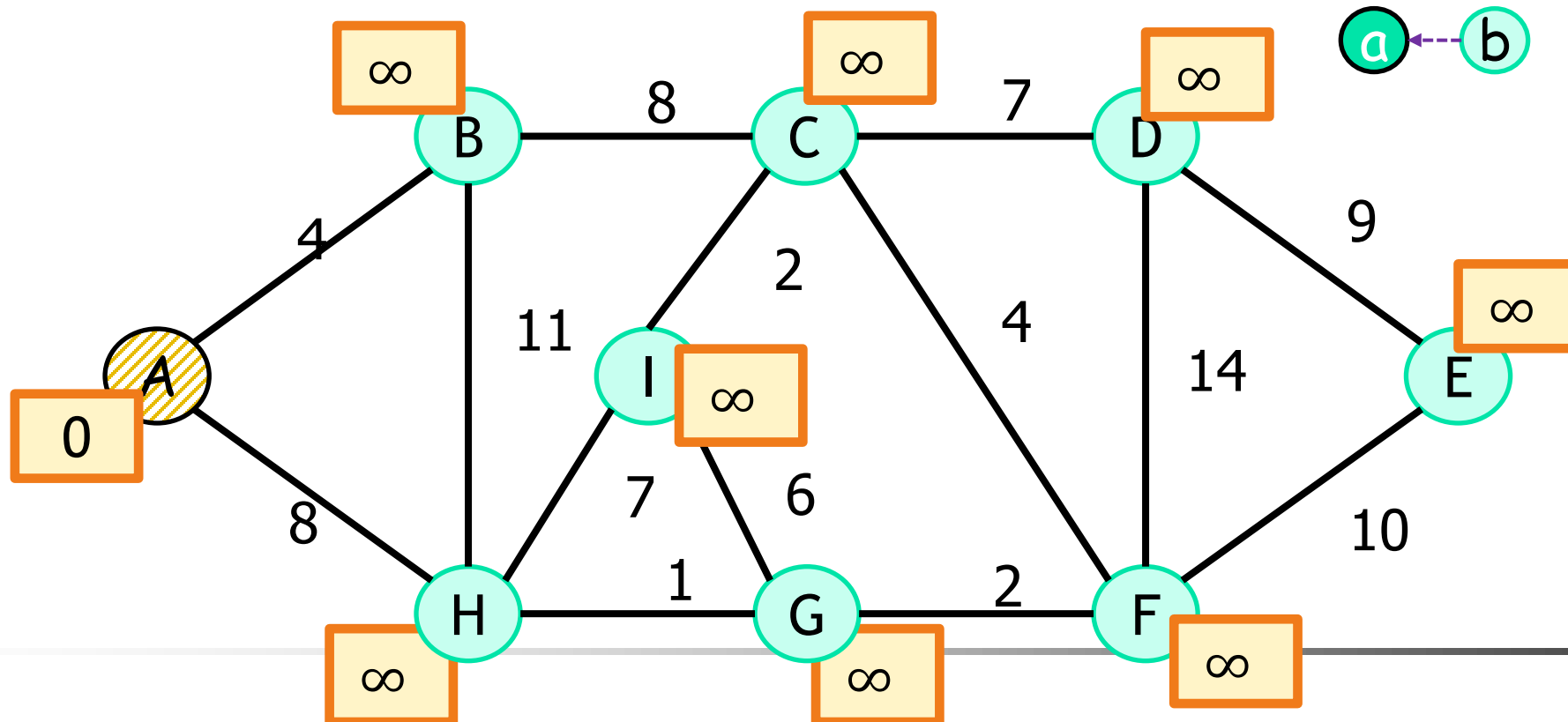
Until all the vertices are **reached**:

- Activate the **unreached** vertex u with the **smallest key**.



 $k[x]$ is the distance of x from the growing tree

 $p[b] = a$, meaning that a was the vertex that $k[b]$ comes from.

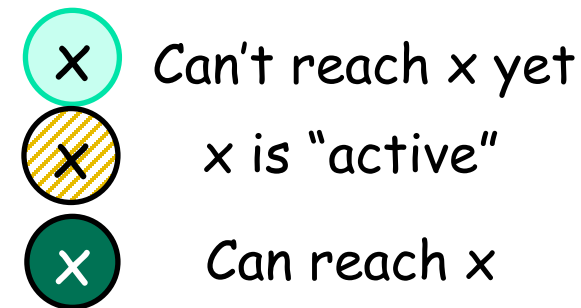


Efficient implementation

Every vertex has a key and a parent

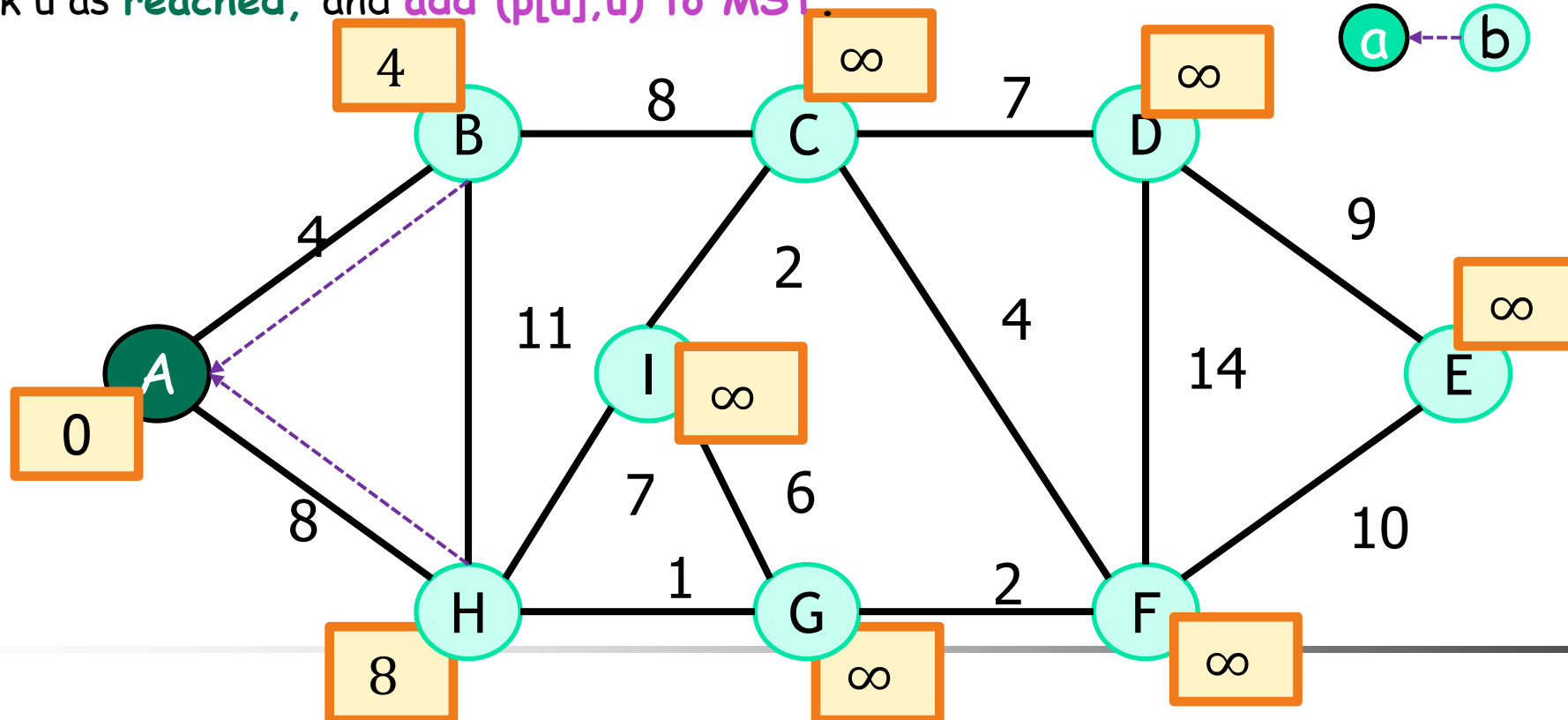
Until all the vertices are **reached**:

- Activate the **unreached** vertex u with the **smallest key**.
- **for each** of u 's neighbors v :
 $k[v] = \min(k[v], \text{weight}(u,v))$
 if $k[v]$ updated, $p[v] = u$
- Mark u as **reached**, and **add $(p[u], u)$ to MST**.



$k[x]$ is the distance of x from the growing tree

$p[b] = a$, meaning that a was the vertex that $k[b]$ comes from.

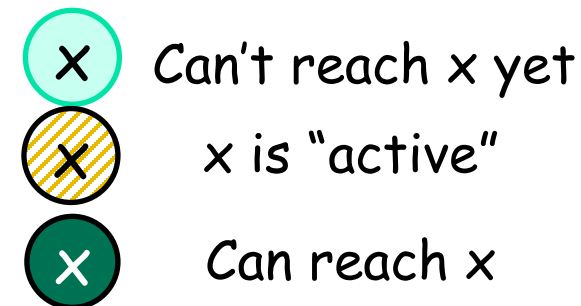


Efficient implementation

Every vertex has a key and a parent

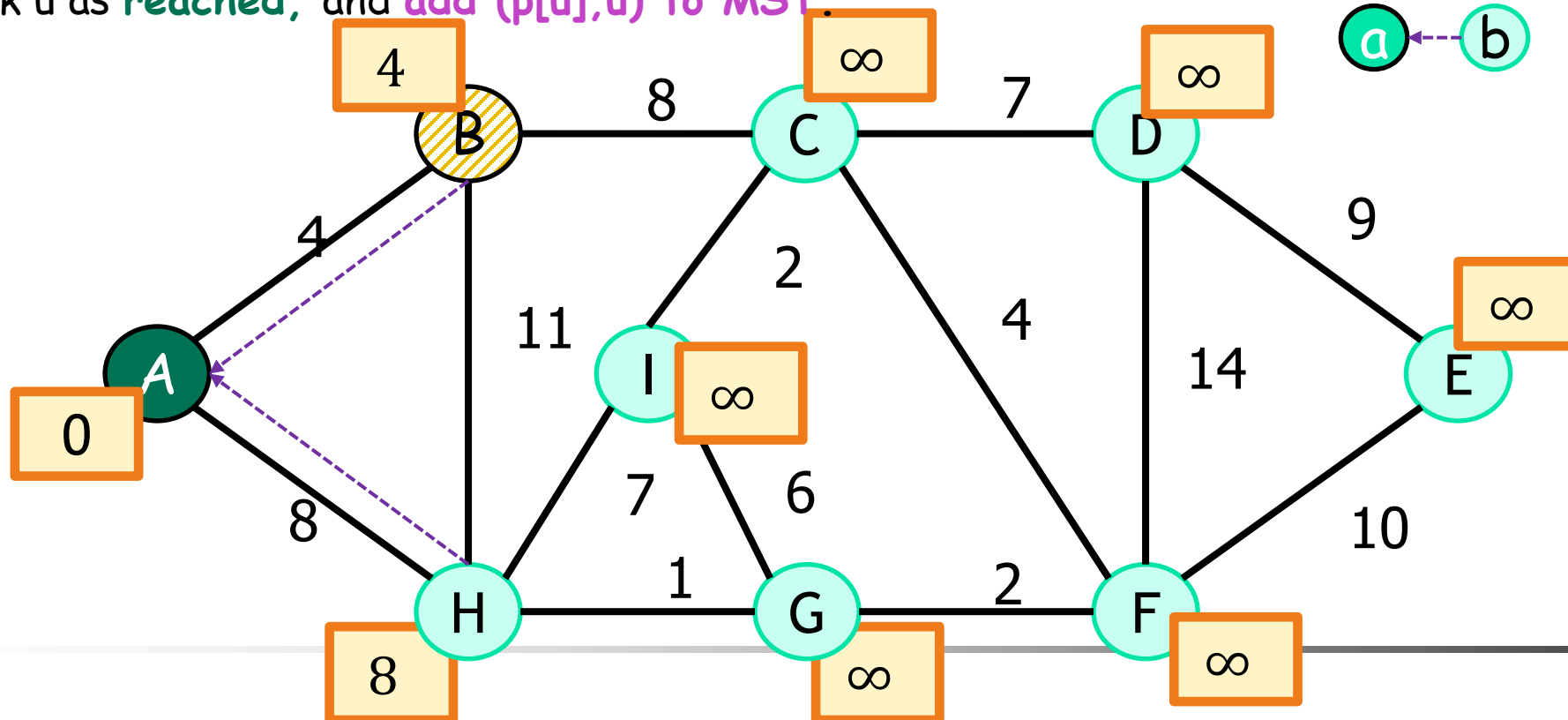
Until all the vertices are **reached**:

- Activate the **unreached** vertex u with the **smallest key**.
- **for each** of u 's neighbors v :
 $k[v] = \min(k[v], \text{weight}(u,v))$
 if $k[v]$ updated, $p[v] = u$
- Mark u as **reached**, and **add $(p[u], u)$ to MST**.



$k[x]$ $k[x]$ is the distance of x from the growing tree

a $\leftarrow$ b $p[b] = a$, meaning that a was the vertex that $k[b]$ comes from.

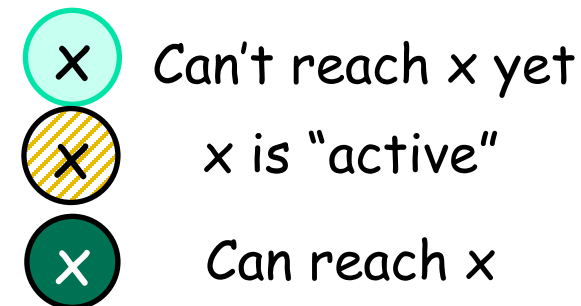


Efficient implementation

Every vertex has a key and a parent

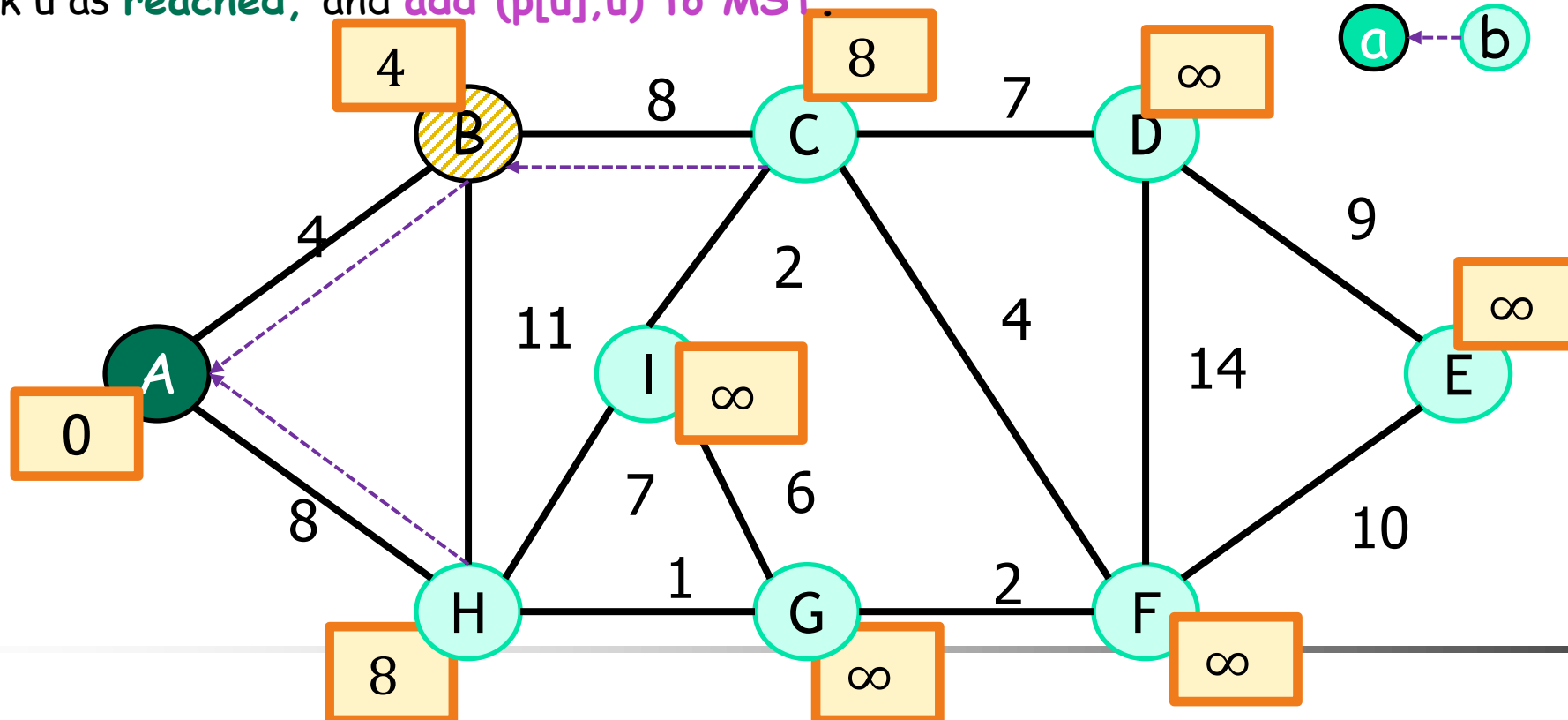
Until all the vertices are **reached**:

- Activate the **unreached** vertex u with the **smallest key**.
- **for each** of u 's neighbors v :
 $k[v] = \min(k[v], \text{weight}(u,v))$
 if $k[v]$ updated, $p[v] = u$
- Mark u as **reached**, and **add $(p[u], u)$ to MST**.



 $k[x]$ is the distance of x from the growing tree

 $p[b] = a$, meaning that a was the vertex that $k[b]$ comes from.

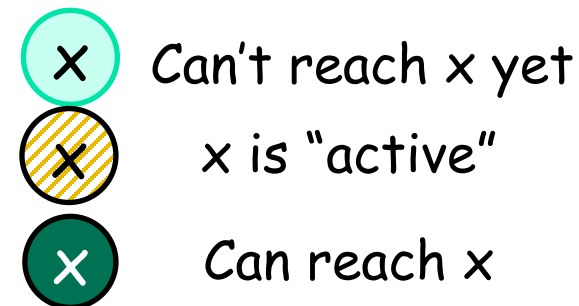


Efficient implementation

Every vertex has a key and a parent

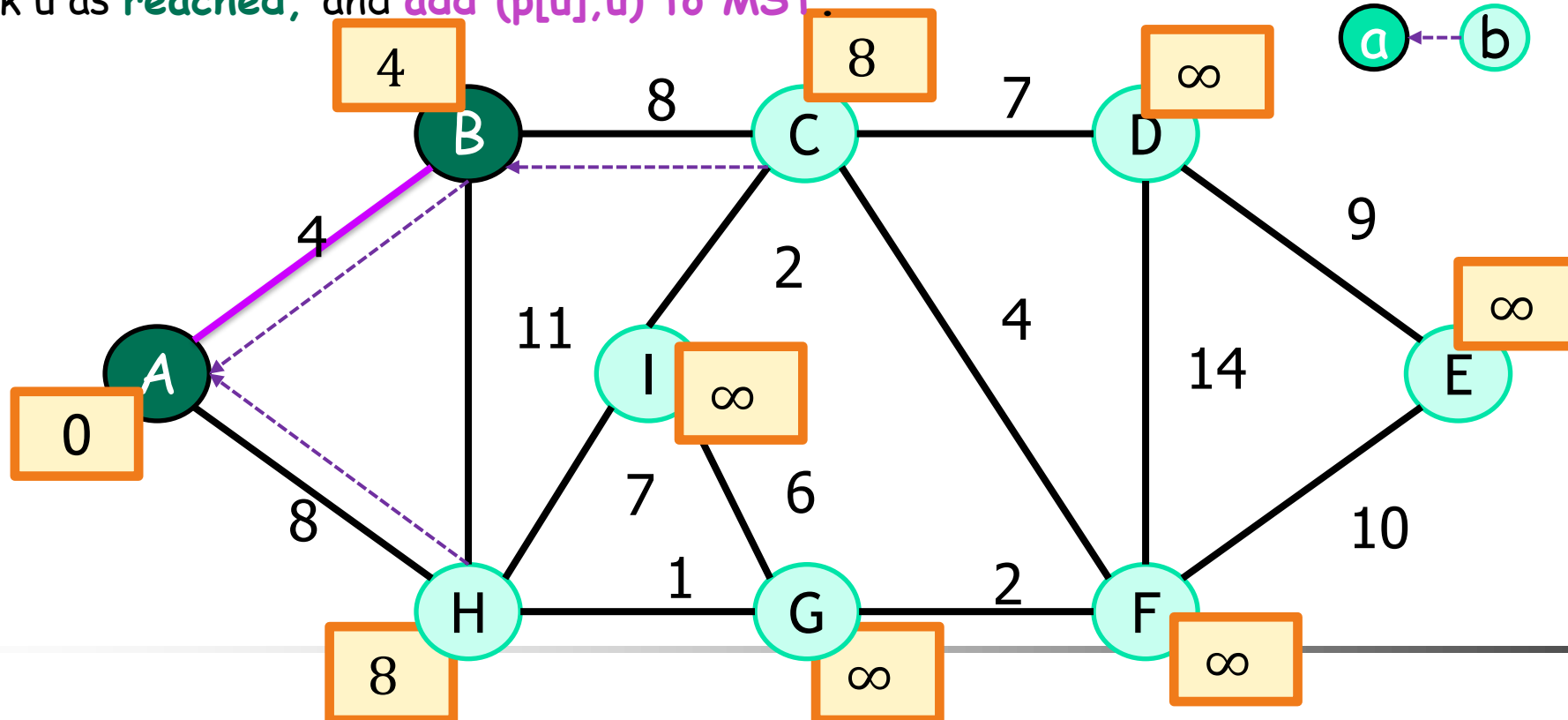
Until all the vertices are **reached**:

- Activate the **unreached** vertex u with the **smallest key**.
- **for each** of u 's neighbors v :
 $k[v] = \min(k[v], \text{weight}(u,v))$
 if $k[v]$ updated, $p[v] = u$
- Mark u as **reached**, and **add $(p[u], u)$ to MST**.



$k[x]$ $k[x]$ is the distance of x from the growing tree

a $\leftarrow$ b $p[b] = a$, meaning that a was the vertex that $k[b]$ comes from.

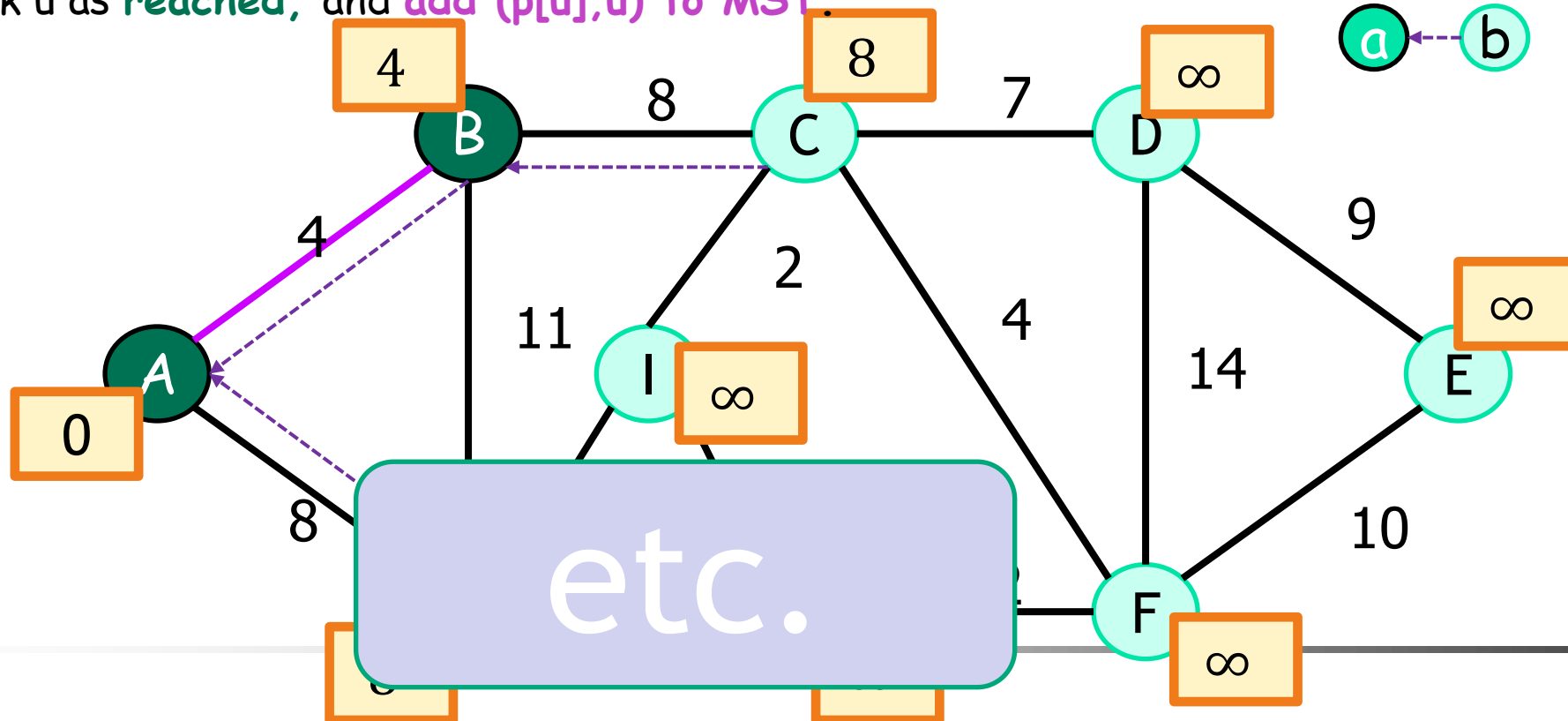
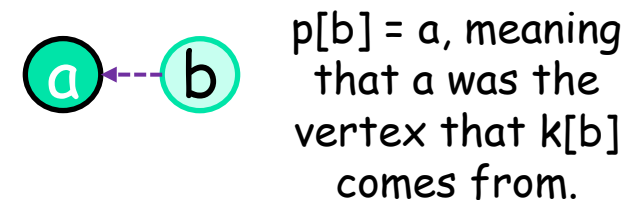
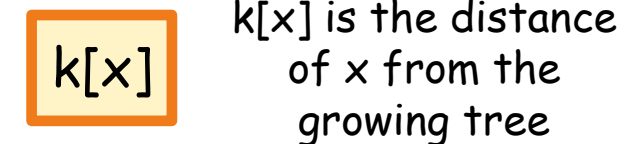
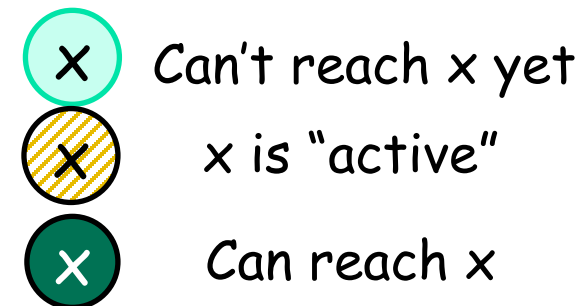


Efficient implementation

Every vertex has a key and a parent

Until all the vertices are **reached**:

- Activate the **unreached** vertex u with the **smallest key**.
- **for each** of u 's neighbors v :
 $k[v] = \min(k[v], \text{weight}(u,v))$
 if $k[v]$ updated, $p[v] = u$
- Mark u as **reached**, and **add $(p[u], u)$ to MST**.



Prim's algorithm

Prim($G = (V, E)$, starting vertex s):

$\text{key}(v) = \infty, \forall v \in V$

$\text{key}(s) = 0$

$Q = (\text{key}(v), v), \forall v \in V$

$p(v) = \text{NULL}, \forall v \in V$

$A = \emptyset$

while Q is not empty:

$u = \text{ExtractMin}(Q)$

if $u \neq s$ **then**

$A = A \cup \{(p(u), u)\}$

for each neighbor v of u :

if $v \in Q$ and $w(u, v) < \text{key}(v)$:

$\text{key}(v) = w(u, v)$

$\text{DecreaseKey}(\text{key}(v), v)$

$p(v) = u$

return A

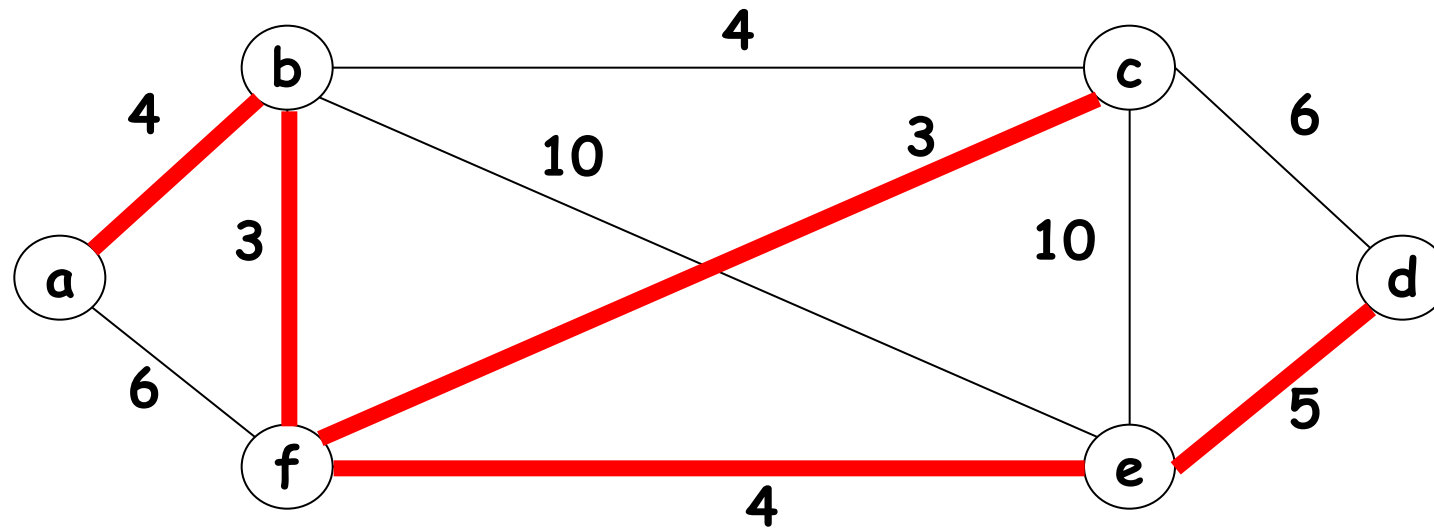
If a binary min-heap is used:

ExtractMin: $O(\log n)$

DecreaseKey: $O(\log n)$

Total: $O(n \log n + m \log n) = O(m \log n)$

Exercise – Find MST for this graph



order of (edges) selection: (a,b), (b,f), (c,f), (f,e), (e,d)

Prim and Kruskal

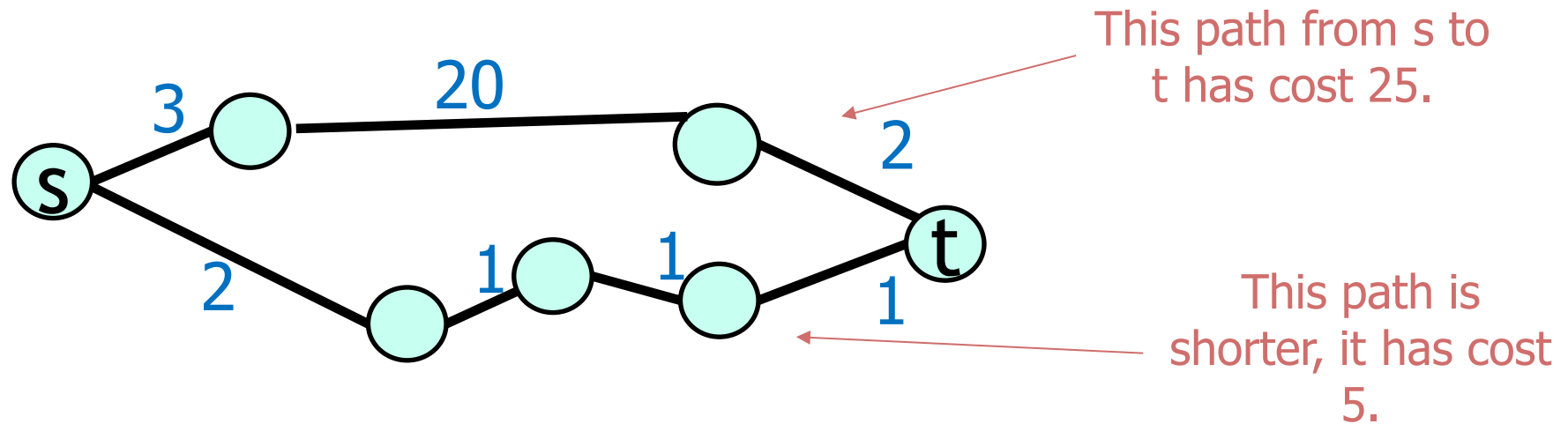
- Both Prim and Kruskal are greedy algorithms for MST.
- Kruskal:
 - Grows a forest.
 - Time $O(m \log n)$ with a disjoint-set data structure
- Prim:
 - Grows a tree.
 - Time $O(m \log n)$ with a binary min-heap
 - Time $O(m + n \log n)$ with a Fibonacci heap*

* Refer to textbook Chapter 19

Shortest path

Shortest path problem

- What is the **shortest path** between u and v in a weighted graph?
 - the **cost** of a path is the sum of the weights along that path
 - The **shortest path** is the one with the minimum cost.



- The **distance** $d(u,v)$ between two vertices u and v is the cost of the the shortest path between u and v .

Single-source shortest-paths

Consider a directed connected graph G

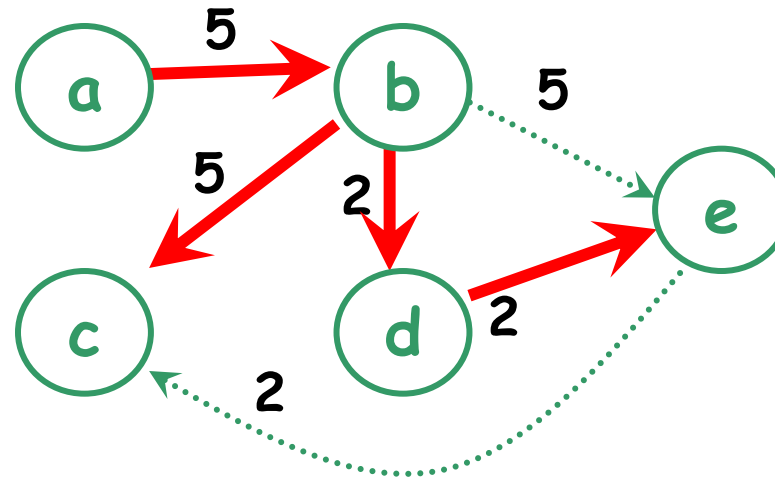
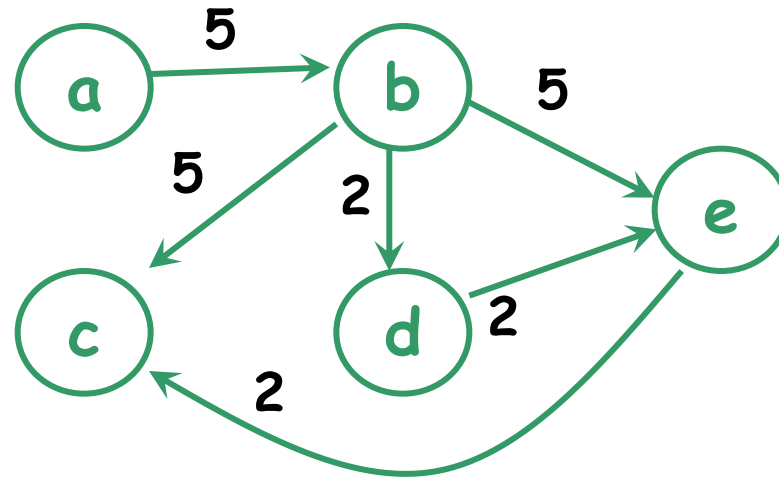
- The edges are labelled by weight

Given a particular vertex called the source

- Find shortest paths from the source to all other vertices (shortest path means the total weight of the path is the smallest)

Example

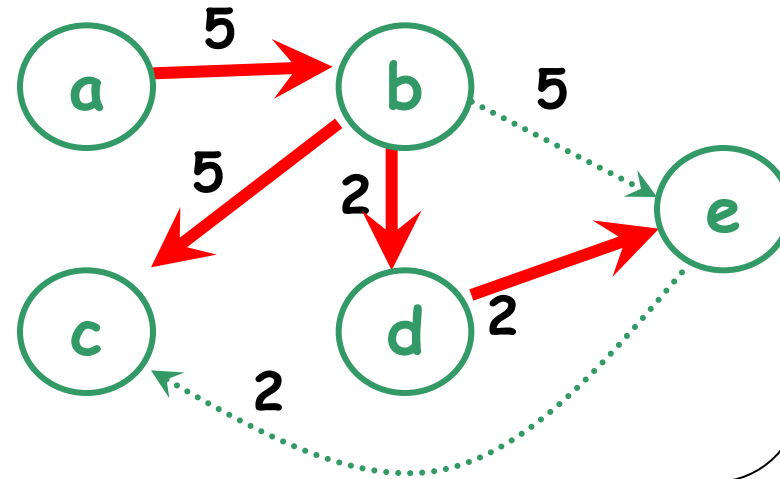
Directed Graph G
(edge label is weight)
a is source vertex



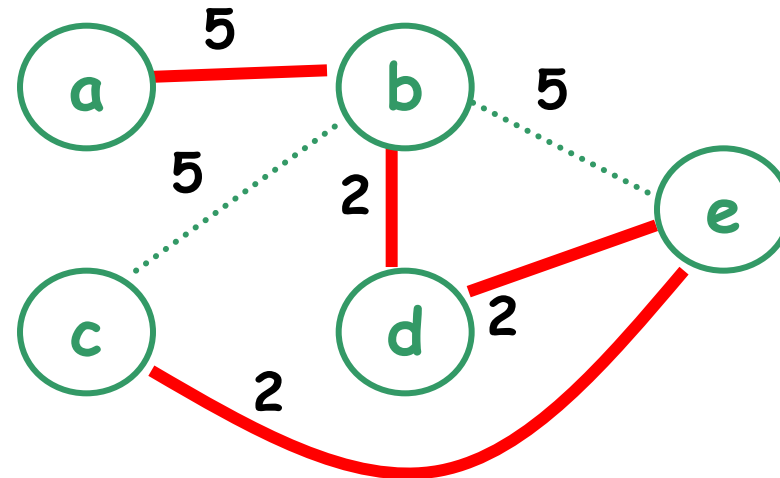
thick lines: shortest path
dotted lines: not in shortest path

Single-source shortest paths vs MST

Shortest paths from a



What is the difference between MST and shortest paths from a?

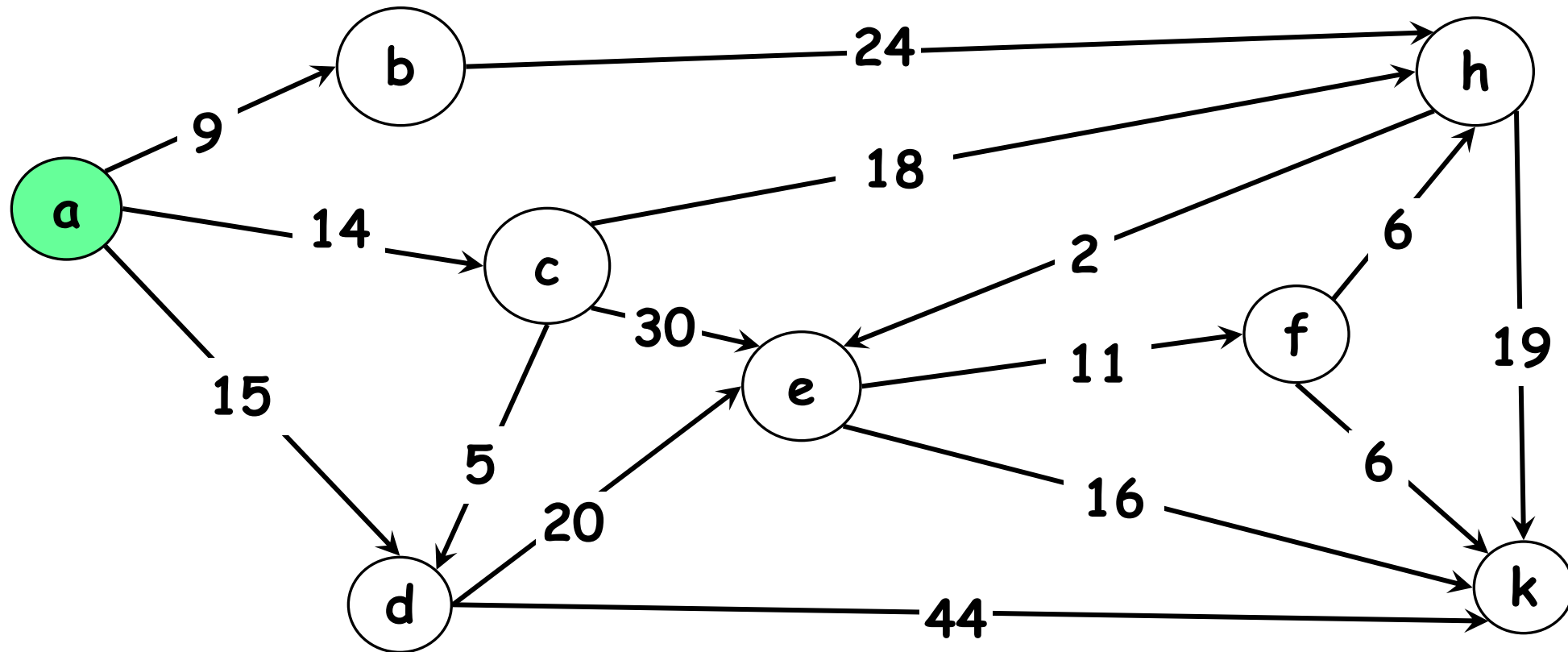


MST

Dijkstra algorithm

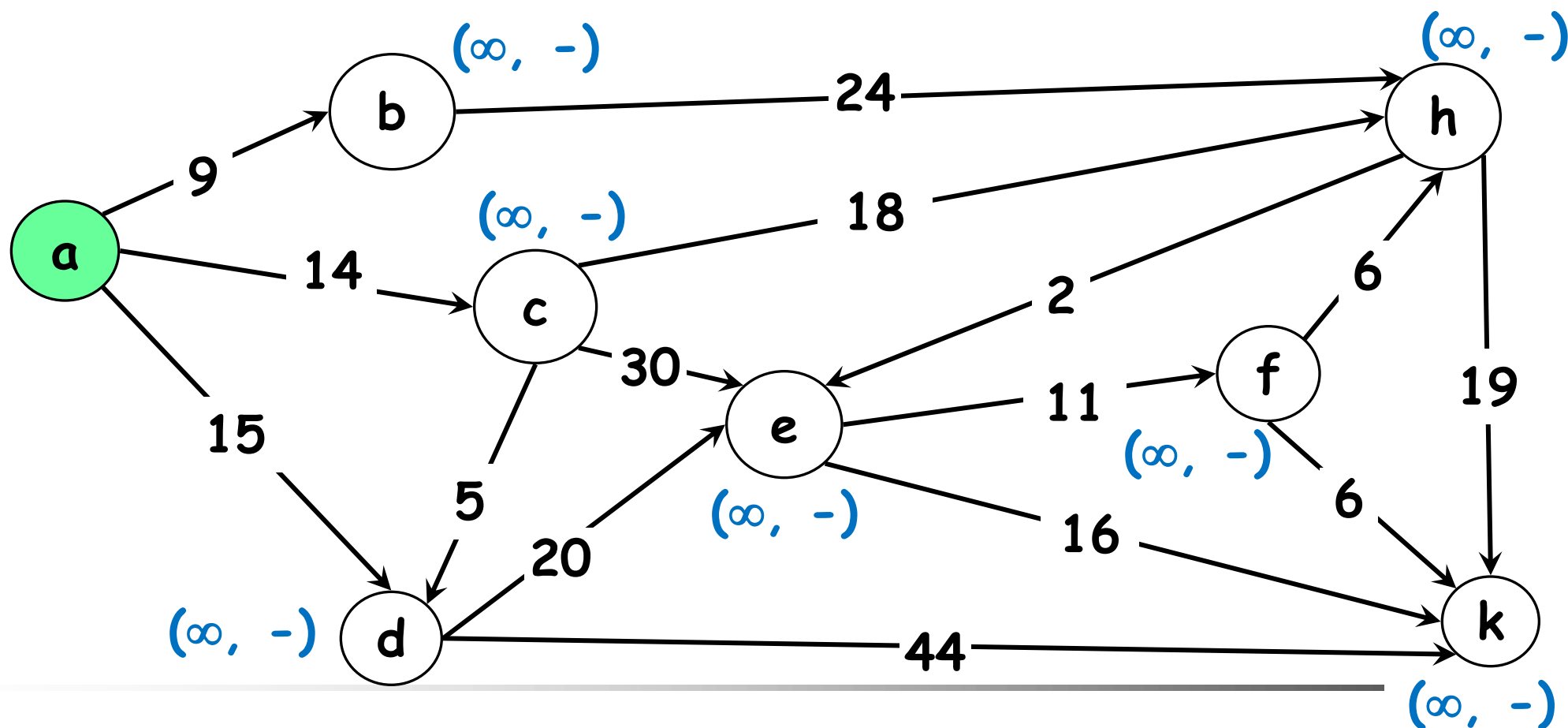
Dijkstra's algorithm

Suppose vertex **a** is the source, we now show how Dijkstra's algorithm works



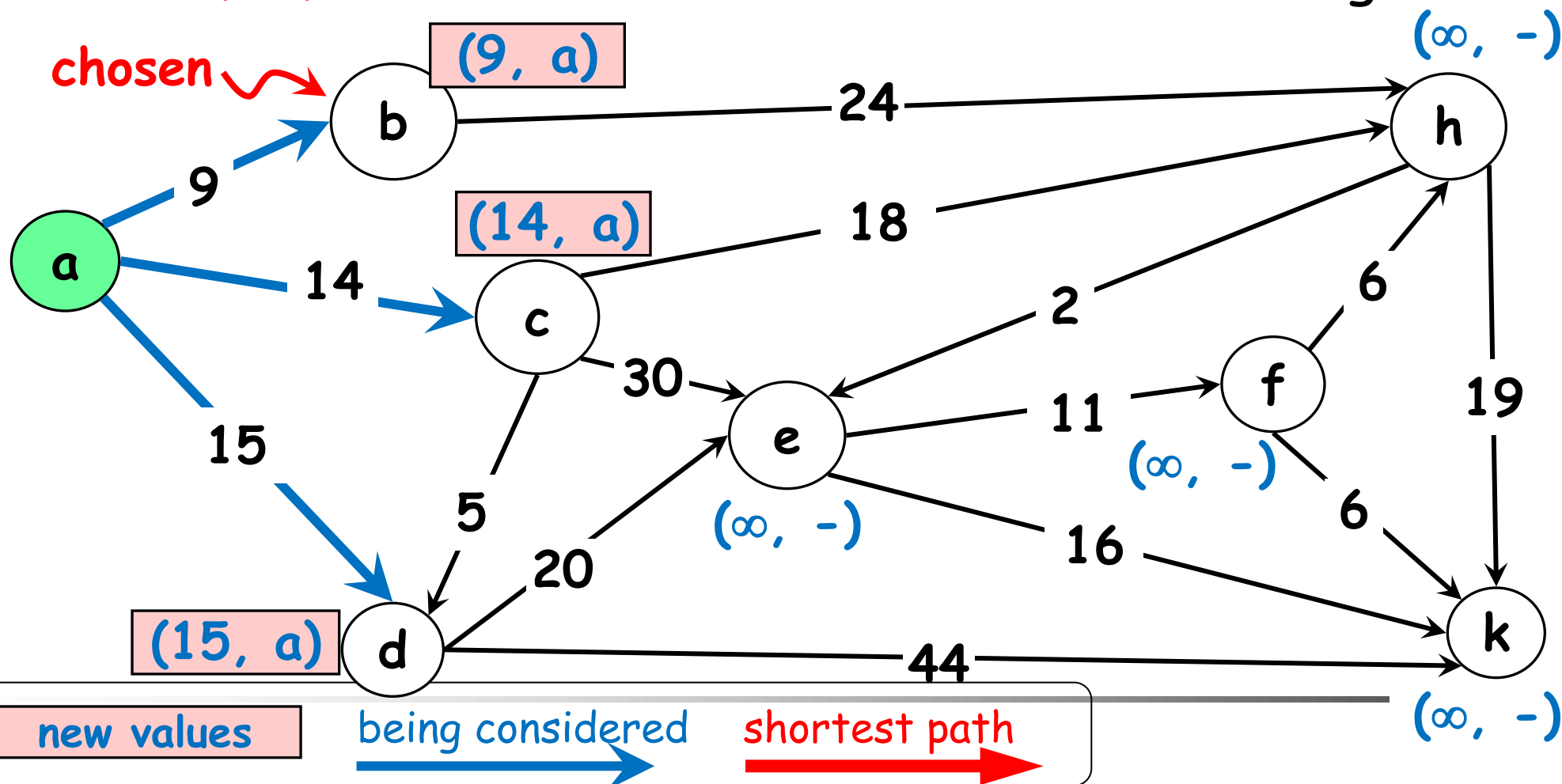
Dijkstra's algorithm

Every vertex v keeps 2 labels: (1) the weight of the current shortest path from a ; (2) the vertex leading to v on that path, initially as $(\infty, -)$



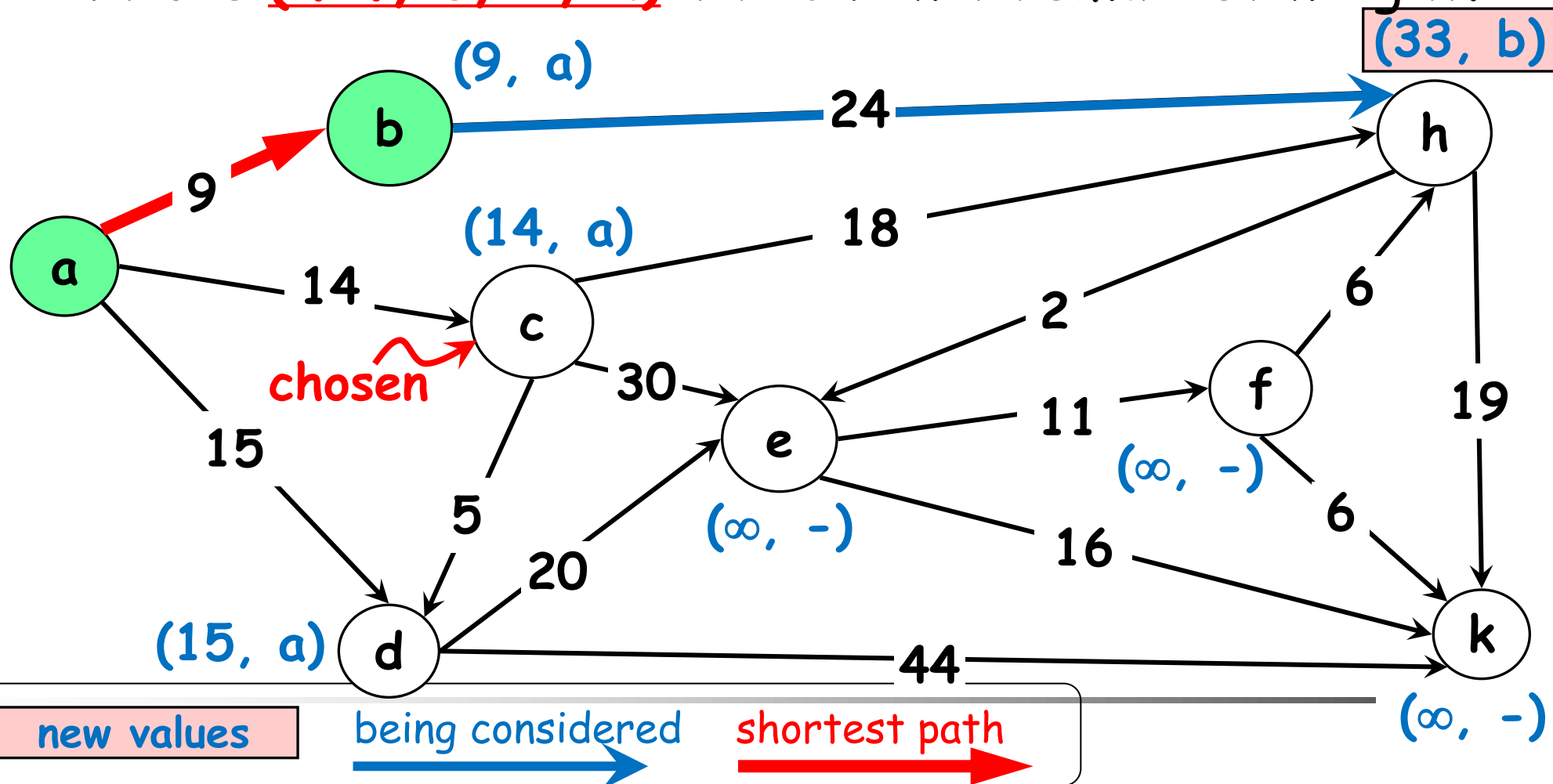
Dijkstra's algorithm

For every neighbor u of a , update the weight to the weight of (a, u) and the leading vertex to a . Choose from b, c, d the one with the smallest such weight.



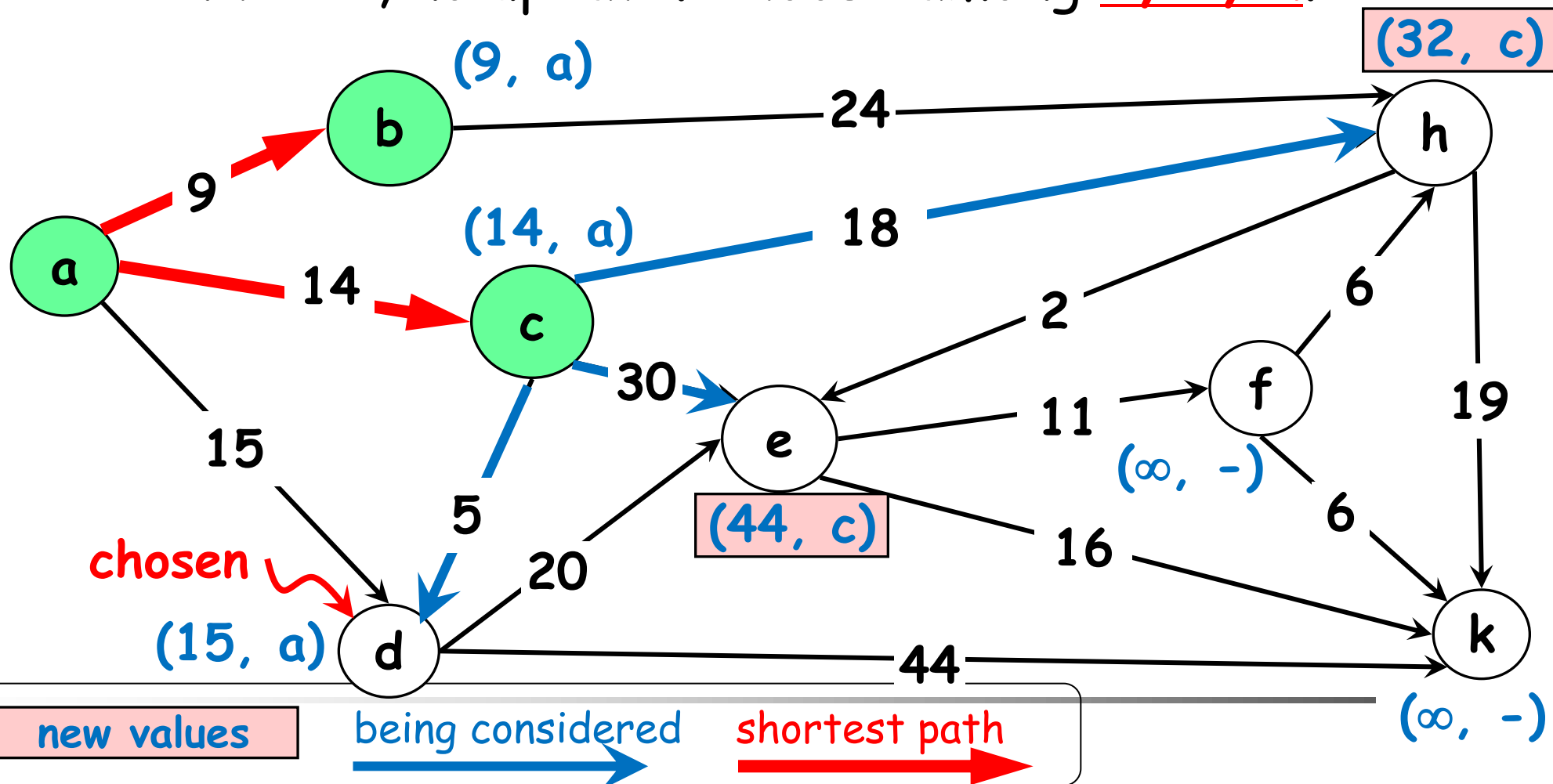
Dijkstra's algorithm

For every un-chosen neighbor of vertex **b**, update the weight and leading vertex. Choose from **ALL** un-chosen vertices (*i.e.*, **c, d, h**) the one with smallest weight.



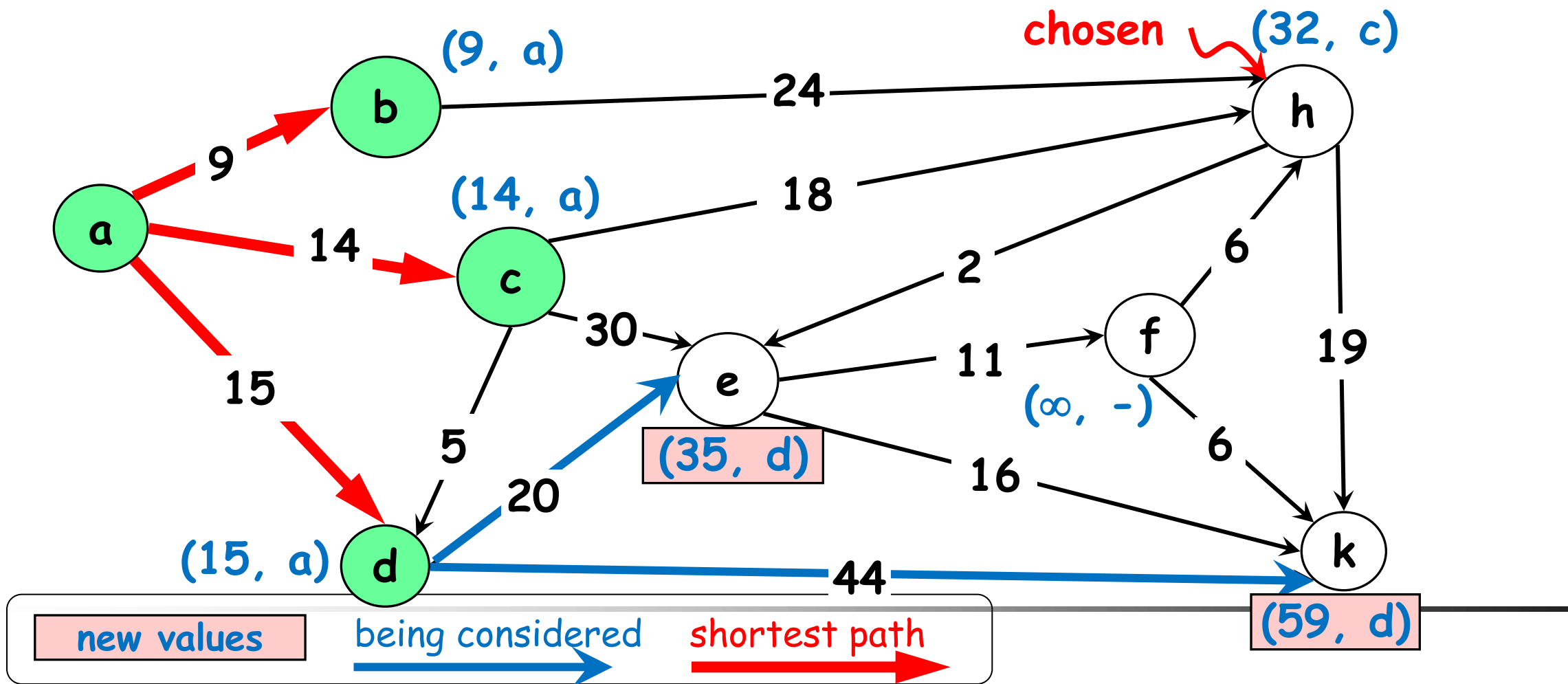
Dijkstra's algorithm

If a new path with smallest weight is discovered, e.g., for vertices **e**, **h**, the weight is updated. Otherwise, like vertex **d**, no update. Choose among **d**, **e**, **h**.



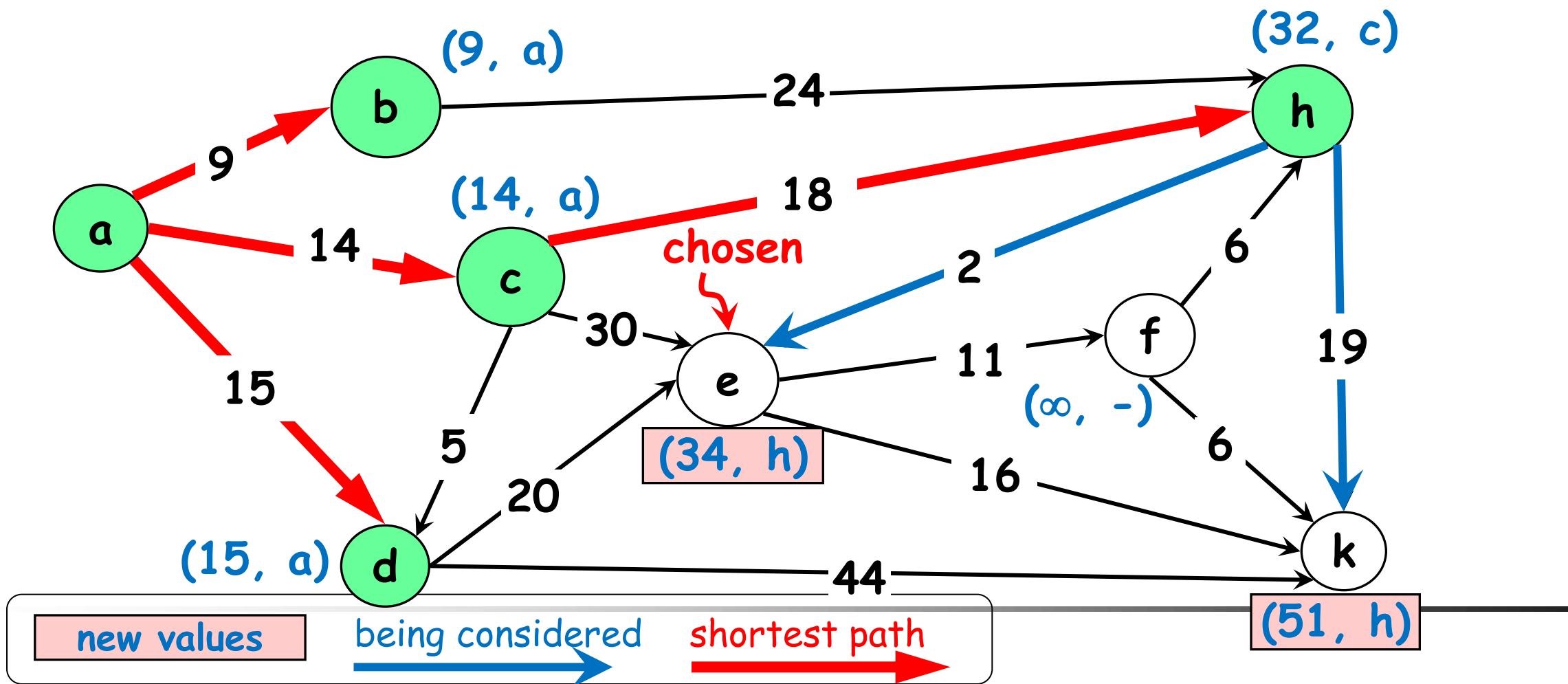
Dijkstra's algorithm

Repeat the procedure. After **d** is chosen, the weight of **e** and **k** is updated. Choose among **e**, **h**, **k**. Next vertex chosen is **h**.



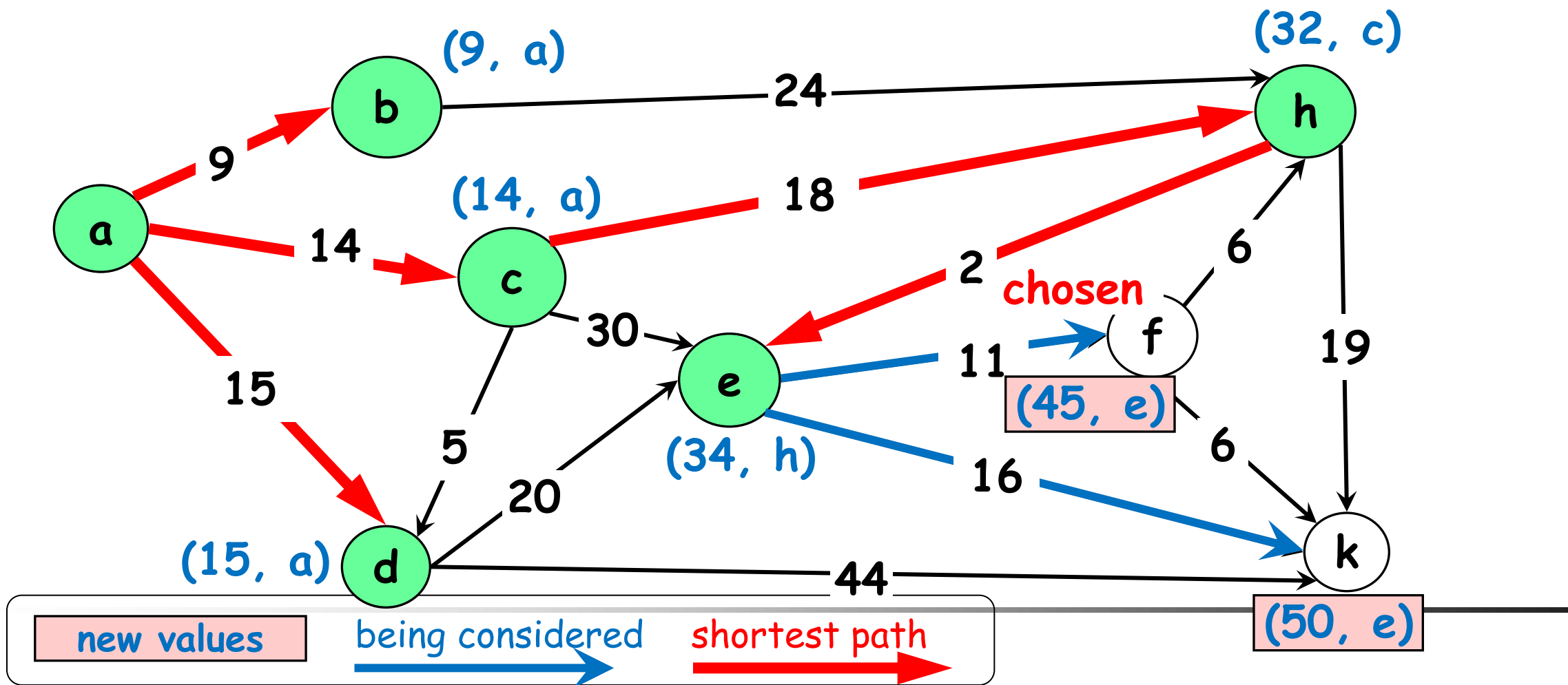
Dijkstra's algorithm

After **h** is chosen, the weight of **e** and **k** is updated again. Choose among **e**, **k**. Next vertex chosen is **e**.



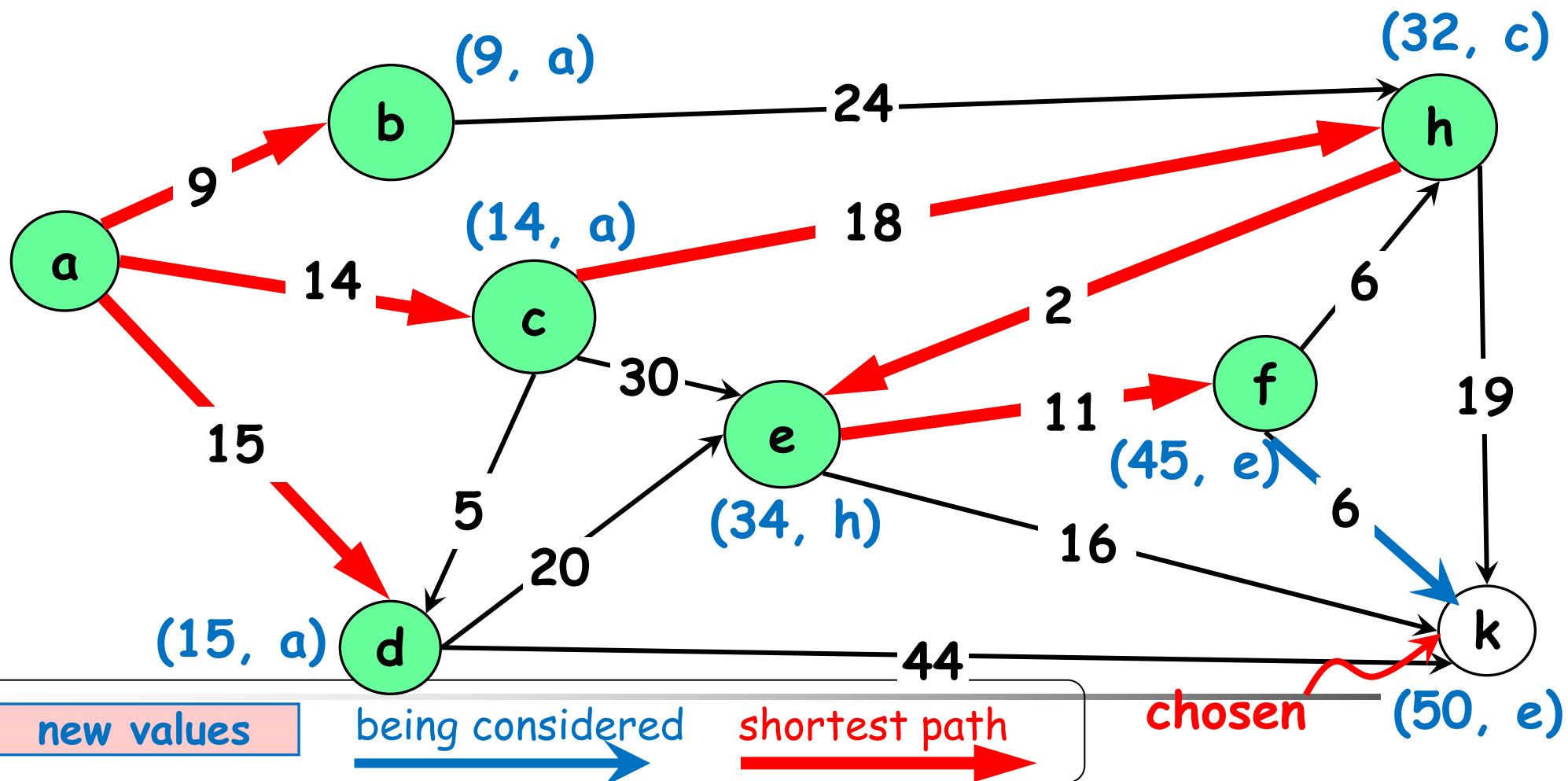
Dijkstra's algorithm

After **e** is chosen, the weight of **f** and **k** is updated again. Choose among **f**, **k**. Next vertex chosen is **f**.



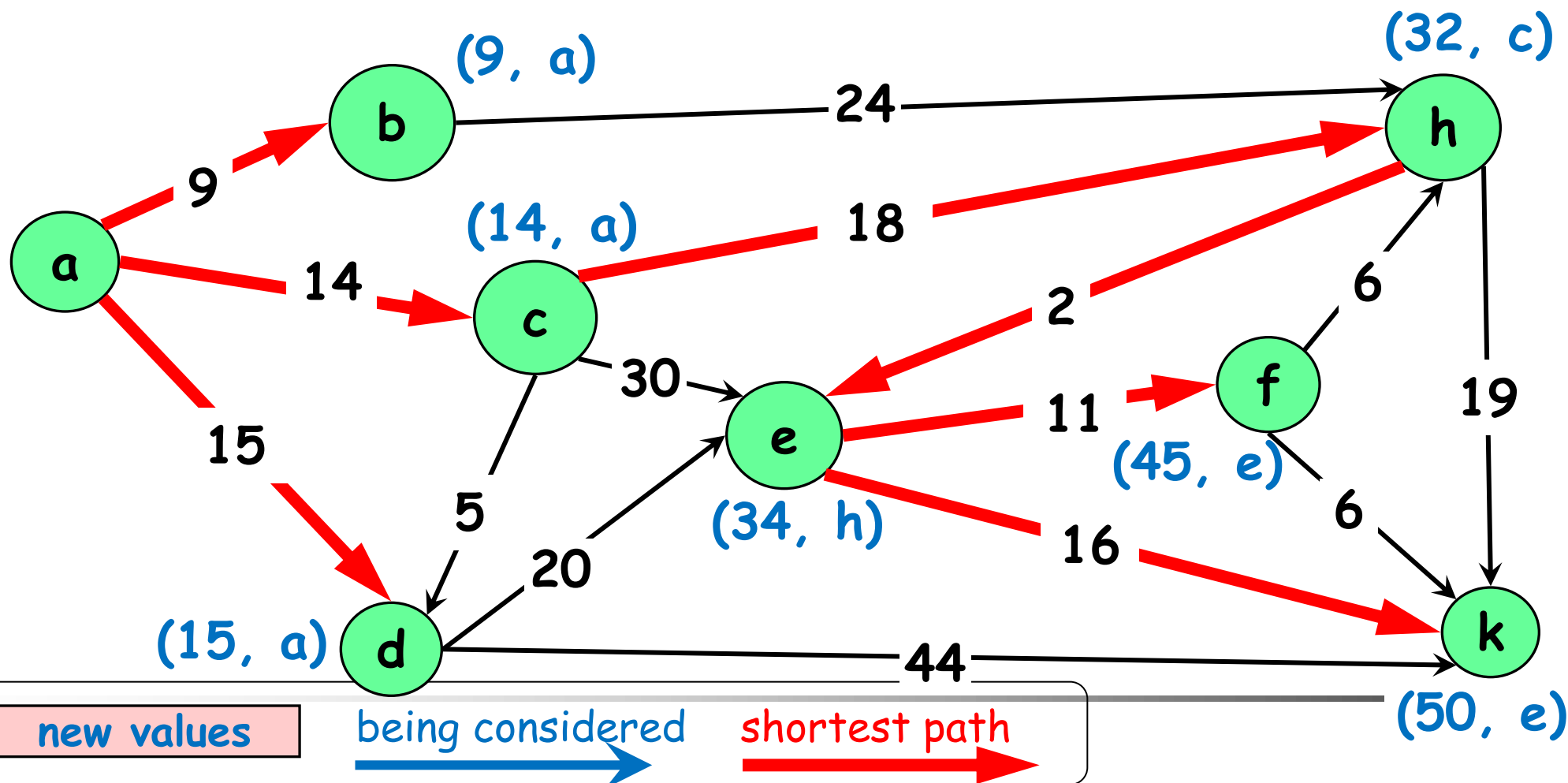
Dijkstra's algorithm

After **f** is chosen, it is NOT necessary to update the weight of **k**. The final vertex chosen is **k**.

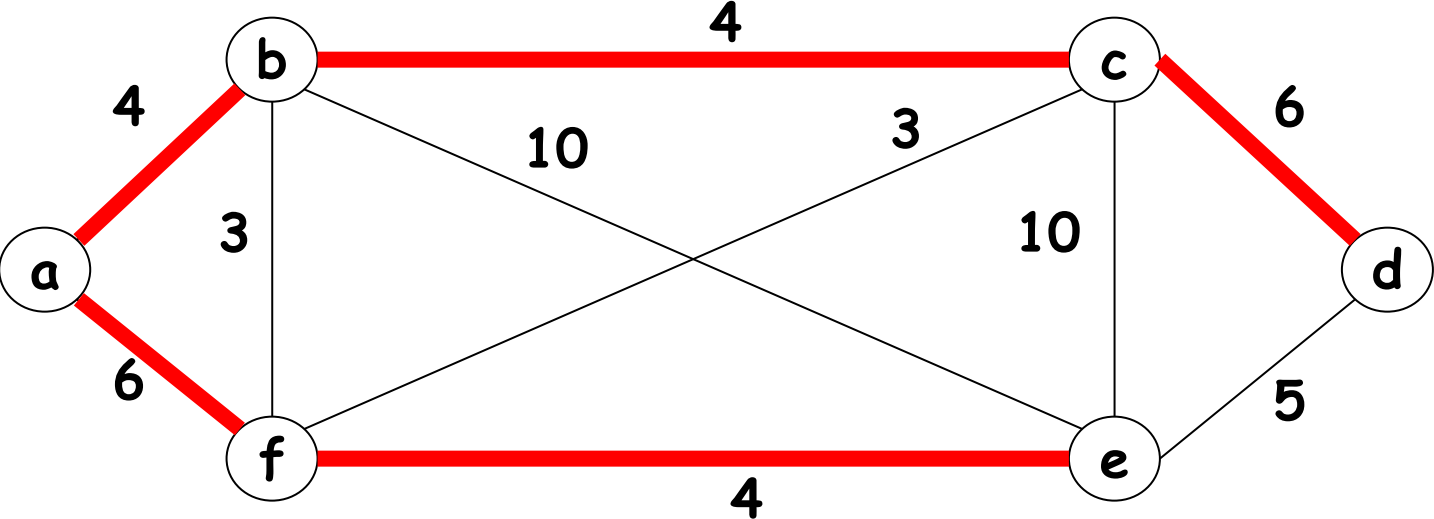


Dijkstra's algorithm

At this point, all vertices are chosen, and the shortest path from **a** to every vertex is discovered.



Exercise – Shortest paths from a



Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	∅ 4	/ a
c	∅ 8	/ b
d	∅ 14	/ c
e	∅ 14 10	/ b f
f	∅ 6	/ a

order of (edges) selection: (a,b), (a,f), (b,c), (f,e), (c,d)

Dijkstra's algorithm

Dijkstra($G = (V, E)$, starting vertex s):

$\text{key}(v) = \infty, \forall v \in V$

$\text{key}(s) = 0$

$Q = (\text{key}(v), v), \forall v \in V$

$p(v) = \text{NULL}, \forall v \in V$

$A = \emptyset$

while Q is not empty:

$u = \text{ExtractMin}(Q)$

if $u \neq s$ **then**

$A = A \cup \{(p(u), u)\}$

for each neighbor v of u :

if $v \in Q$ and $\text{key}(v) > \text{key}(u) + w(u, v)$:

$\text{key}(v) = \text{key}(u) + w(u, v)$

$\text{DecreaseKey}(\text{key}(v), v)$

$p(v) = u$

return A

If a binary min-heap is used:

ExtractMin: $O(\log n)$

DecreaseKey: $O(\log n)$

Total: $O(n \log n + m \log n) = O(m \log n)$

Bellman-Ford Algorithm

Bellman-Ford Algorithm

- Basic idea:
 - Instead of picking the u with the smallest $\text{key}(u)$ to update, just update all of the u 's by relaxation.
- Can handle negative edge weights.
- Slower than Dijkstra's algorithm

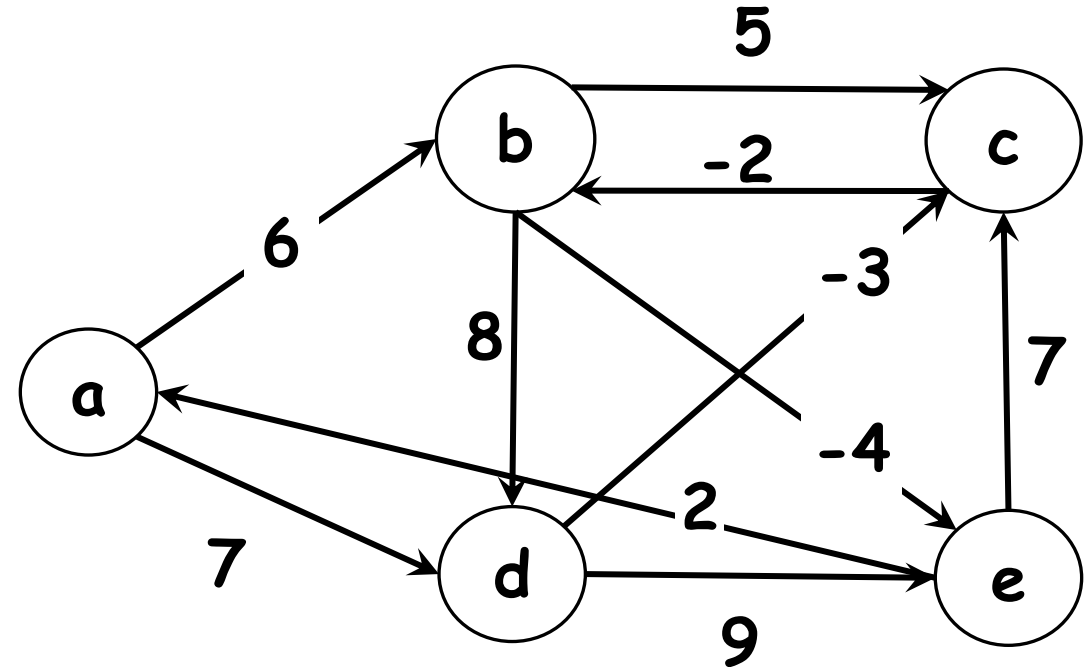
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	∞	-
c	∞	-
d	∞	-
e	∞	-



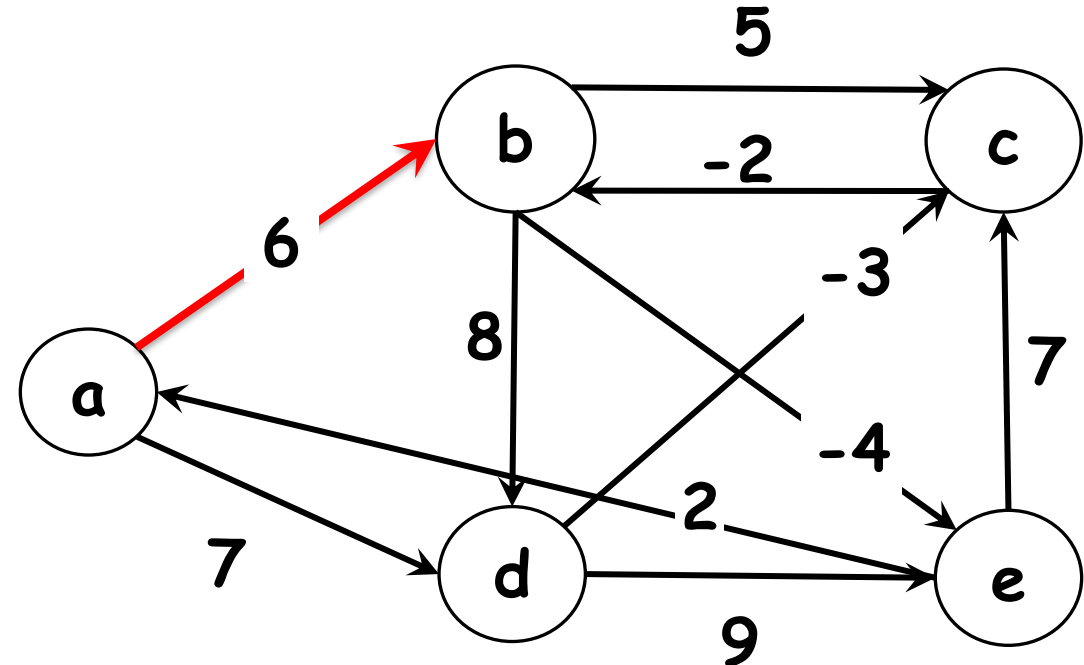
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	∞	-
d	∞	-
e	∞	-



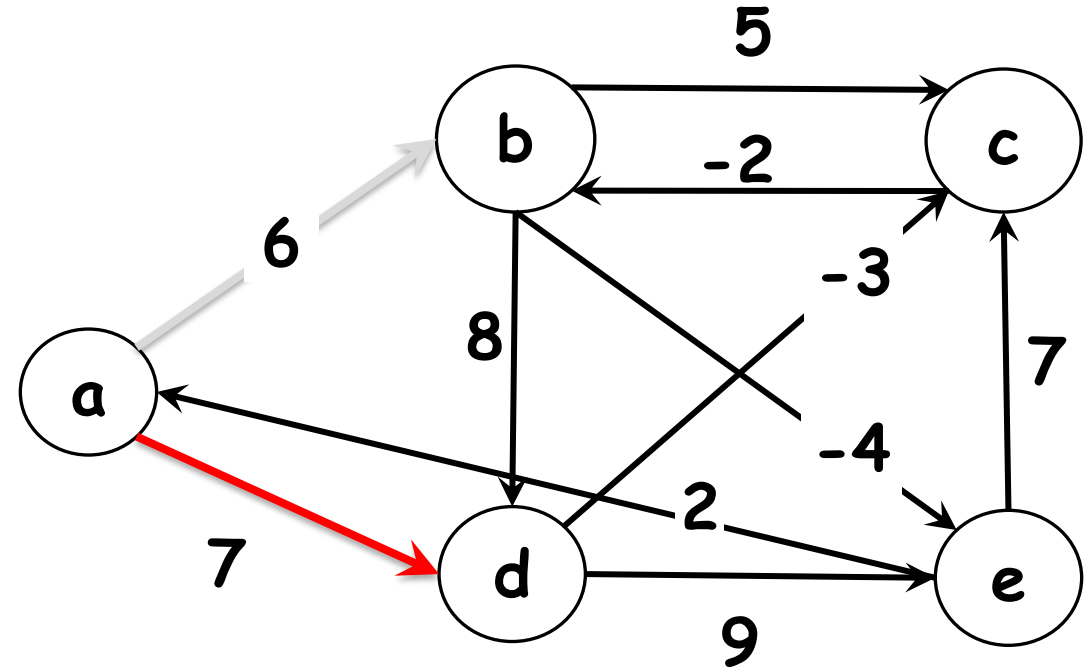
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	∞	-
d	7	a
e	∞	-



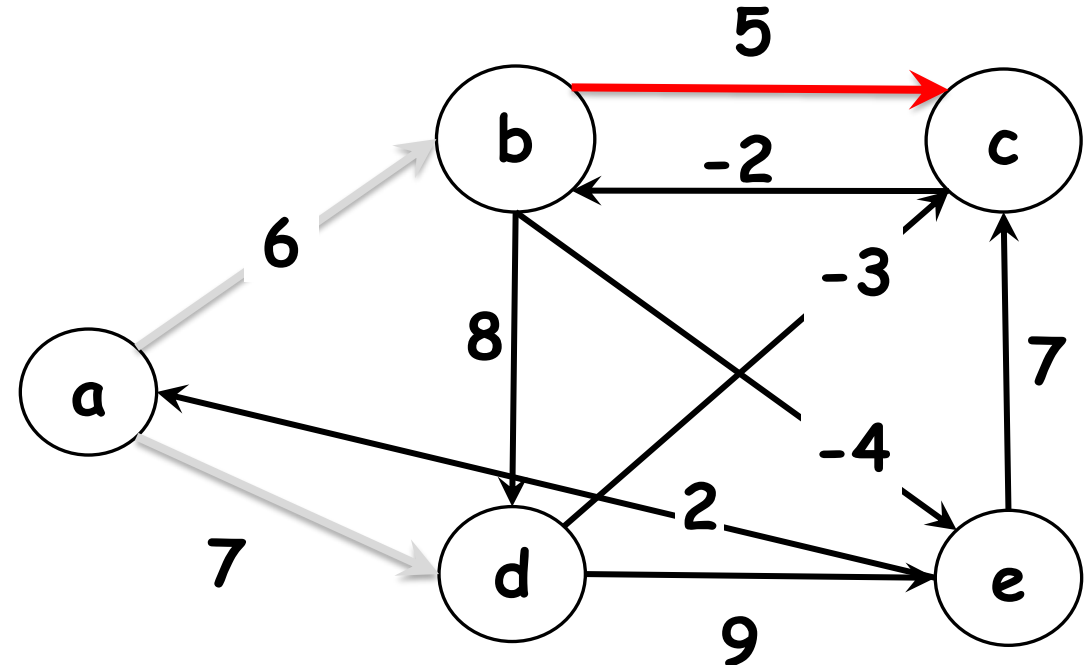
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	11	b
d	7	a
e	∞	-



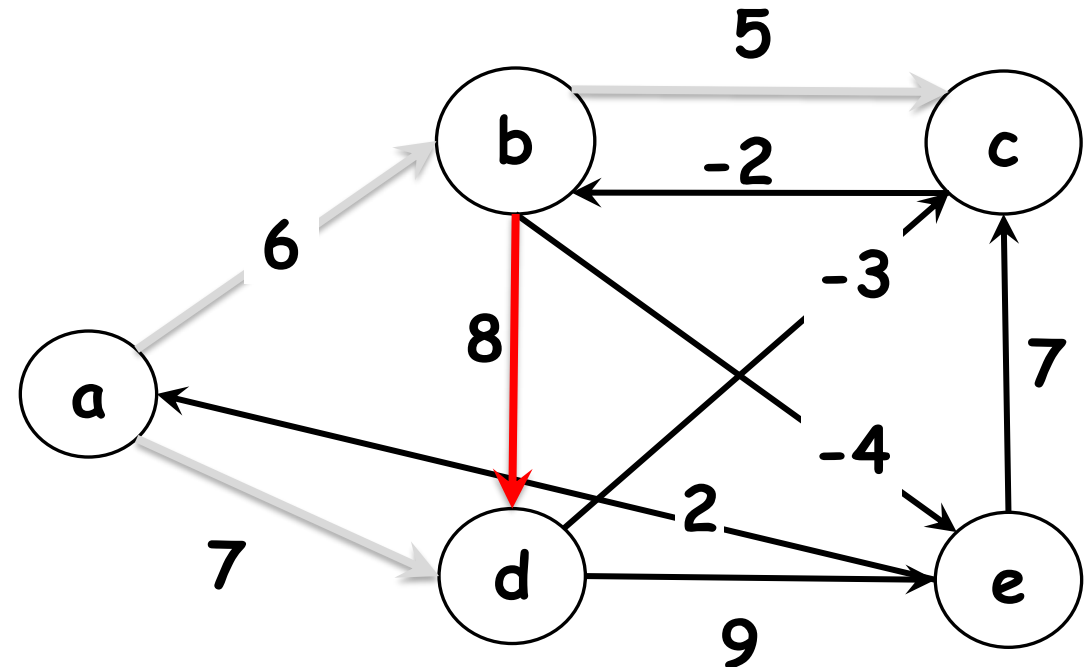
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	11	b
d	7	a
e	∞	-



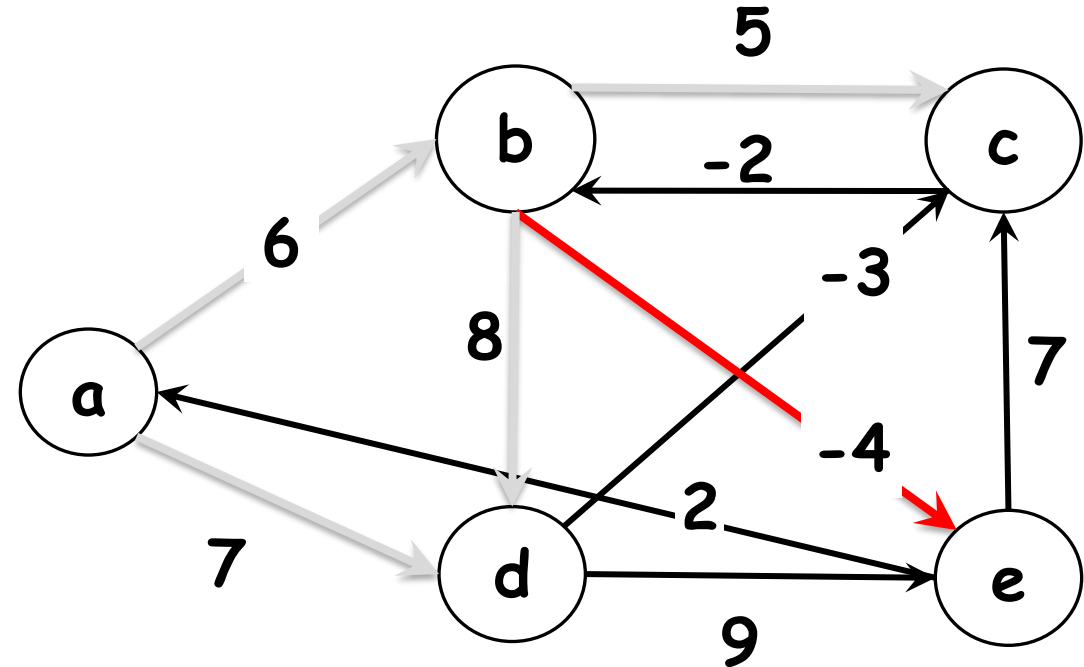
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	11	b
d	7	a
e	2	b



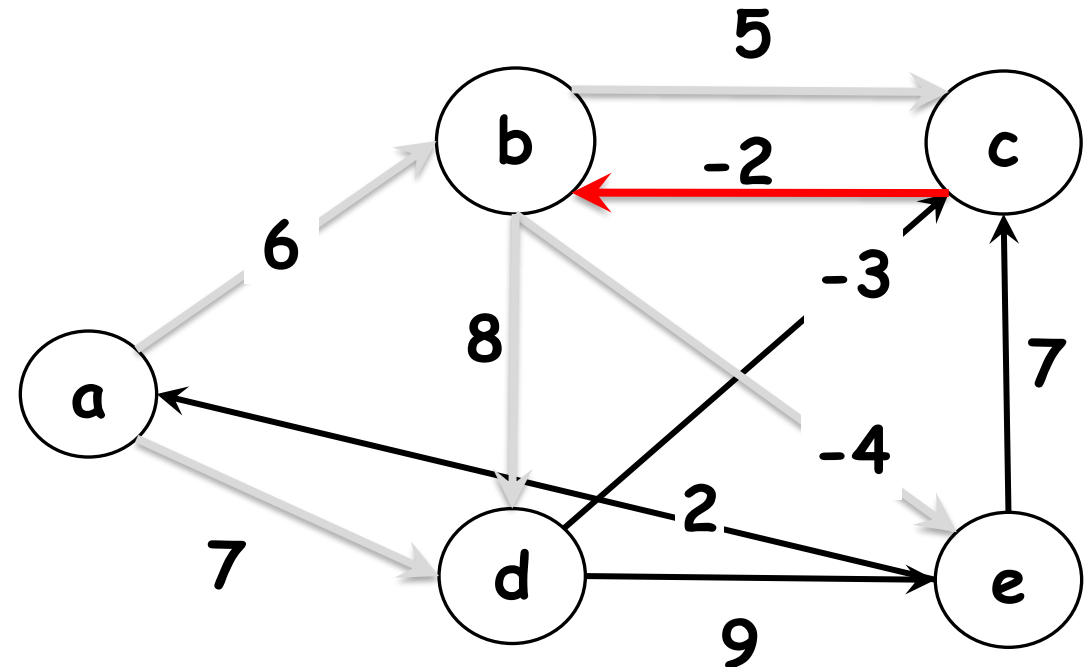
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e),$
 $(c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	11	b
d	7	a
e	2	b



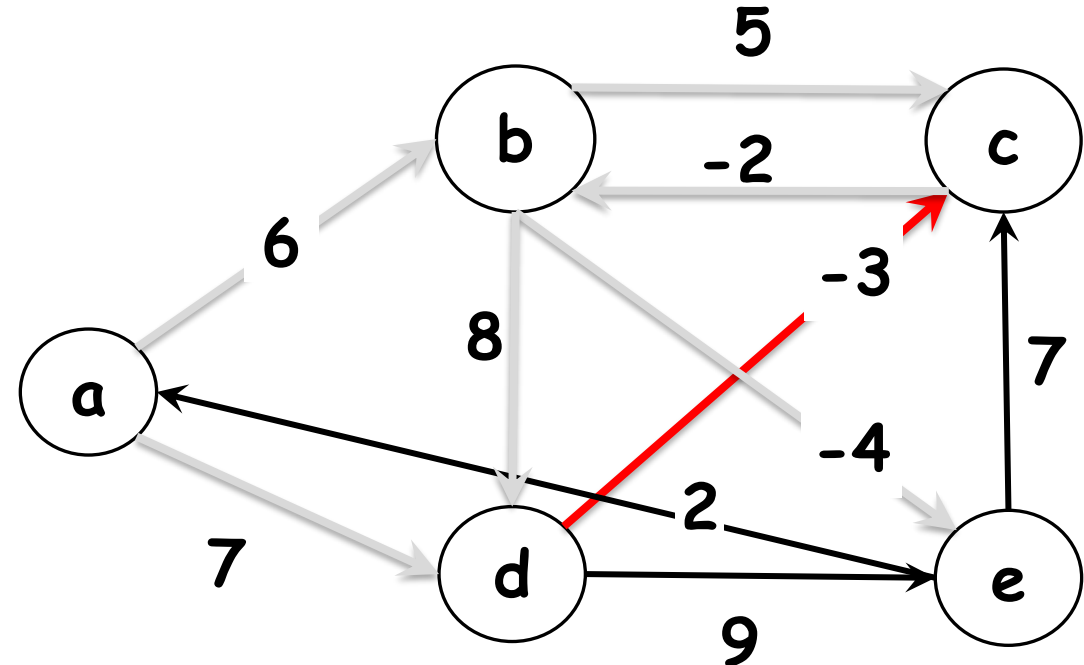
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if  $\text{key}(v) > \text{key}(u) + w(u, v)$  :  
       $\text{key}(v) = \text{key}(u) + w(u, v)$   
       $p(v) = u$ 
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



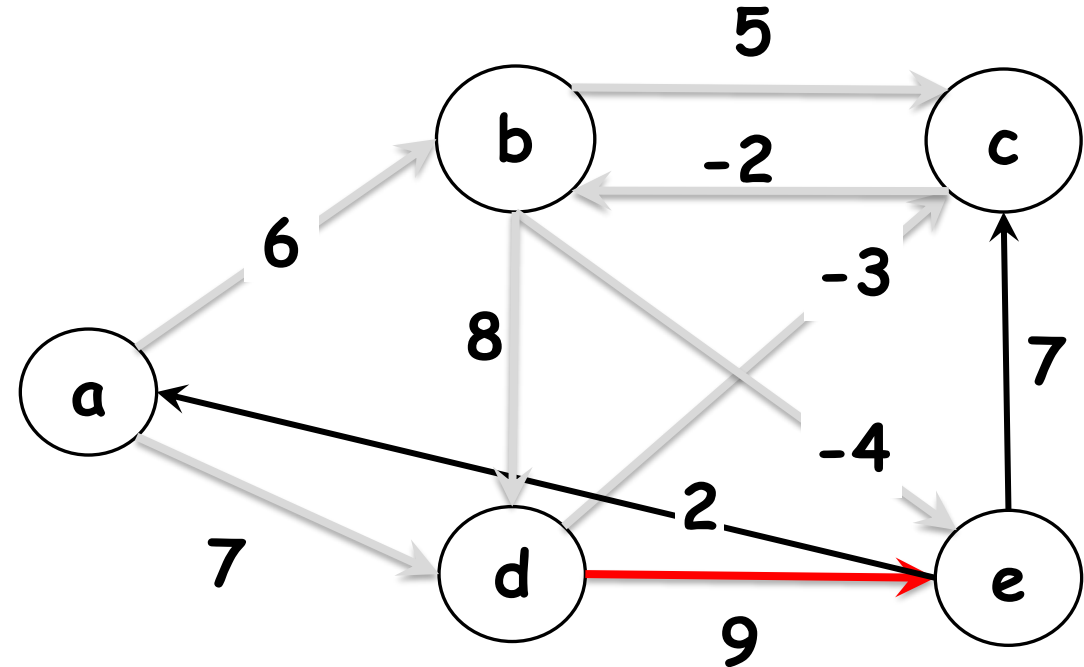
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



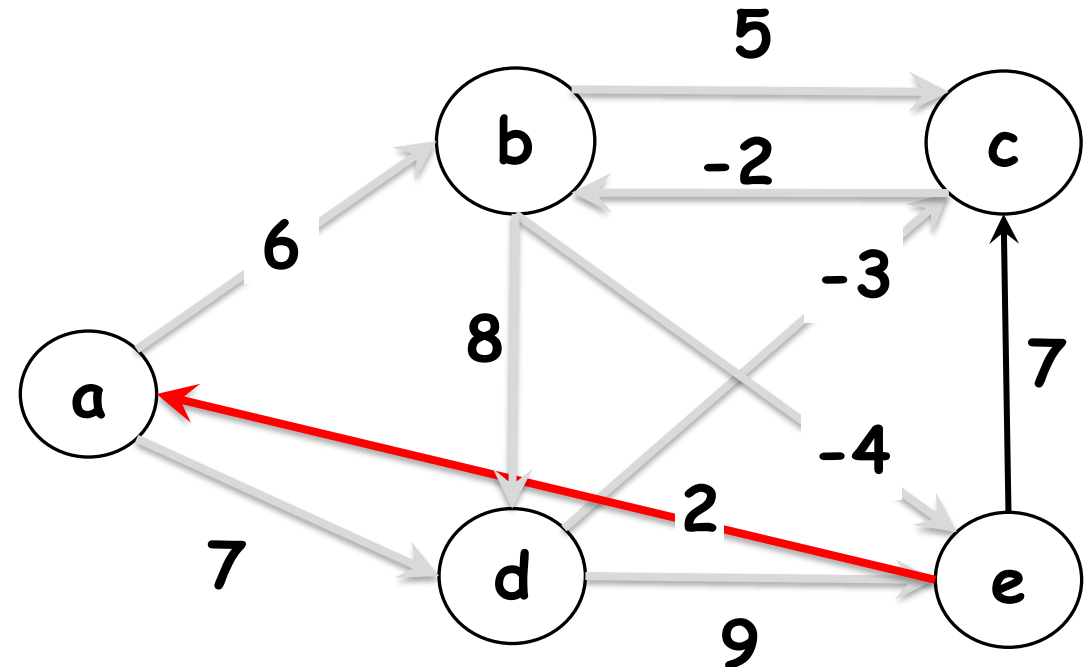
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



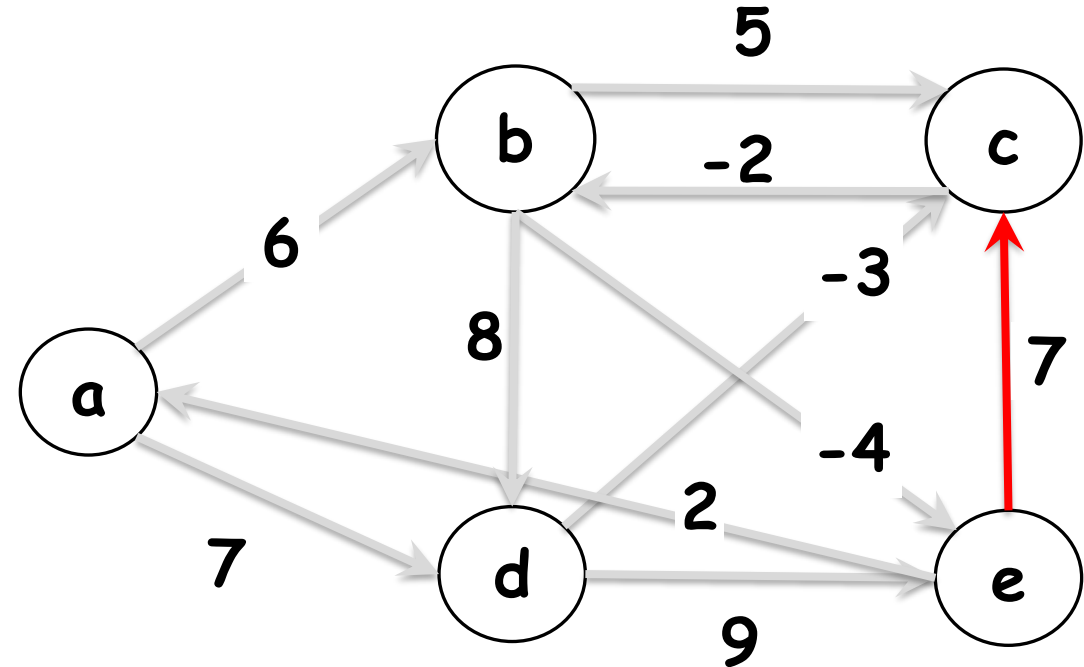
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 1$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



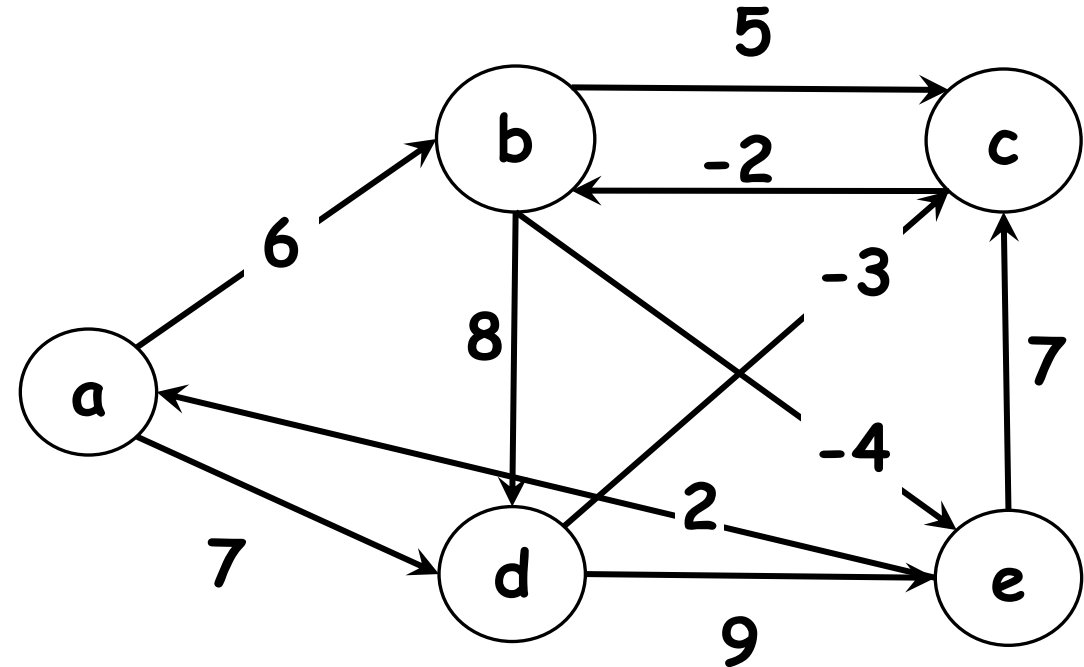
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

i = 2

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



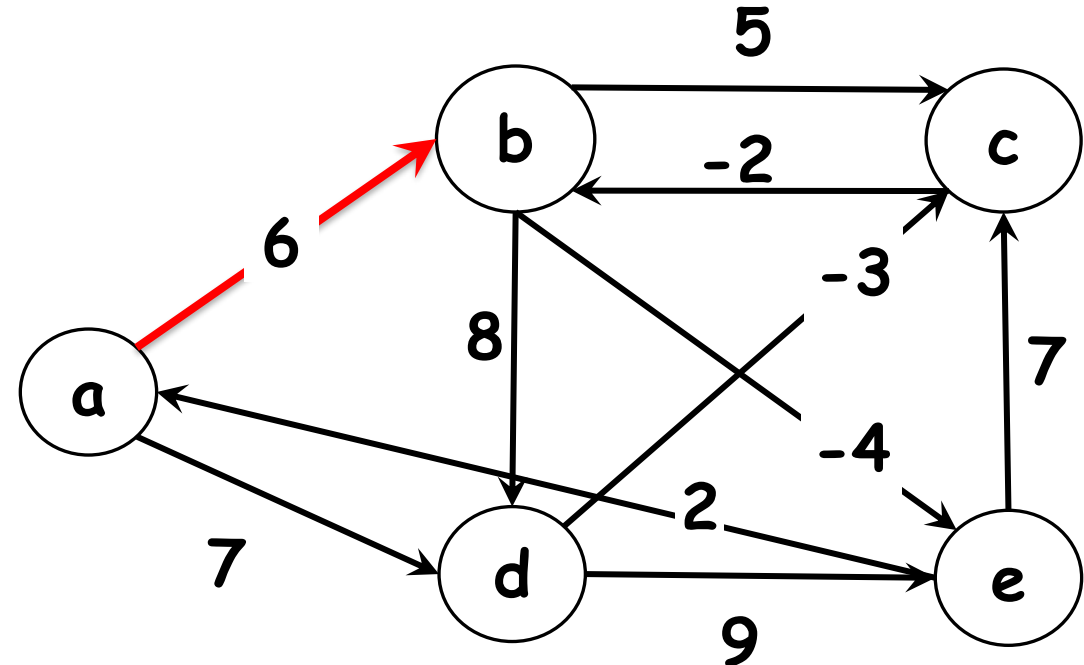
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if  $\text{key}(v) > \text{key}(u) + w(u, v)$  :  
       $\text{key}(v) = \text{key}(u) + w(u, v)$   
       $p(v) = u$ 
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



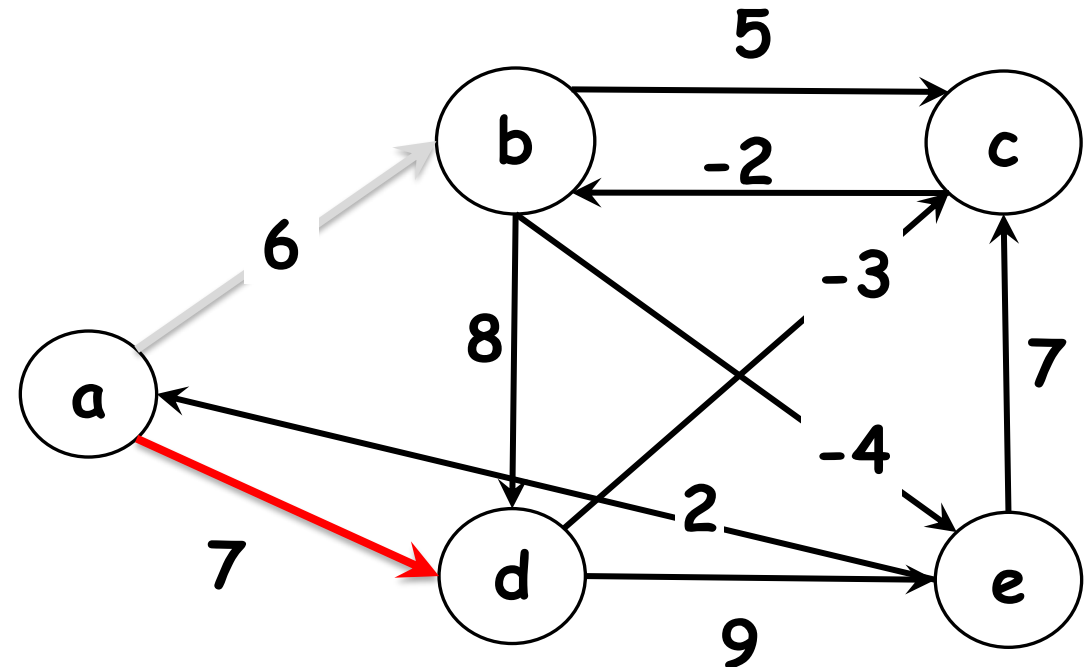
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



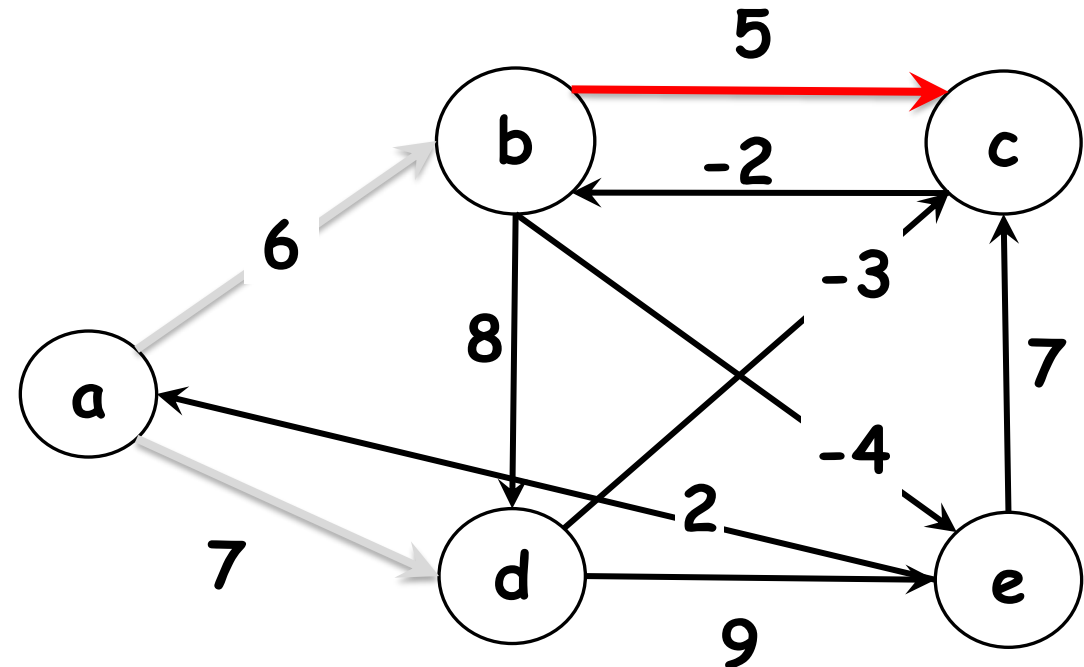
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



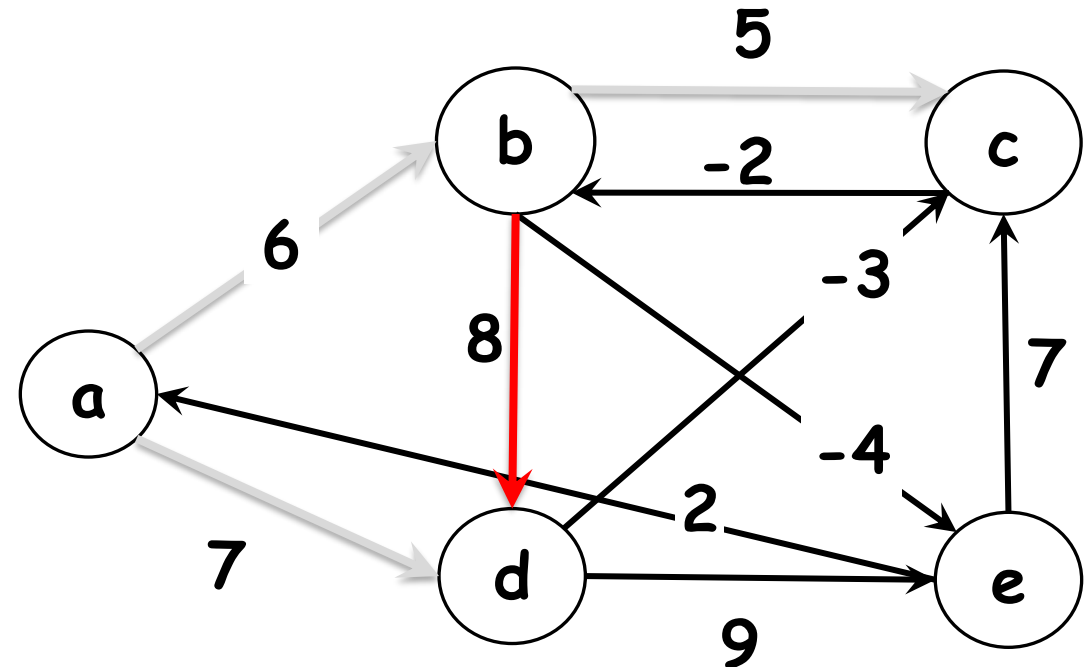
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



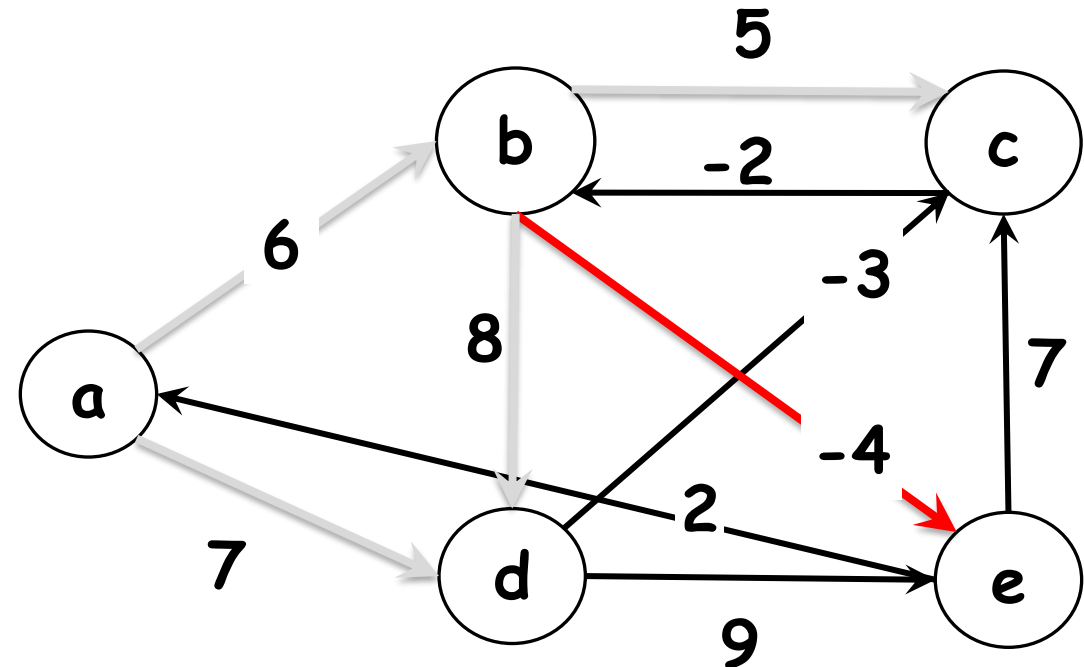
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	6	a
c	4	d
d	7	a
e	2	b



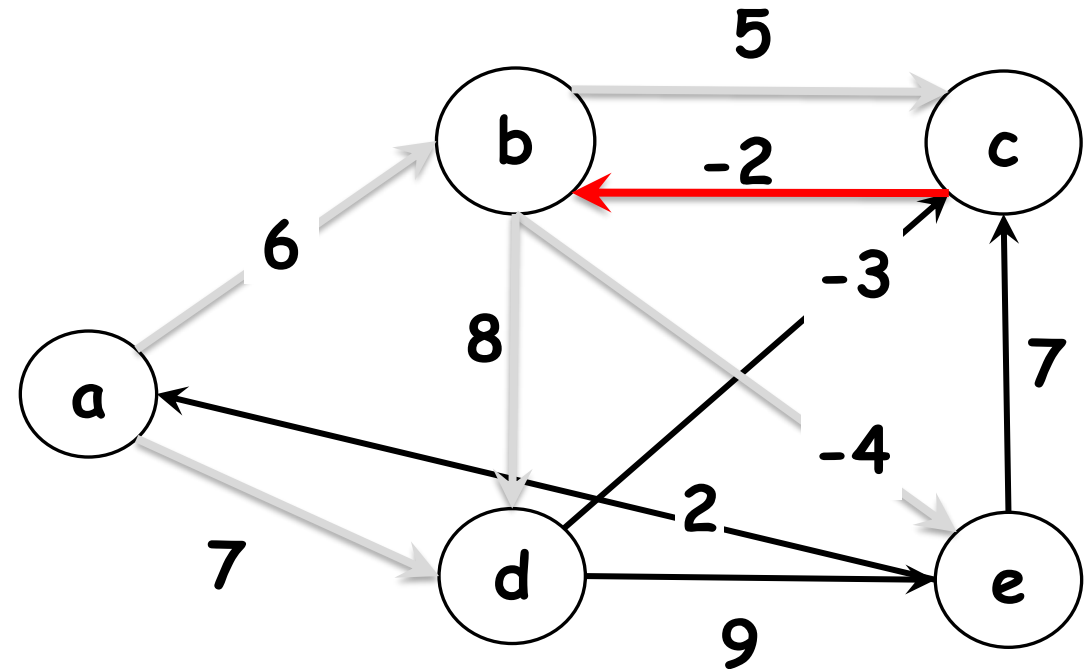
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e),$
 $(c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	2	c
c	4	d
d	7	a
e	2	b



Bellman-Ford Algorithm

```

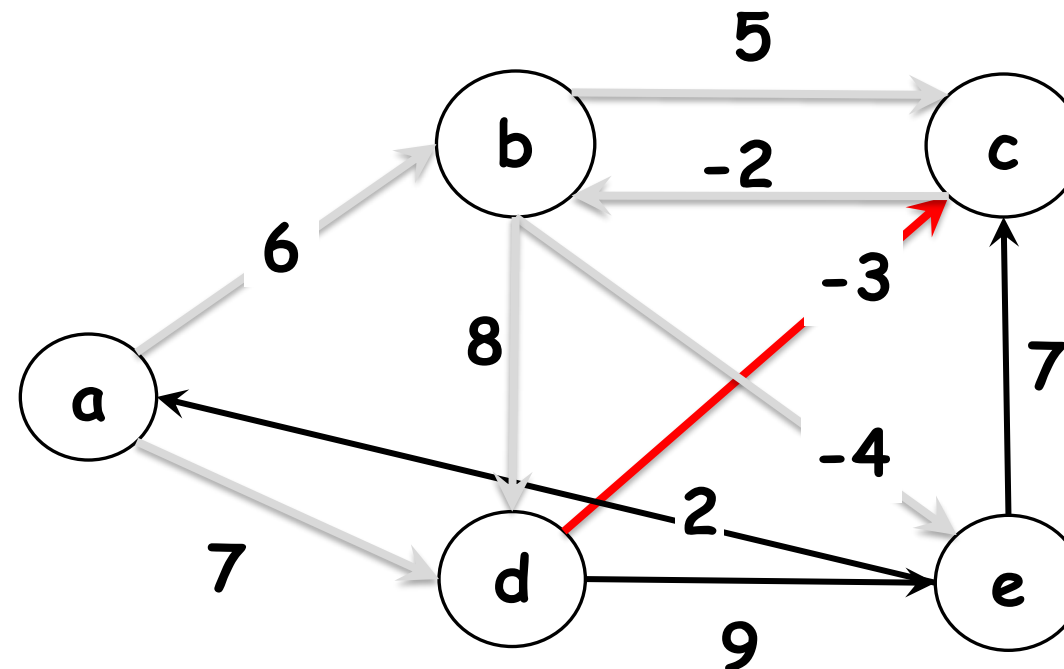
for  $i=1, \dots, n-1$ :
    for each edge  $(u,v)$  in  $E$ :
        if  $\text{key}(v) > \text{key}(u) + w(u, v)$  :
             $\text{key}(v) = \text{key}(u) + w(u, v)$ 
             $p(v) = u$ 

```

$i = 2$

$$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	2	c
c	4	d
d	7	a
e	2	b



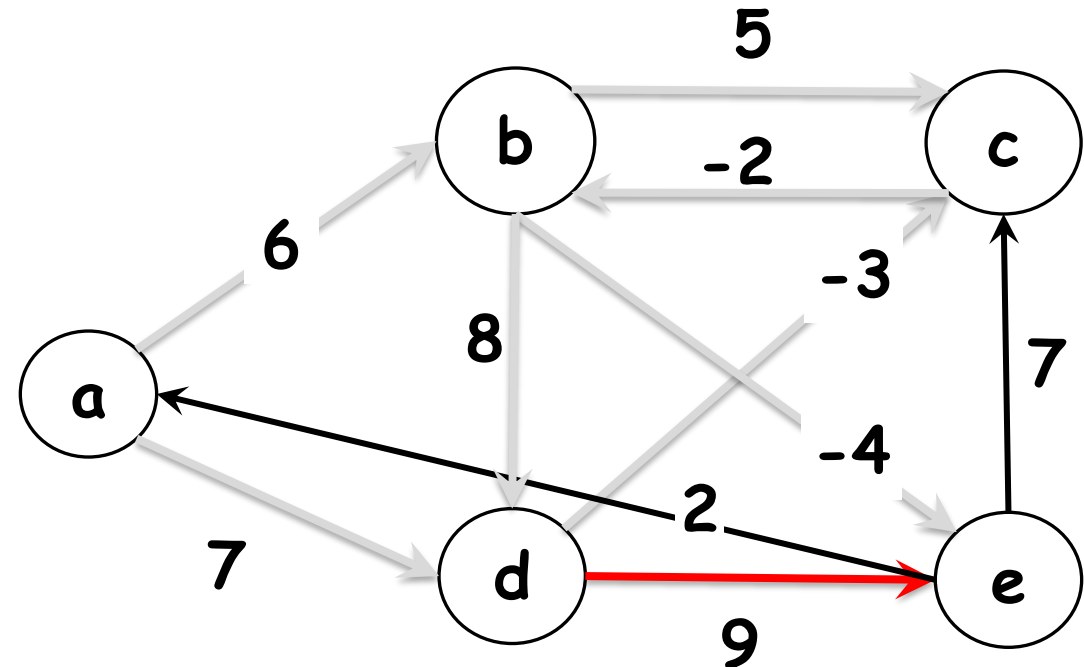
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	2	c
c	4	d
d	7	a
e	2	b



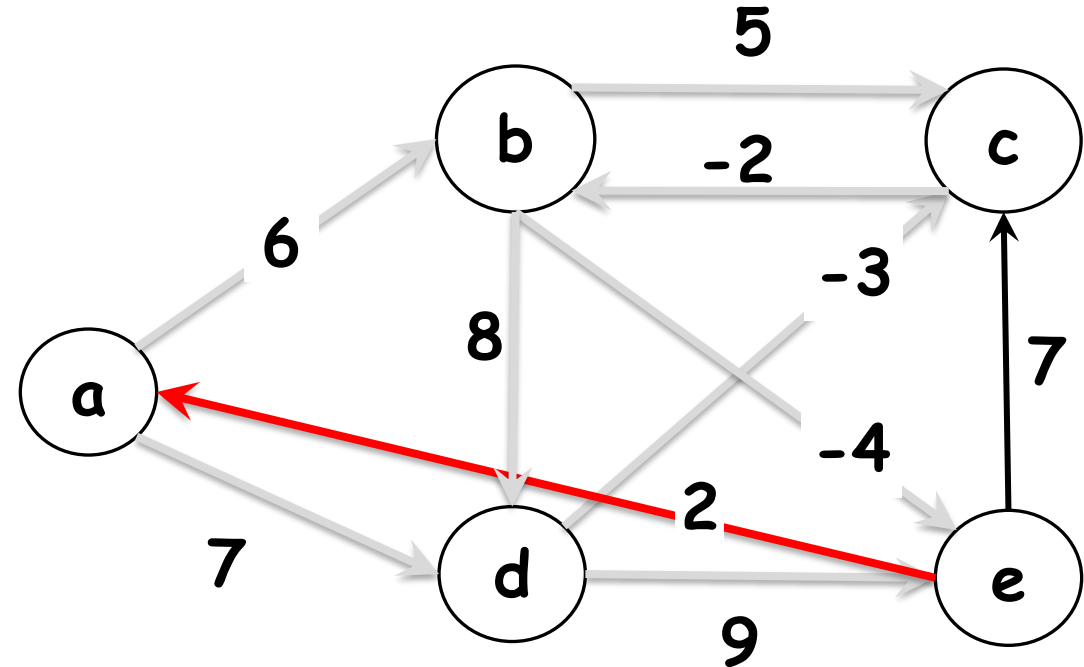
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	2	c
c	4	d
d	7	a
e	2	b



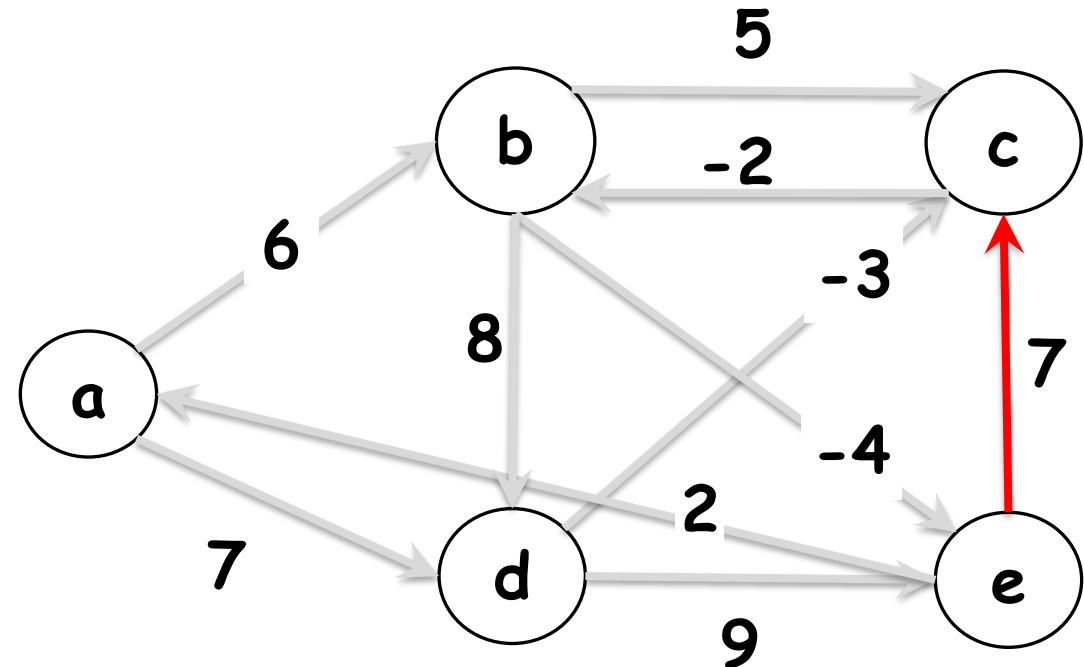
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v) :  
      key(v) = key(u) + w(u, v)  
      p(v) = u
```

$i = 2$

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	2	c
c	4	d
d	7	a
e	2	b



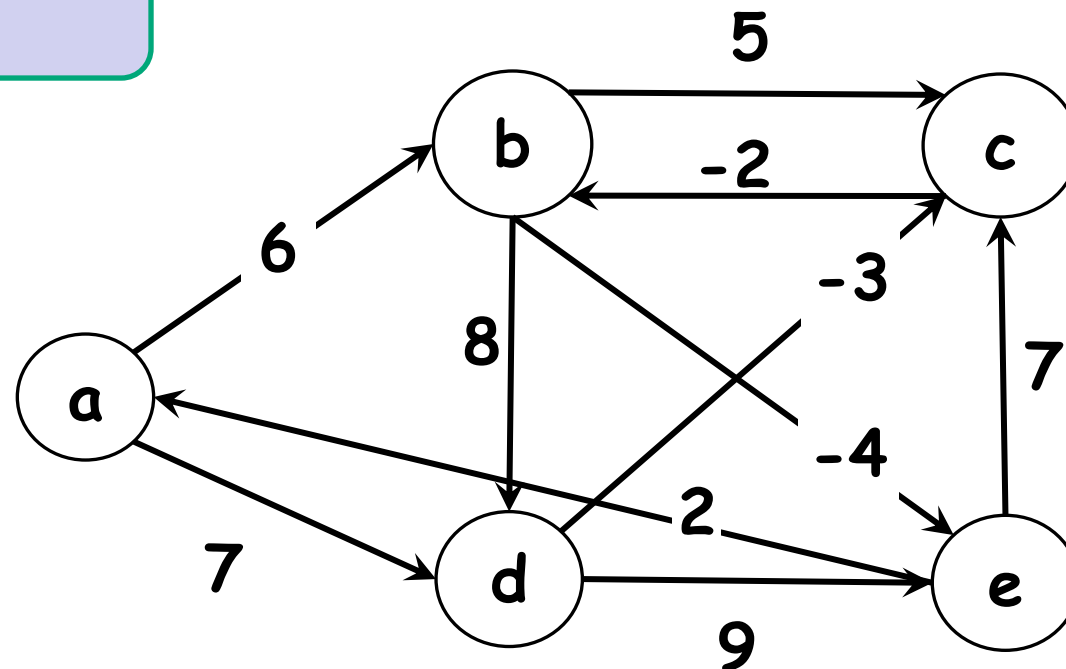
Bellman-Ford Algorithm

```
for i=1,...,n-1:  
  for each edge (u,v) in E:  
    if key(v) > key(u) + w(u, v):
```

Then, i=3,4

$E = \{(a,b), (a,d), (b,c), (b,d), (b,e), (c,b), (d,c), (d,e), (e,a), (e,c)\}$

Vertex	Shortest Distance from a	Previous vertex
a	0	-
b	2	c
c	4	d
d	7	a
e	-2	b



Bellman-Ford algorithm

Bellman-Ford($G = (V, E)$, starting vertex s):

$key(v) = \infty, \forall v \in V$

$key(s) = 0$

$p(v) = \text{NULL}, \forall v \in V$

for $i=1, \dots, n-1$:

 for each edge (u, v) in E :

 if $key(v) > key(u) + w(u, v)$:

$key(v) = key(u) + w(u, v)$

$p(v) = u$

return key, p

$O(nm)$

Summary: shortest path

- BFS:

- (+) $O(n+m)$
- (-) only unweighted graphs

- Dijkstra's algorithm:

- (+) weighted graphs
- (+) $O(m \log n)$ if you implement it right.
- (-) no negative edge weights

- The Bellman-Ford algorithm:

- (+) weighted graphs, even with negative weights
- (-) $O(nm)$

Learning Outcome

- Understand what Minimum Spanning Tree is
 - Able to apply Kruskal's algorithm to find minimum spanning tree
 - Able to apply Prim's algorithm to find minimum spanning tree
- Understand what Single-source shortest path is
 - Able to apply Dijkstra algorithm to solve shortest path problem
 - Able to apply Bellman-Ford algorithm to solve shortest path problem